

Agent Kung-fu

从提示词到第一性原理的十层智能体工程

史春奇 (Chunqi SHI)

2026-09-27

目录

1 十层路线图	6
1.1 欢迎入门	6
1.2 十层总览	7
1.3 十层架构展开：控制对象如何逐层迁移	8
1.4 本书怎么读	18
1.5 这本书不承诺什么	19
2 导论：智能体升级，本质是控制权迁移	20
2.1 模型能力与系统能力不是一回事	20
2.2 十层路线图：两大层域与七段演进	20
2.3 在未知地图上行动：局部智能为什么需要全局控制	21
2.4 从提示模型到治理生命周期：Anthropic 指导建议的逐年迁移	24
2.5 AGI 的典型演进步骤与本书的位置	27
2.6 三条智能体路线如何汇入 AGI 议题	29
2.7 两条平行的流水线	30
2.8 离线模式不是假数据模式	31
2.9 一条贯穿全书的问题	31
2.10 阅读公约	31
3 环境与复现	32
3.1 目标	32
3.2 1. Python 项目	32
3.3 2. Quarto 与 PDF	32
3.4 3. Ollama 实时路径	33
3.5 4. 三种入口，各司其职	33
3.6 检查	33
4 Sakila：同一道题，十种控制面	35
4.1 数据来源与安全边界	35
4.2 数据概况	35
4.3 贯穿全书的基线查询	36
4.4 为什么固定问题很重要	36
4.5 先验证守卫	36
5 实验方法：怎样公平比较十层	38
5.1 研究问题	38
5.2 控制变量	38
5.3 两条实验轨道	38
5.4 观察指标	39
5.5 通过标准	40
5.6 当前验证覆盖与空白	40
5.7 读结果的方法	40

I	第一部 从提示到工具：给智能体一套可控招式	41
6	Level 1: 让提示词指挥流程	42
6.1	能力验收实验	42
6.2	从问答到任务：上一层为什么不够	42
6.3	心智模型	45
6.4	实现路线图	45
6.5	问题贯穿：运行 Level 1	45
6.6	SQL 守卫拒绝演练	47
6.7	从评测框架看 Level 1 的场景局限	47
6.8	何时停在这一层	49
6.9	升级门槛	49
6.10	章末练习	49
7	Level 2: 把能力从文本搬进代码	51
7.1	能力验收实验	51
7.2	从“请这样做”到“你只能这样做”	51
7.3	SkillRegistry 是权限目录	52
7.4	运行代码拥有的 skill	52
7.5	离线与实时的真实差异	53
7.6	OpenClaw：通用智能体的工程突破，而非通用智能证明	53
7.7	本层仍然不能保证什么	55
7.8	何时停在这一层	55
7.9	升级门槛	55
7.10	章末练习	56
II	第二部 从协议到飞轮：让协作可追踪、可裁决、可纠错	57
8	Level 3: 让边界可以被追踪	58
8.1	能力验收实验	58
8.2	工具调用之外的问题	58
8.3	一次请求的时序	59
8.4	Harness：稳定性不是模型属性，而是系统产物	59
8.5	追踪一次离线运行	62
8.6	Codex workers 的真实边界	64
8.7	本层新增了什么，也没新增什么	65
8.8	何时停在这一层	66
8.9	章末练习	66
9	Level 4: 用 Harness 封住智能体行为	67
9.1	能力验收实验	67
9.2	问题：可追踪不等于可控制	67
9.3	架构：一个 bundle、两个子智能体、一个执行宿主	67
9.4	Polars skill 是运行输入，不是装饰标签	69
9.5	宿主白名单：模型规划，代码授权	69
9.6	一次 collect 的 Polars 计算	69
9.7	实验：离线教材仍执行真实数据路径	70
9.8	十项评价：不是让生成者给自己打分	71
9.9	实时模式：OmniGenT client 连接 Codex harness	71
9.10	结论与升级门槛	72
9.11	章末练习	72

10 Level 5: 让失败成为下一轮输入	73
10.1 能力验收实验	73
10.2 问题: 怎样探索完整数据空间, 并在目标达成时停下	73
10.3 从单项修复到 PRD 与 SDLC 三层 LOOP	74
10.4 实验: 用 17 轮完成“探索—计算—停止”	76
10.5 实时 loop 怎样工作	77
10.6 评价器决定飞轮方向	78
10.7 结论与升级门槛	78
10.8 章末练习	78
 III 第三部 从数据契约到知识生产: 让系统理解关系并积累经验	 79
11 Level 6: 把规则和关系写进类型与图	80
11.1 能力验收实验	80
11.2 问题: 反馈知道结果差, 却不知道表怎样相连	80
11.3 从技能目录到功能图	81
11.4 实验一: 类型化 manifest	83
11.5 实验二: 从 DDL 恢复关系	84
11.6 Graph、lineage 与 ontology 不相同	86
11.7 确定性 gate 与 DeepEval	86
11.8 结论与升级门槛	87
11.9 章末练习	87
 12 Level 7: 让系统记得昨天	 88
12.1 能力验收实验	88
12.2 问题: 图每次都能重建, 但解释每次都要重来	88
12.3 开源知识生产的三条架构路线	89
12.4 为什么知识驱动的生产正在进入复杂问题求解	90
12.5 实验: 从数据库生成知识快照	91
12.6 为什么 Markdown 是有价值的中间层	93
12.7 “LLM Wiki” 需要准确解释	93
12.8 知识积累的新风险	93
12.9 结论与升级门槛	94
12.10 章末练习	94
 13 Level 8: 从知识记录到知识生产	 95
13.1 能力验收实验	95
13.2 本体工程的开源实现版图	95
13.3 趋势: 从生成文件转向治理知识生产生命周期	97
13.4 认识论重组: 本体不是知识本身, 而是对可说之物的显式承诺	97
13.5 问题: 页面写了关系, 但系统不知道关系的类型	99
13.6 实验: 生成并解析 canonical OWL	99
13.7 Schema graph、ontology 与业务真理	100
13.8 Canonical 与 candidate 必须分层	101
13.9 评价	101
13.10 结论与升级门槛	101
13.11 章末练习	102

IV 第四部 从问题到创新：从关联分析走向可证伪机制	103
14 Level 9：先问能不能识别，再问怎么优化	104
14.1 能力验收实验	104
14.2 问题：门店 2 收入更高，能推出什么	104
14.3 实验：扩大描述性证据，但不越过边界	104
14.4 DAG：把假设变成可争论对象	105
14.5 本层产物的信任等级	106
14.6 从识别缺口到下一步实验	106
14.7 结论与升级门槛	107
14.8 章末练习	107
15 Level 10：从不变量推导可证伪创新	108
15.1 能力验收实验	108
15.2 问题：DAG 说明不知道什么，然后呢	108
15.3 实验：读取第一性原理结构	108
15.4 从机制到可证伪实验	109
15.5 实现事实：Level 10 怎样继承 Level 9	110
15.6 第一性原理与自由头脑风暴的差别	110
15.7 结论：最高层也没有获得更大权限	110
15.8 章末练习	111
16 结论：选择最低但足够的控制层	112
16.1 十层不是排行榜，而是选择矩阵	112
16.2 控制权迁移图	113
16.3 十条工程心法	114
16.4 一条建议学习路线	114
16.5 中英文同步维护规则	114
16.6 术语速查	115
16.7 最后的读者测试	115
参考资料	116

1 十层路线图

1.1 欢迎入门

大模型可以写 SQL，但“会写 SQL”并不等于“拥有一个可靠的智能体系统”。真正决定系统能否进入生产环境的，往往不是模型再聪明一点，而是下面这些问题有没有清楚答案：

- 谁决定下一步动作？
- 模型能调用哪些能力？
- 多个参与者意见冲突时谁裁决？
- 失败怎样进入下一轮，而不是被悄悄忽略？
- 经验怎样沉淀为可复用知识？
- 什么时候只能描述关联，什么时候可以提出因果干预？

本书用一个固定案例回答这些问题：让本地模型 `ornith-1.5:9b` 分析 Sakila 影视租赁数据库。十层架构不是十个换名字的聊天脚本，而是十次**控制权迁移**：从 Prompt 迁移到代码、协议、治理、反馈、数据关系、持久知识、受约束的知识生产、因果问题，最后到可证伪实验。

💡 读完后的能力

你将能够为具体任务选择“最低但足够”的层级，并说明为什么需要升级、升级后新增了什么控制面、还有什么风险没有解决。

1.2 十层总览

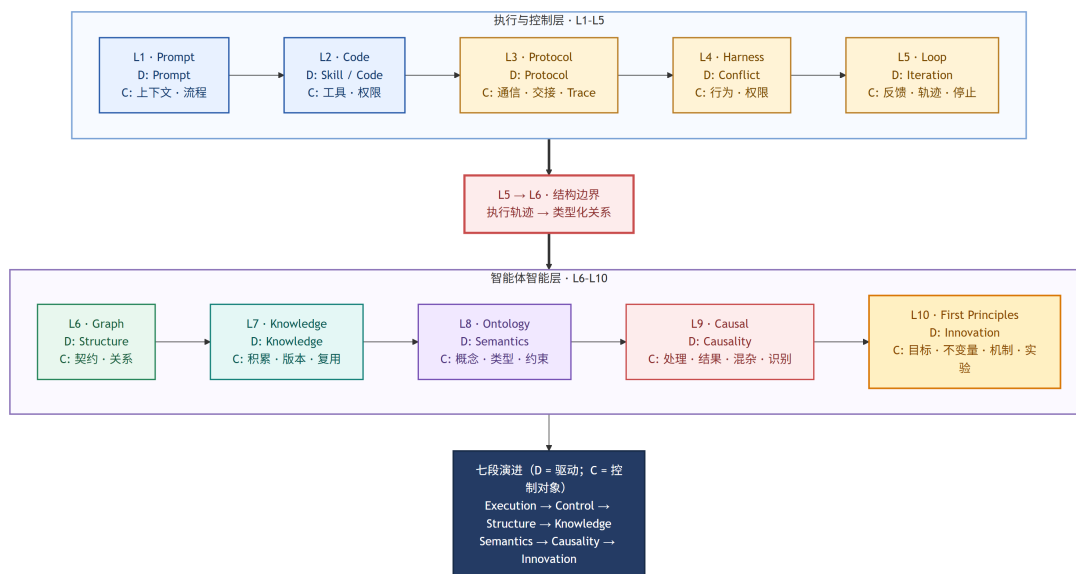


图 1.1: 十层智能体路线图：前五层建立执行与控制，后五层把结构、知识、语义、因果和创新变成显式控制对象

表 1.1: 十层的主导驱动、控制对象与核心技术举例

层级	建议名称	核心驱动	Agent 控制什么?	核心技术举例
L1	Prompt Agent	提示词	Token / 上下文	LiteLLM + LangGraph + Jinja2
L2	Code Agent	技能 / 代码	工具 / 代码	LiteLLM + OpenAI Agents
L3	Protocol Agent	协议	智能体通信	Codex Python SDK + A2A/ MCP/JSON-RPC
L4	Harness Agent	冲突	智能体行为	Harness + OmniGenT
L5	Loop Agent	迭代	智能体轨迹	Ralph Loop
L6	Graph Engineering Agent	结构	数据 / 协同	Harness + Graph Engineering
L7	Knowledge Driven Agent	知识	知识积累	LLM Wiki
L8	Ontology Driven Agent	语义	概念 / 类型 / 关系	Ontology (本体)
L9	Causal Driven Agent	因果	问题 / 原因 / 结果	因果模型
L10	First-Principles Driven Agent	创新	假设 / 定律 / 原理	第一性原理

表 表 1.1 给出本书采用的简明分类，路线图节点中的 C 则把同一控制对象展开为可实现的子对象；两者不是两套相互竞争的层级定义。需要引用某层的标准名称、主导驱动或控制对象时，以表格为准。

1.3 十层架构展开：控制对象如何逐层迁移

下面十张图把参考草图改写为可审计的控制架构，而不是逐像素复刻。蓝色表示输入或调用端，金色表示控制机制，红色表示校验与决策门，绿色表示数据、证据或产出，紫色表示知识与语义资产，灰色表示外部边界。箭头表示显式契约或证据流，不表示产品已经接入；除非对应层的可执行 demo 明确实例化并验证某项集成，本节出现的产品名一律只作架构举例。

1.3.1 L1 Prompt Agent：让提示词指挥固定流程

L1 的 Prompt 承载任务语义，模型产生候选；真正掌握执行顺序与权限的仍是固定工作流和代码守卫。因此，这一层控制的是上下文与流程，而不是开放式行动。

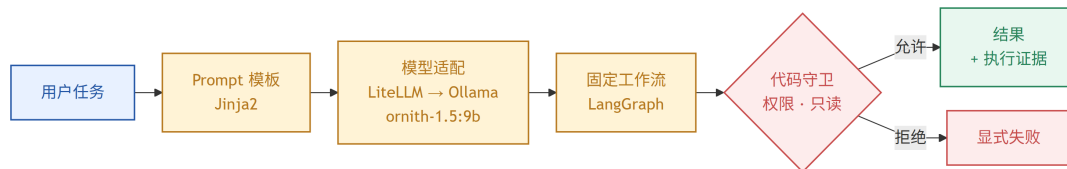


图 1.2: L1 Prompt Agent：提示词承载任务语义，固定工作流与代码守卫掌握执行

1.3.2 L2 Code Agent：把能力从文本搬进代码

L2 用 SDK 构造角色化 Agent，并以代码化 Skill 明确输入、输出、权限和错误。Python、Search 与 SQL 是授权工具的架构示例；三条分支还不是跨 Agent 协议协作。

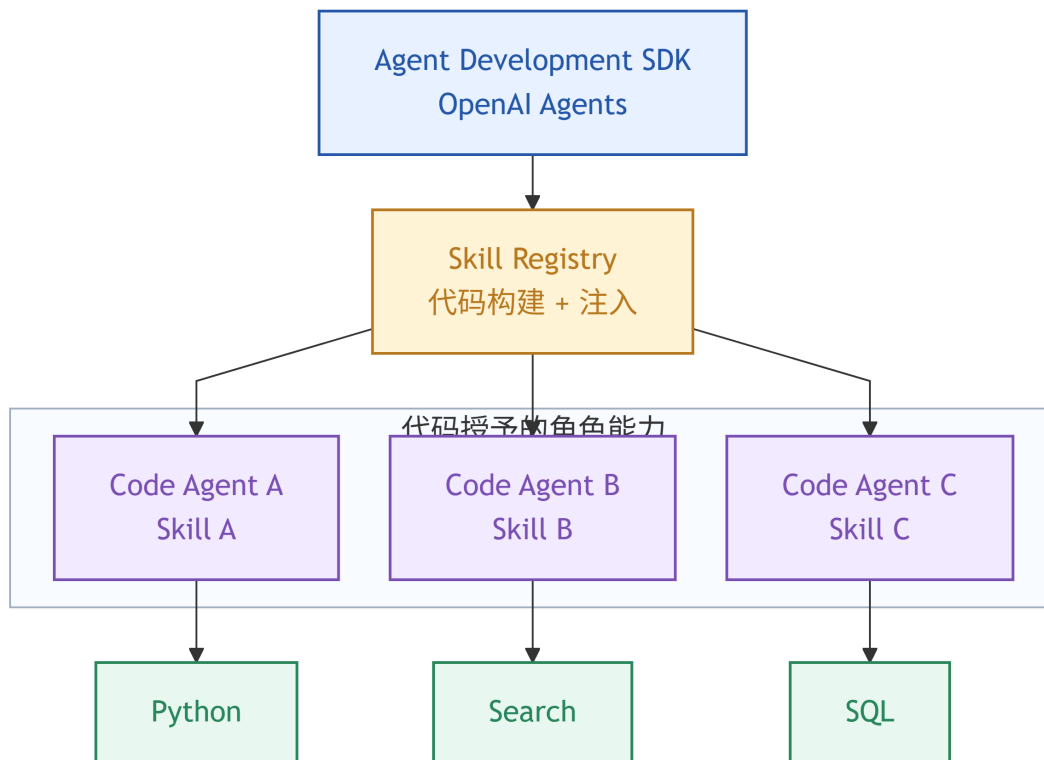


图 1.3: L2 Code Agent：开发 SDK 将代码化 Skill 注入角色化 Agent，并映射到显式授权工具

1.3.3 L3 Protocol Agent：让动态 Codex 工作线程通过协议协作

L3 不再使用 OpenAI Agents SDK 作为运行时。agent.md 声明 Reader 与 Calculator 两个具名 worker，Pydantic 先把声明转成经过验证的合约，再由 Codex worker factory 通过 Codex Python SDK 为每个 worker 创建一个线程。线程均为临时、只读并拒绝全部审批请求；它们经本地 app-server JSON-RPC 产生结构化动作，而不是直接执行任意代码。worker 名称保存在宿主 Trace 中，因为 ephemeral thread 不允许更新名称等元数据。宿主白名单只允许读取数据或调用 Polars 计算，随后把证据写入 MCP 事件。A2A 委派、JSON-RPC turn 与 MCP 结果共享同一个 correlation ID，因而可在一条 Trace 中复原。

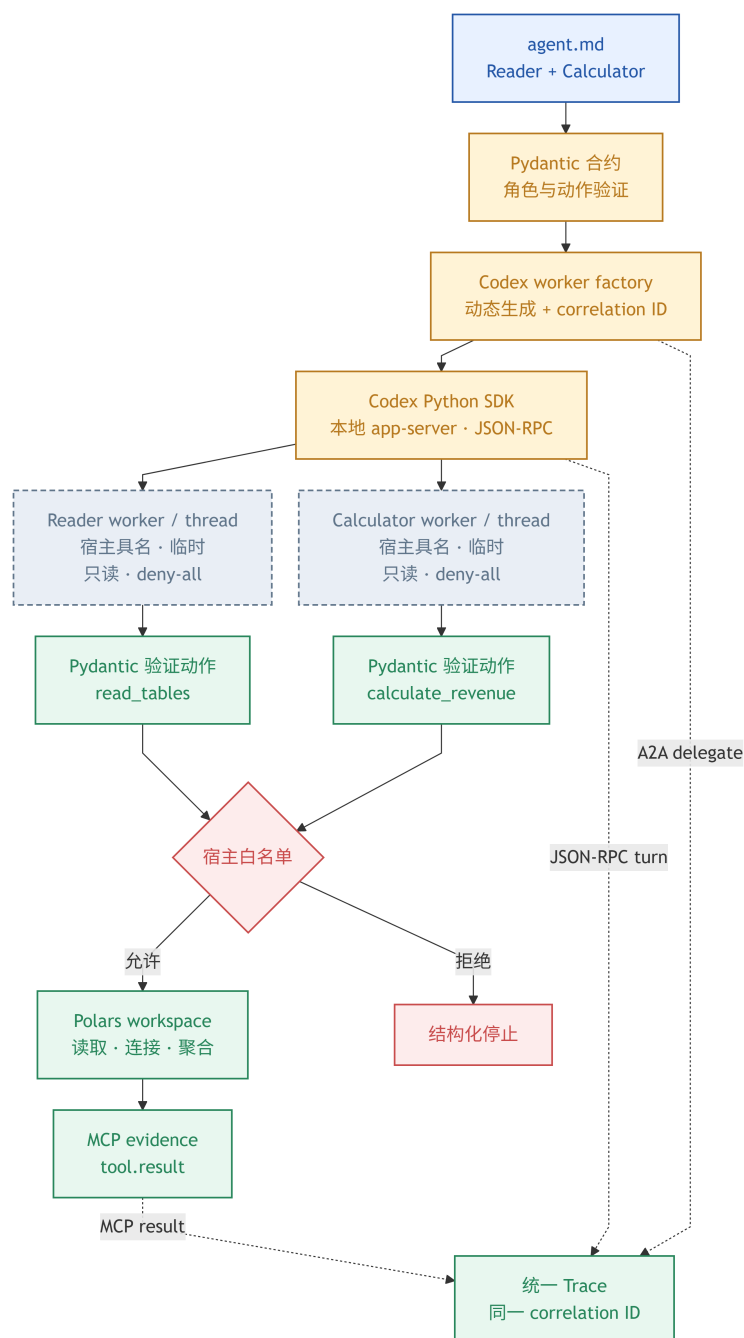


图 1.4: L3 Protocol Agent：agent.md 动态生成两个受限 Codex 工作线程，宿主白名单执行动作，并以 A2A、JSON-RPC 与 MCP 形成可关联 Trace

1.3.4 L4 Harness Agent：把行为冲突封进执行边界

L4 用 OmniGenT bundle 固定生成两个 Codex 子智能体：Reader 先提出读表动作，Calculator 再提出收入计算动作。root 与两个子配置声明 sandbox: auto、空 write_paths、allow_network: false 和 start_in_scratch: true，OmniGenT 0.11 的 Codex executor 使用 approvalPolicy=never。这些设置是纵深防御；Windows Job Object 只隔离进程树与资源，并不强制文件系统或网络策略。真正可跨平台验证的硬边界仍是模型只提出类型化动作、宿主只执行固定白名单。宿主再核对委派证据、动作数量、顺序、参数与 skill digest，最后用十项确定性检查和 SQL baseline 验收。

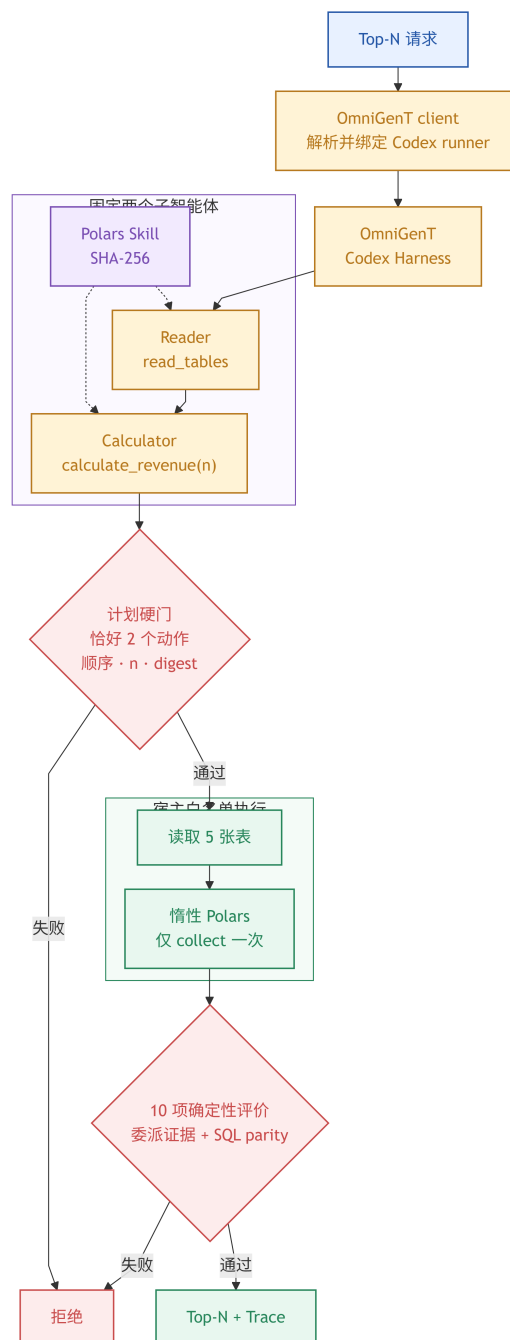


图 1.5: L4 Harness Agent：OmniGenT 生成两个 Codex 子智能体，宿主白名单执行并独立评价

1.3.5 L5 Loop Agent：让失败成为下一轮输入

L5 包含两个嵌套循环：验证失败携带证据回到 Execute；单项需求通过后，如果 PRD 尚未完成，则下一项工作回到 Plan。尝试次数按单项需求独立计数；达到上限时，系统把证据和未满足要求升级给人工或负责人，结构化停止并保留 PRD 的未完成状态。

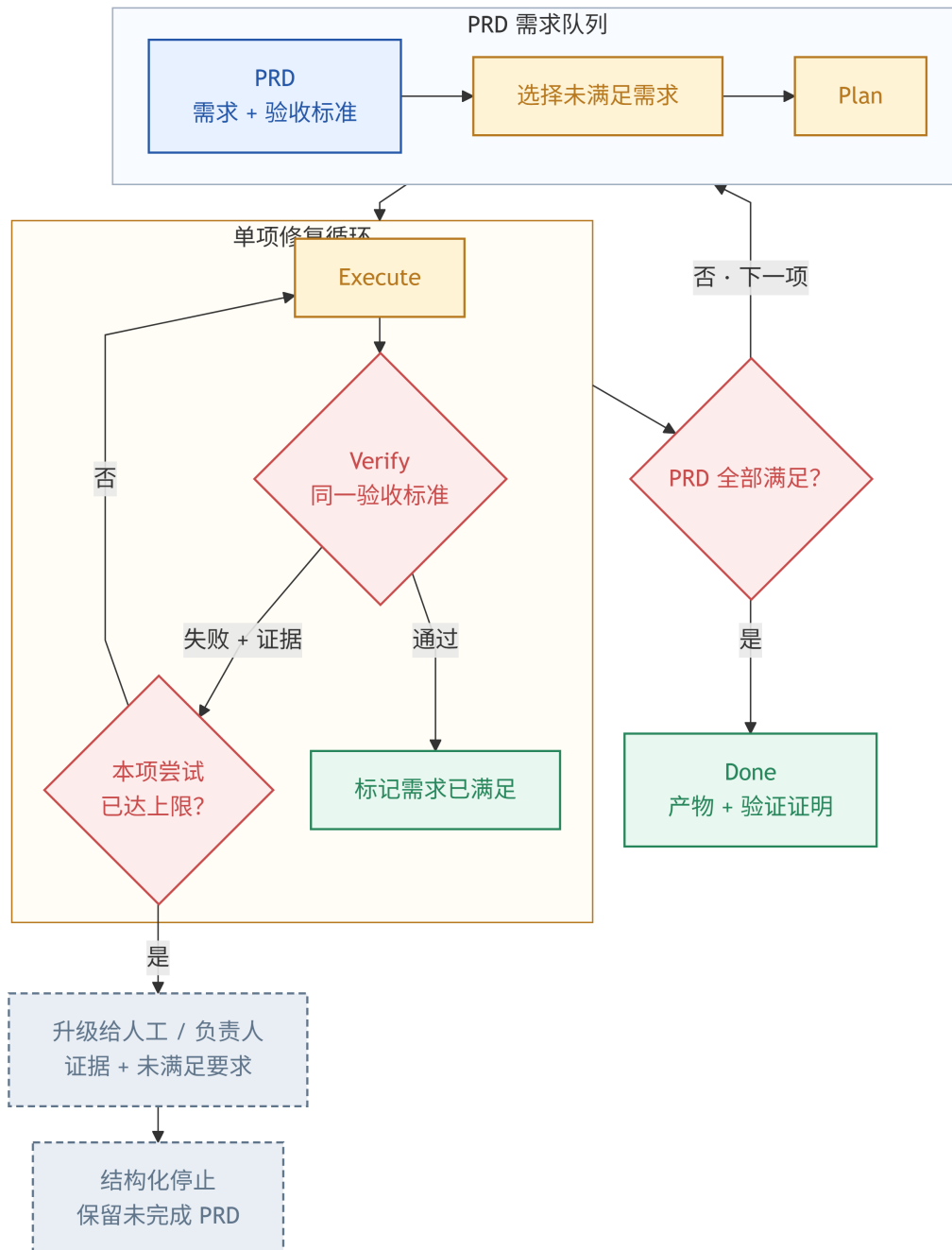


图 1.6: L5 Loop Agent：PRD 驱动需求循环，验证失败驱动局部修复循环

1.3.6 L6 Graph Engineering Agent：把协同写进关系图

L6 的关键不是“把流程画成图”，而是让节点依赖、交换协议、数据契约与质量门成为可执行关系。多个工程 Loop 因此围绕同一类型化图协同，而不再只共享自然语言上下文。

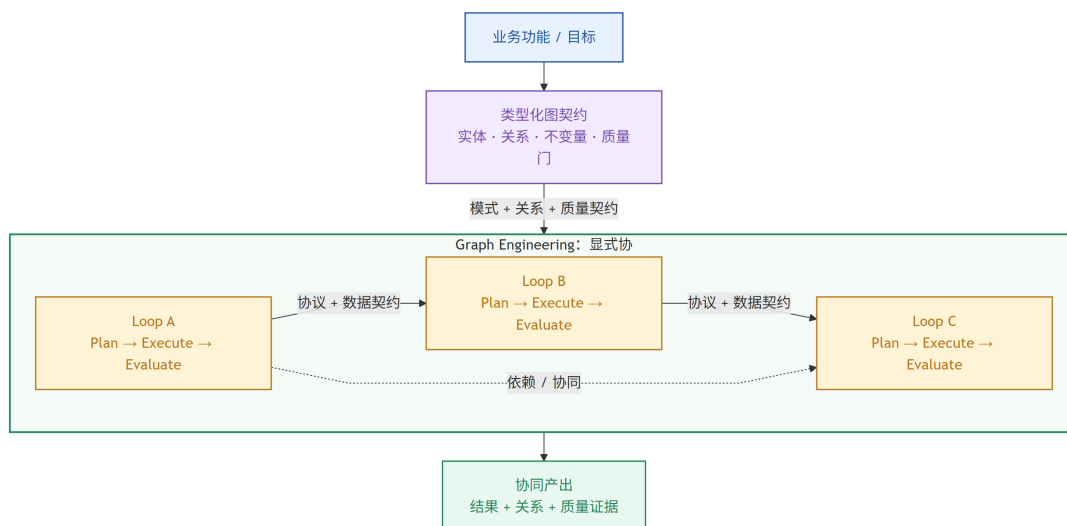


图 1.7: L6 Graph Engineering Agent：用显式关系、协议与数据契约编排多个工程闭环

1.3.7 L7 Knowledge Driven Agent：让系统记得昨天

L7 把跨运行知识显式化。图中将反馈循环按时间展开：上一版本 Wiki 被检索并注入当前协同图，本轮轨迹、决策和评价再生成下一版本 Wiki，避免把“记忆”误解成一次运行中的隐藏状态。

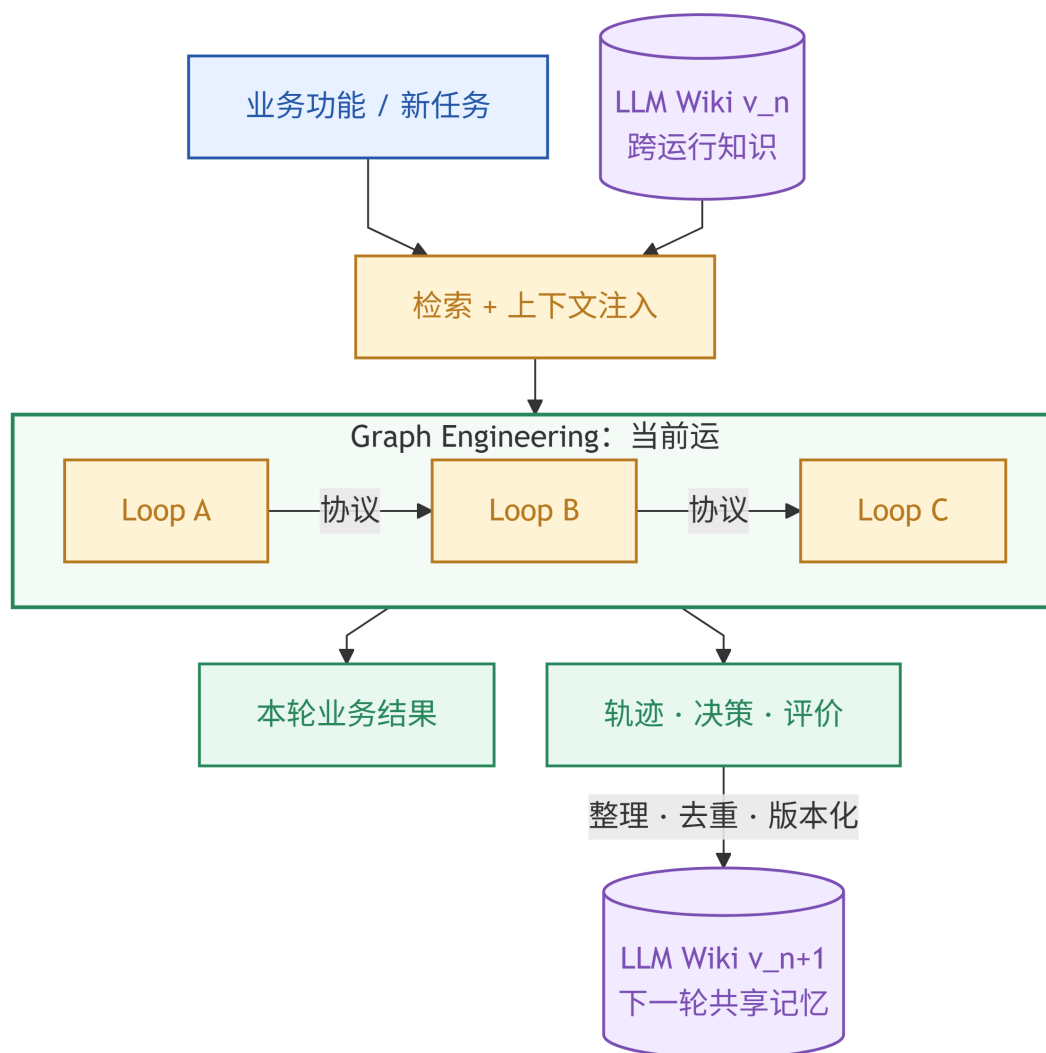


图 1.8: L7 Knowledge Driven Agent：把执行经验沉淀为可检索、可复用的版本化共享记忆

1.3.8 L8 Ontology Driven Agent：从知识记录走向知识生产

L8 把 Ontology 放在 Wiki 与执行图之前：概念、类型、关系和约束先定义可接受的知识形状，再允许候选事实进入下一版本。语义一致性是准入条件，不是事实天然正确的保证。

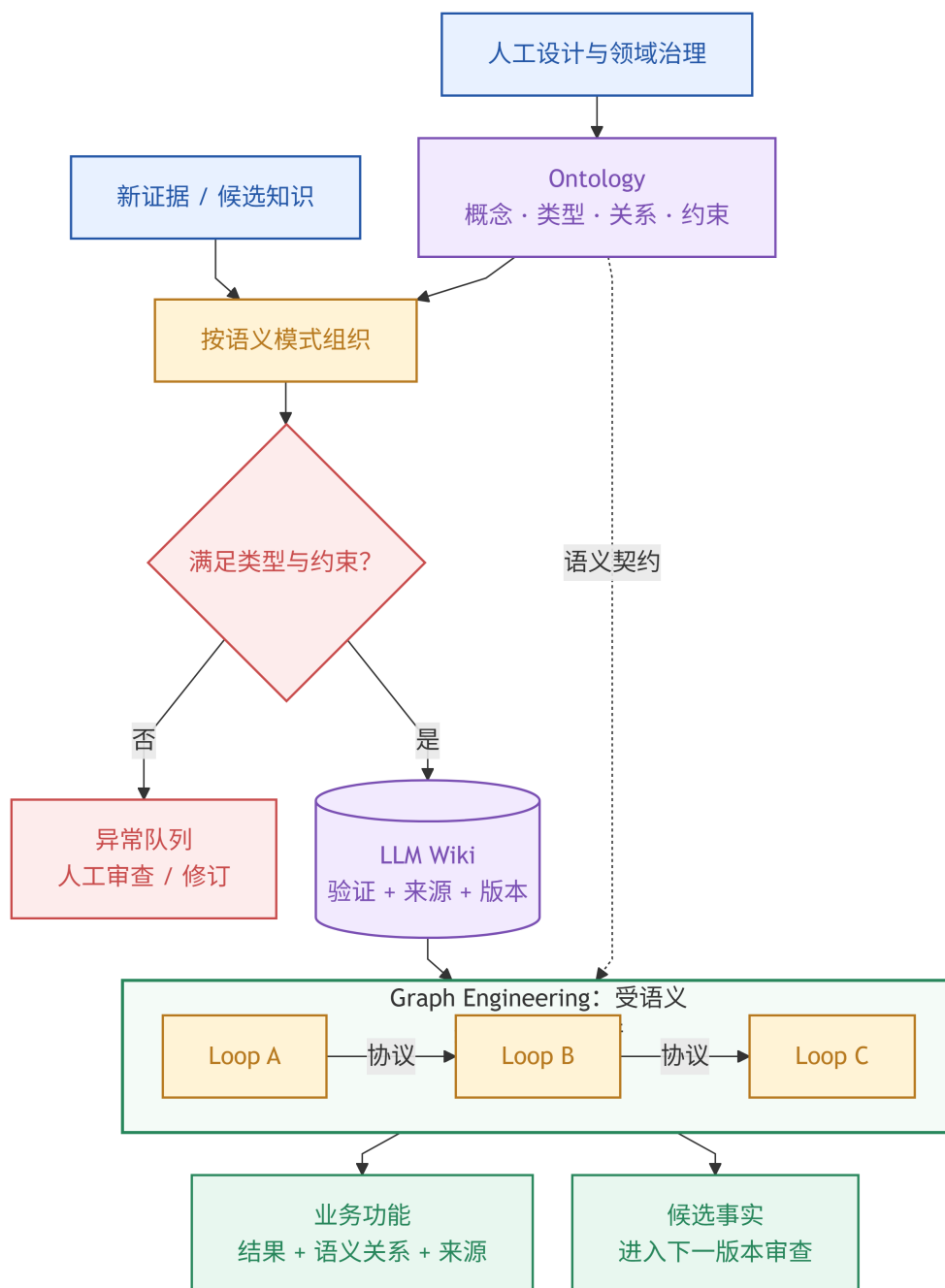


图 1.9: L8 Ontology Driven Agent：以概念、关系与约束指导知识组织和图工程执行

1.3.9 L9 Causal Driven Agent：先识别问题，再谈因果答案

L9 的评价门检查处理、结果、混杂、时间顺序与识别假设，而不只比较答案表面是否正确。图把一次运行的结论封装为版本化证据包，供后续独立运行读取：不可识别时记录缺口与所需新证据；只有假设足够时，系统才输出带条件的因果候选。

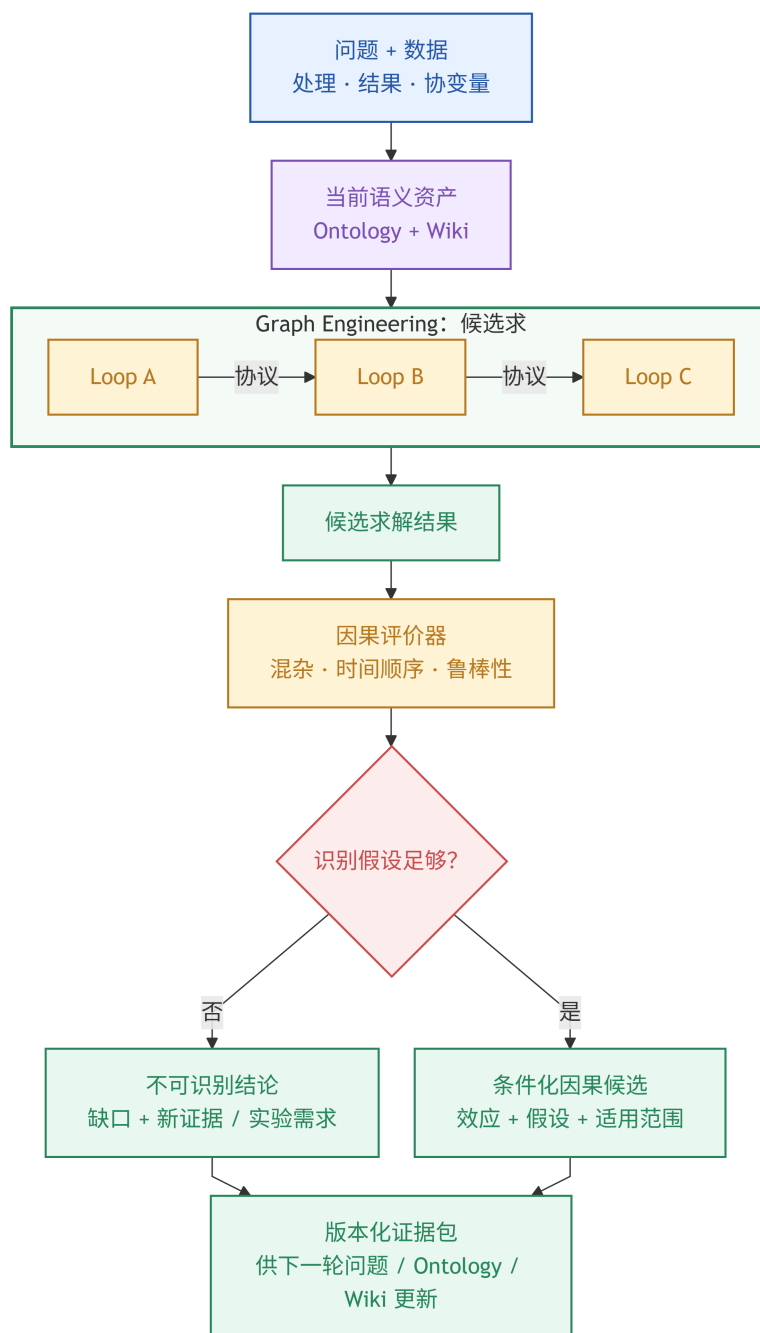


图 1.10: L9 Causal Driven Agent：以因果问题和识别门驱动 Ontology、Wiki 与求解证据共同演化

1.3.10 L10 First-Principles Driven Agent: 重新定义问题并提出可证伪创新

L10 不直接接受既有问题定义。它先重写目标与成功标准，区分事实和惯例，寻找不变量并推导机制，再把候选机制交给 Ontology、Wiki 与 Graph Engineering 执行可证伪实验；失败证据进入下一次“归零重来”。

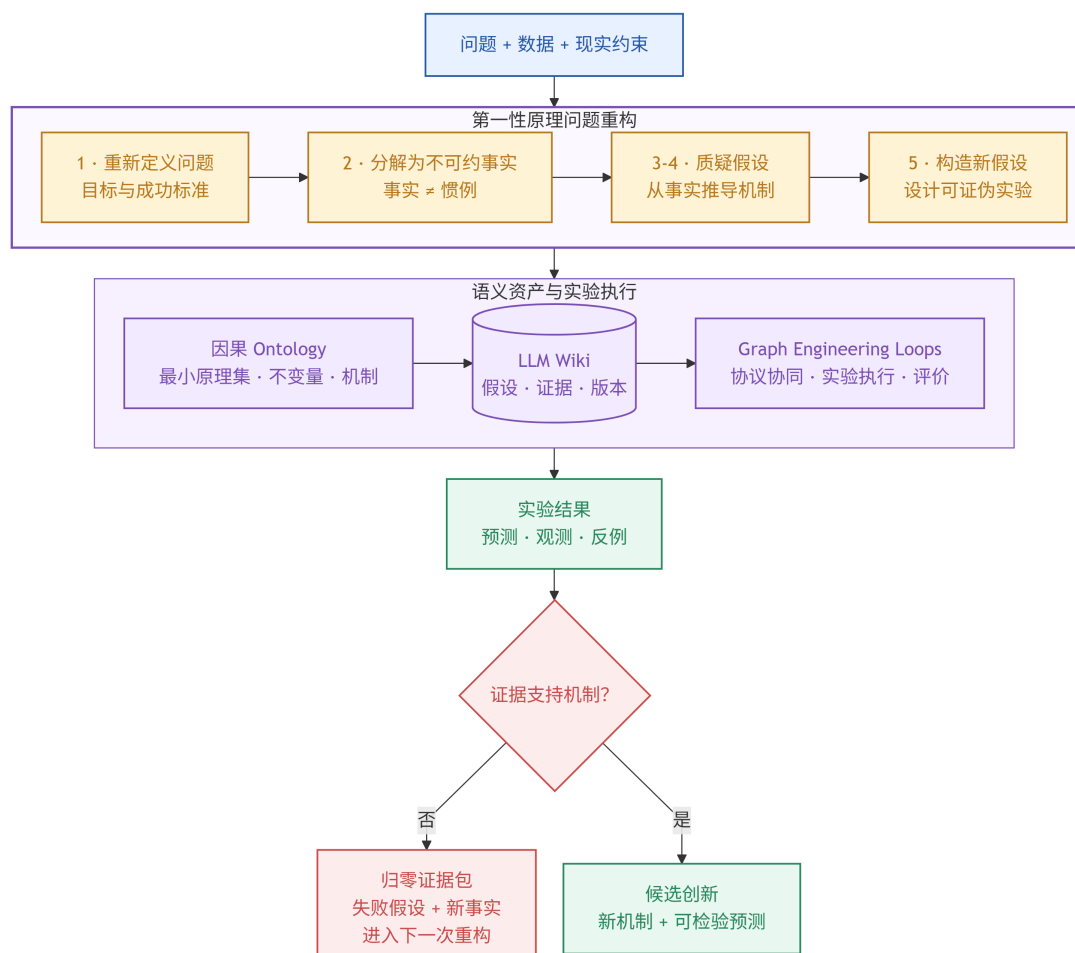


图 1.11: L10 First-Principles Driven Agent: 从不可约事实与不变量出发，以可证伪实验产生候选创新

1.4 本书怎么读

第一次阅读建议从前往后。每一层都用同一个门店问题，因此差异来自控制面，而不是案例变化。第二次阅读可以直接查阅 章节 16 的选择矩阵，再回到目标章节。

每章固定回答五件事：

1. 上一层为什么不够；
2. 本层把哪种控制权搬到了哪里；
3. 仓库里的代码实际上怎样运行；
4. 离线教学路径与真实服务路径有什么差异；
5. 什么条件出现时才值得进入下一层。

1.5 这本书不承诺什么

本书不是生产级安全认证，也不把框架名称当成能力证明。特别是 Level 9–10: Sakila 是观察性交易数据，缺少随机分配与足够的时间变化协变量，因此书中不会声称已经识别任何因果效应。提出 DAG 或实验设计，只是把“还不知道什么”表达得更精确。

2 导论：智能体升级，本质是控制权迁移

2.1 模型能力与系统能力不是一回事

把更强模型接到数据库，通常能提高自然语言理解和 SQL 生成质量；但这不会自动产生权限边界、冲突裁决、停止条件、知识版本或因果识别。模型能力回答“它可能做得多好”，系统能力回答“它被允许怎样做、失败时会发生什么、证据能否复查”。

本项目把后者拆成十层。Level 越高，不表示所有任务都应该使用它。一个只读 SQL 查询助手通常停在 Level 2 就足够；只有当跨进程协作需要审计时才值得进入 Level 3；只有当系统必须跨运行积累知识时才需要 Level 7。

！ 十层不是简单叠罗汉

各章突出一个主导机制。实现会复用前面的数据内核，但不保证每层把前层所有第三方运行时完整叠加。例如 Level 6 的重点是 typed harness 和 schema graph，不是再次启动 OpenAI Agents Runner。

2.2 十层路线图：两大层域与七段演进

如果只按工具名称阅读，十层很容易被误解成十套 Agent Framework 的技术堆叠。更准确的读法是追踪主导控制对象怎样逐层上移：从提示上下文、工具与通信，进入行为、轨迹与反馈，再进入数据关系、知识、语义、因果和基本假设。框架组件会复用，层级之间却由不同的工程问题定义。

本书因此把路线划成两个层域。**L1-L5 是 Agent Execution & Control Layer (智能体执行与控制层)**：L1-L2 先让意图可以被执行，L3-L5 再让跨智能体行动可通信、可裁决、可反馈。**L6-L10 是 Agent Intelligence Layer (智能体智能层)**：系统不再只管理一次执行，而把结构、知识、语义、因果模型和基本假设变成可检查、可变更、可验证的对象。

L5 → L6 是这张路线图最关键的分界。L5 已经能够根据失败重规划轨迹，但它操作的主要对象仍是“这一次怎样做”；L6 开始显式描述数据库 schema 中有哪些类型、约束和关系。从这里开始，智能体才有稳定载体去积累知识、约束概念、提出因果问题，最终检查问题背后的假设。这里的“Agent Intelligence”是一个**系统工程层域名称**，不表示模型智力被分成五级，也不构成达到 AGI 的证据；它同样不是对模型、开发者或组织的能力认证。

这十层进一步压缩成七段演进语义：**Execution** 把意图变成运行，**Control** 让运行有边界和反馈，**Structure** 把数据关系显式化，**Knowledge** 让结果跨运行积累，**Semantics** 让概念和约束一致，**Causality** 把相关性改写为识别问题，**Innovation** 则回到目标、不变量和原理，提出可证伪的候选机制。后续阶段复用与其问题有关的前序证据契约，但不要求重放全部前序 runtime；因此“层数越高”既不等于简单堆栈，也不保证答案必然更正确。

💡 一句话概括十层

L1 让 LLM 按流程工作，L2 让 Agent 通过代码化 Skill 调用获授权工具，L3 让 Agent 通过协议协作，L4 让 harness 裁决竞争 proposal，L5 让 Agent 通过反馈迭代，L6 让 Agent 操作类型化关系，L7 让 Agent 积累知识，L8 让 Agent 用 Ontology 组织可验证的候选语义资产，L9 让 Agent 建模因果问题与识别缺口，L10 让 Agent 从第一性原理重新定义问题并提出可证伪的候选创新机制。

2.3 在未知地图上行动：局部智能为什么需要全局控制

现实世界智能体的核心难题不是再增加一个模型，而是把局部推断接入长期存在的业务语义、权限和责任结构。由此得到本书的控制问题：**现实世界不会一次性把完整状态交给智能体；模型只能从局部观测中压缩线索，而目标、对象、权限、动作与责任必须由更高层结构维持。**

公开材料只支持一个范围较窄但可核验的结论。2024 年的厂商公告称，Claude 3/3.5 通过 Palantir AIP 与 AWS 基础设施进入满足 IL6 要求的美国政府环境；AWS 的说明把用途概括为处理海量数据、识别模式、整理文档和为关键决定提供信息 (Palantir Technologies 2024 年; Lian 和 Krpata 2024 年)。Anthropic 此前只公开确认 Claude 3 Haiku 与 Sonnet 可通过 AWS Marketplace 和 GovCloud 服务政府用户 (Anthropic 2024 年 b)。这些材料证明的是**产品可用性与公开架构**，不是独立的任务效果实验；缺少公开一手证据的具体应用效果和数据库路线判断不作为已核验事实。

2.3.1 合作模式不是“三家公司拼模型”，而是分配控制面

Palantir 的官方文档把 Ontology 描述为连接数据、对象、属性、关系、动作、函数、模型和应用的开发基础，并允许 LLM/agent 从受治理的上下文调用这些资源 (Palantir Technologies 2026 年)。这意味着 **Ontology 与 LLM 不是二选一**：LLM 擅长在模糊语言和局部证据中提出解释、候选关系与下一步；Ontology/AIP 把这些候选对齐到真实业务对象、允许的动作和可追踪工作流；AWS 承载计算、身份、日志与安全边界；人则保留任务目的、风险容忍度、禁区、升级条件和不可逆决定。下面的图是本书依据公开材料做出的架构综合，不是对本书当前 Ollama/Sakila runtime 已集成 Palantir、AWS 或 Claude 的宣称。

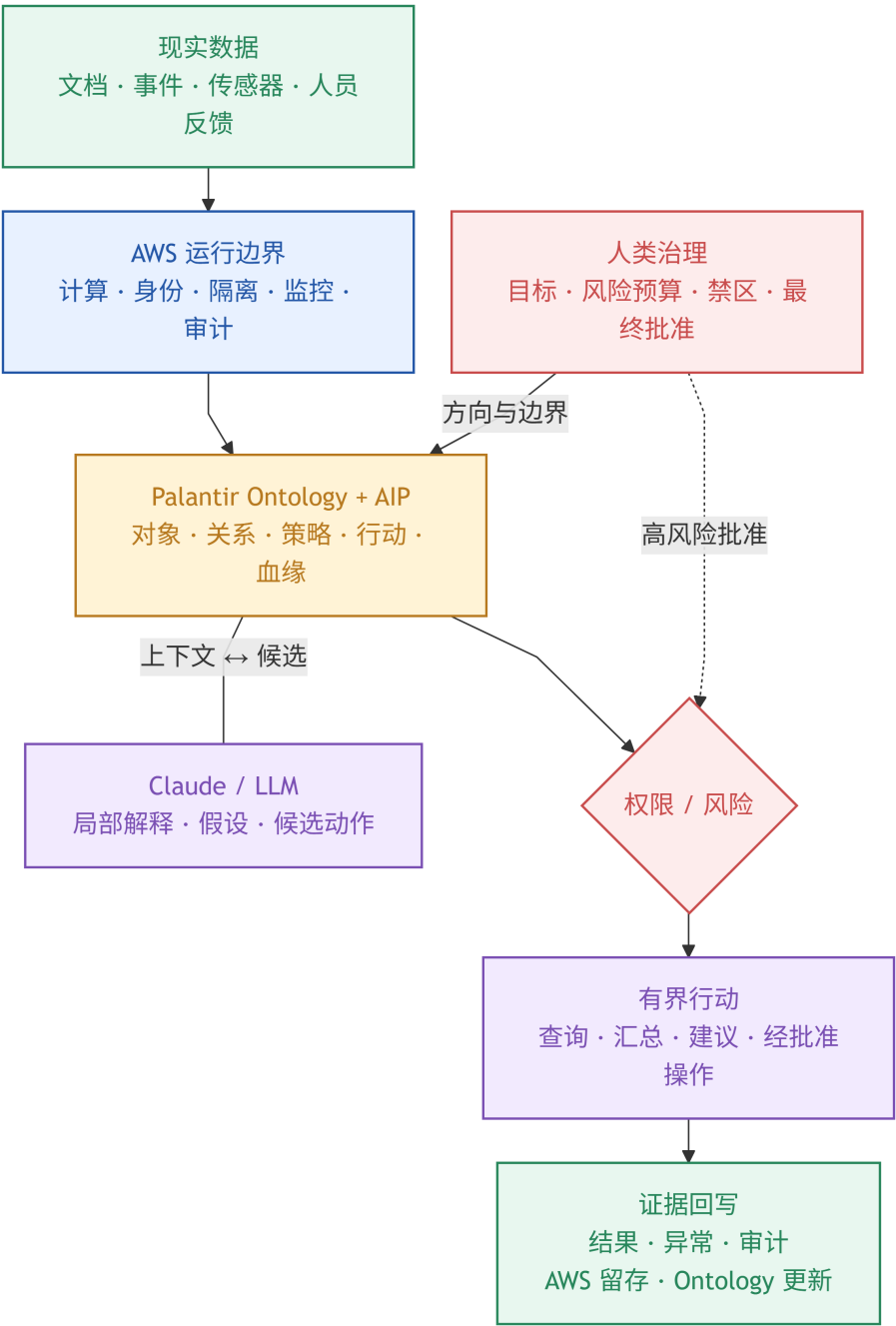


图 2.1: Anthropic–Palantir–AWS 合作的控制面解读：局部推断、业务语义、安全运行与人类责任分层

表 2.1: LLM、Ontology 与运行基础设施的互补控制面

控制层	擅长解决的问题	不应被误认为	必须交给下一层的证据
人类治理	定义任务价值、风险预算、禁止行动、停止与升级条件	对每一步微操，或把责任交给模型	可执行的目标、约束、审批和例外规则
LLM 局部推断	阅读非结构化材料、聚合局部线索、生成假设和候选动作	拥有完整世界状态、稳定权限模型或天然正确性	假设、置信依据、来源和待验证问题

控制层	擅长解决的问题	不应被误认为	必须交给下一层的证据
Ontology / AIP	把语言对齐到对象、关系、动作、策略和工作流	“知识图谱自动保证真相”，或替代模型的开放语义能力	对象版本、关系来源、动作权限和 lineage
AWS 运行边界	提供计算、隔离、身份、监控和审计承载	自动理解业务意义，或独立证明应用效果	调用记录、访问决定、运行指标和异常
现实反馈	暴露行动结果、状态变化和新异常	静态、完整、无噪声的训练集	可回放观测、反例、漂移与结果标签

2.3.2 海量数据的难点不是“装得下”，而是组织后仍能行动

现实系统同时包含表、文档、日志、图像、传感器和人的判断；同一对象可能有多个标识、时间尺度和权限域。数据量增加还会扩大冲突、延迟、重复、缺失和来源不明，而不是自动增加知识。数字孪生综述进一步指出，领域仍面临保真度、数据所有权、技术实现和多个虚拟实体整合等知识缺口 (Jones 等 2020 年)。因此，海量数据组织至少要回答四个连续问题：这条记录指向哪个现实对象？它在什么时间和条件下有效？谁可以据此采取什么动作？动作后出现的新证据怎样更新原判断？

表 2.2: 复杂现实中的数字孪生控制难题

现实复杂性	数据为何难组织	AI 的局部优势	仍需的全局控制
异构与保真度差距	多源标识、粒度、时钟和权限不同；噪声与缺失会随规模扩散	从文本和局部样本中提取候选实体、关系与异常	主数据、来源、时间语义、权限和可验证对象映射
非线性、耦合与漂移	磨损、环境和策略改变使过去关系失效；局部相关不等于机制	快速提出模式、相似案例与待检验解释	版本化状态、物理/业务约束、漂移检测和反事实验证
稀缺标签与罕见事件	高价值失败少见，正确答案昂贵，连孪生本身也需校准	把少量证据压缩成优先调查的假设	专家复核、仿真/测试、置信校准和不确定性升级
实时性与算力约束	全量重算赶不上状态变化，复杂模型又提高时延与成本	在当前视野内选择最有信息量的下一步	预算、超时、降级路径、增量更新和停止条件

2.3.3 “探索游戏地图”是控制论类比，不是厂商能力声明

部分可观测决策研究早已形式化了这种处境：行动者看不到完整状态，只能依据观测维护一个关于世界的“信念状态”，行动后再用新观测修正它 (Kaelbling 等 1998 年)。语言智能体中的 ReAct 把推理和行动交错，Voyager 则展示了在开放环境中通过反馈积累技能的探索结构 (Yao 等 2023 年; Wang 等 2024 年)。三者不能直接证明企业或政府部署效果，但共同说明：复杂现实更像一张带战争迷雾的地图，而不是一次 SQL 查询就能读完的表。

游戏地图类比中，高层指导不是逐步遥控。它定义“要达到什么目标、哪里不可进入、一次探索可以花多少资源、什么时候必须停下或呼叫人”；智能体在边界内选择下一处观

测、提出局部假设、执行可逆动作，再把结果写回地图。**AI 的优势是局部压缩，不是全局全知**。真正的系统升级，是把局部探索权交给智能体，同时把全局目标、稳定关系、风险闸门和最终责任提升到可执行的控制层。

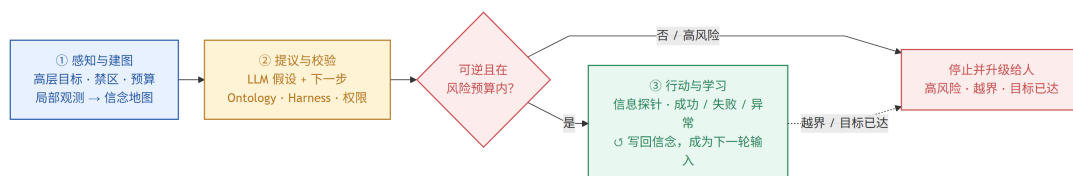


图 2.2: 未知地图上的有界探索：高层给方向，智能体以观测—行动—更新逐步消除不确定性

i 证据边界与本书实现边界

合作公告是供应商的一手架构声明，不是独立 benchmark；POMDP、ReAct、Voyager 和数字孪生综述提供的是问题结构与研究类比，也不等于部署效果证明。以上控制栈是本书的综合分析。当前演示仍使用本地 Ollama、Sakila 与各层 Python harness，并未连接 Palantir、AWS 或 Claude。

2.4 从提示模型到治理生命周期：Anthropic 指导建议的逐年迁移

Anthropic 关于 AI 原生 SDLC 的官方文章强调，代码生成加速后，瓶颈会移到计划、评审、测试、部署和治理 (Claxton 2026 年)。本书据此提炼的关键命题不是“让 AI 接管所有决定”，而是：旧控制目标仍要保留，但执行、证据和强制机制必须随控制权迁移而重写。

这恰好解释了书名中的“控制权迁移”。迁移的不是最终责任，而是四类控制的承载位置：人的隐性经验变成版本化上下文，逐行盯执行变成对计划和风险的闸门，口头规范变成 tests、hooks 与 sandbox，阶段性交接变成可以触发下一步的机器可读产物。人仍拥有意图、风险接受、例外升级和最终批准；智能体只在可观察、可撤销、权限有界的范围内获得执行权。

2.4.1 2023–2026：指导对象从“回答”扩展到“组织运行”

Anthropic 在 2023 年发布 Claude 时，重点是可预测、可引导的助手行为；同期公开的 constitution 把价值原则用于模型自我批评、修订和基于 AI 反馈的训练 (Anthropic 2023 年 b, 2023 年 a)。此时控制主要位于**模型行为与单次交互**：人给出任务，模型生成回答，显式原则约束输出。

2024 年的建议开始把模型放进运行结构。《Building Effective Agents》区分预定义 workflow 与由模型动态决定步骤的 agent，并建议从简单、可组合模式开始；MCP 又把数据和工具连接重写为标准化双向边界 (Anthropic 2024 年 a, 2024 年 c)。控制对象由“怎样写一句提示”扩展为**谁决定下一步、通过什么接口调用什么资源**。

2025 年的变化发生在仓库与 harness。ClaudeCode 的实践建议把 CLAUDE.md、工具 allowlist、计划、测试和 Git 工作流放进同一开发环境；随后 context engineering 把 prompt 扩展为系统指令、工具、MCP、外部数据与消息历史的动态选择，skills 把组织程序知识包装成可发现资源，sandbox 用文件系统与网络边界减少逐次审批，长任务 harness 用进度文件、Git 历史和跨会话交接维持连续性 (Anthropic 2025 年 b, 2025 年 c, 2025 年 d, 2025 年 a; Young 2025 年)。控制开始从人的即时操作外置为版本化仓库知识、执行环境与持久证据。

2026 年的材料进一步把控制提升到生命周期和组织层。Agent eval 指南把任务、trial、grader 与完整 transcript 作为可回归对象；长应用 harness 使用 planner、generator、evaluator 和结构化交接产物；Managed Agents 把 session、harness 与 sandbox 分成可替换的稳定接口；AI 原生 SDLC 则把这些机制贯穿计划、设计、开发、测试、部署和维护 (Anthropic 2026 年 a, 2026 年 b; Rajasekaran 2026 年; Claxton 2026 年)。这不是“自治程度单向增加”，而是**执行权下放与约束、证据、责任同时上移**。

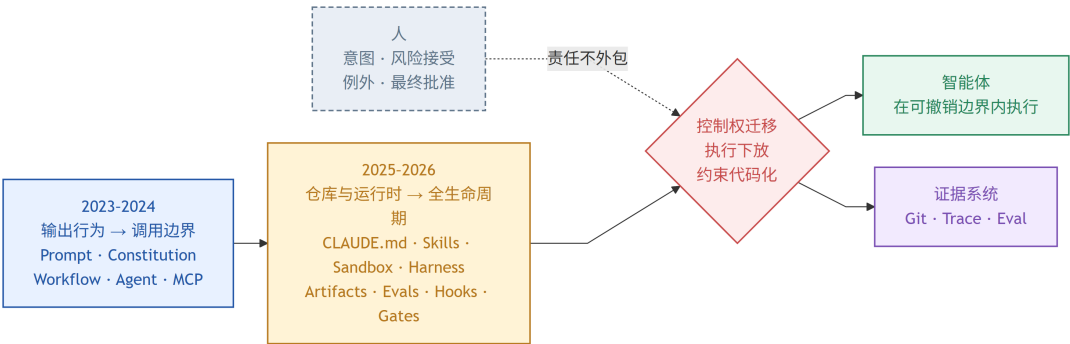


图 2.3: Anthropic 2023–2026 指导重心：控制从模型输出迁移到组织级生命周期

表 2.3: Anthropic 指导建议的逐年控制迁移与证据边界

年份	官方建议主要控制什么	相对上一阶段的关键变化	本书的证据边界
2023	输出行为、可引导性与显式价值原则	控制主要内置于模型训练、system/prompt 和人的逐轮判断	Constitution 表达规范意图，不证明每次行为都符合意图
2024	workflow/agent 的决策分工与 MCP 工具边界	从单轮回答进入多步调用；强调简单模式和标准连接	协议能暴露边界，不能自行提供权限、恢复或正确性
2025	仓库上下文、skills、allowlist、sandbox 与跨会话进度	组织知识和运行限制变成可版本化、可执行的 harness 资产	产品实践是设计证据；其收益依赖任务、模型和环境
2026	eval、session/harness/sandbox 接口和六阶段 SDLC	单个工程师的编码助手扩展为产物驱动的组织运行系统	官方文章是实践指导，不是对所有组织的受控效果实验

2.4.2 产物—闸门循环

AI 原生 SDLC 可以抽象为下面的产物—闸门模型：每个阶段提交一个人可读、机器可执行的版本化产物；被接受的产物触发下一阶段；不变量由确定性机制执行；生产异常重新生成 intent，关闭反馈回路 (Claxton 2026 年)。

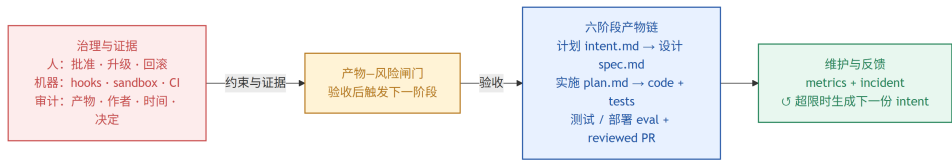


图 2.4: AI 原生 SDLC 的产物—闸门循环：执行权下放，风险责任不下放

表 2.4: 六阶段 SDLC 在本书中的产物、强制机制与责任重构

重写后的阶段	版本化产物与触发	机器侧确定性控制	人保留的责任	本书中的对应关系
计划	intent.md；接受后触发需求与设计	模板、schema、来源和修订历史检查	确认问题、范围、约束与是否立项	Level 1 固定问题和输入契约
设计	spec.md；接受后触发 plan mode	加载品牌、安全、合规和 UX skills，报告冲突	解决政策冲突并批准规格	Levels 2、4 的 skill 与冲突裁决
开发实现	plan.md、代码与测试；计划批准后才允许写入	read-only plan、路径权限、format/lint/secret hooks	审核高风险方案与计划偏离	Levels 3–5 的 trace、policy harness 和 loop
测试	eval task、trial、grader 和 transcript	测试、构建、静态分析、视觉检查与回归门	定义成功标准并校准主观 grader	Levels 5–6 的回归与 DeepEval 边界
部署	带分级发现的 reviewed PR 与部署记录	多层 agent review、CI、审批 hook 和回滚路径	接受受监管/关键代码风险，批准生产变更	Level 4 的权限和确定性 hook
维护	指标、incident 与下一份 intent.md	控制区间检测、诊断触发和只走受控行动路径	处置异常、决定回滚与下一轮优先级	Levels 7–10 的知识、因果问题和实验提案

2.4.3 趋势判断：速度不是常数，控制迁移才是稳定变量

“代码不再是瓶颈”应理解为需要在本组织测量的假设，而不是所有软件工作的普遍事实。一项受控短任务实验报告 Copilot 组完成 JavaScript HTTP server 更快；另一项针对熟悉成熟开源仓库的开发者随机试验却报告早期 2025 AI 工具使完成时间增加 (Peng 等 2023 年; Becker 等 2025 年)。两者任务长度、代码库熟悉度、质量标准和工具形态不同，不能用一个百分比替代另一个。

更稳健的趋势证据来自任务结构。SWE-bench 把真实 GitHub issue、仓库上下文和可执行测试组合成端到端评价，早期结果显示单纯生成代码远不足以解决多数问题 (Jimenez 等 2024 年)；SWE-agent 进一步表明，专门设计的 agent-computer interface 会显著改变导航仓库、编辑文件和运行测试的表现 (J. Yang 等 2024 年)。因此，本书采用的结论不是 “AI

必然让每位工程师更快”，而是：当行动跨越更多阶段，质量越来越取决于上下文、接口、反馈、权限、证据和责任怎样被编排。

！ 控制权迁移不等于人退出循环

低风险、可逆、可自动验收的执行可以下放给智能体；不可逆部署、监管例外、目标冲突和风险接受仍需要明确的人类责任人。成熟度的标志不是减少了多少次点击，而是每项下放都有权限边界、停止条件、验证器、审计记录和升级路径。

2.5 AGI 的典型演进步骤与本书的位置

本书用“对话 → 推理 → 智能体 → 创新 → 组织”描述系统逐步承担的工程角色，并把它与 ANI → AGI → ASI 的能力广度叙事分开。这条角色轨迹是分析框架，不是已经验证的唯一 AGI 量尺：前五项描述工程能力和应用形态，ANI、AGI、ASI 描述能力覆盖范围，不能仅凭名称逐级等同。

一手研究为其中两个能力阶段提供了可观察机制。推理阶段从提示中显式展开中间步骤，进一步发展到用强化学习训练推理行为 (Wei 等 2022 年; DeepSeek-AI 2025 年)。智能体阶段的构件由不同研究分别实证：Toolformer 研究工具调用，ReAct 交错推理与行动，Reflexion 利用语言反馈和情节记忆，Generative Agents 组合记忆、反思与规划，Voyager 在 Minecraft 中积累可复用技能 (Schick 等 2023 年; Yao 等 2023 年; Shinn 等 2023 年; Park 等 2023 年; Wang 等 2024 年)。这些构件共同支持把模型置于“观察—推理—行动—反馈”回路的语言智能体范式；并不意味着每项研究都实现了完整闭环。AgentBench 同时记录了长程推理、决策和指令遵循等持续缺口 (Liu 等 2024 年)。

“本项目已经进入窄域智能体工程”仍不能改写成“已经实现 AGI”。Legg 与 Hutter 的形式化定义把智能与跨广泛环境实现目标联系起来 (Legg 和 Hutter 2007 年)；Chollet 进一步主张评估新技能获取效率，而不是只看已经训练出的任务分数 (Chollet 2019 年)。Morris 等人的 Levels of AGI 框架进一步把相关区分操作化：用**通用性（能力广度）**与**性能（能力深度）**组成二维能力分级，并把自主性、风险与人机交互视为部署维度 (Morris 等 2024 年)。下图把“工程演进叙事”与“AGI 证据门”明确解耦。

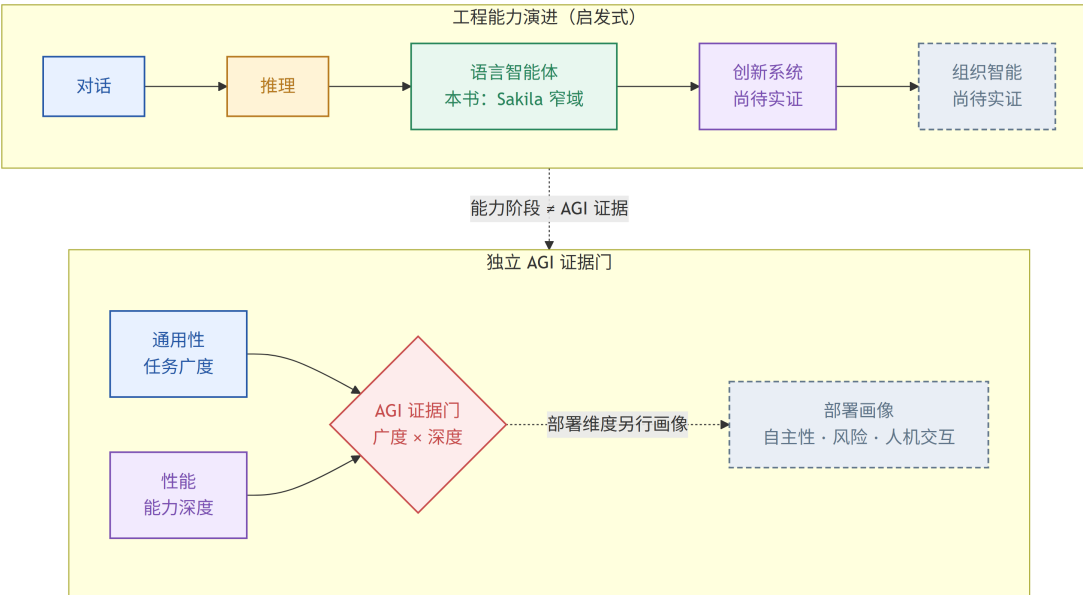


图 2.5: 从产业能力演进叙事到独立 AGI 证据门：本书位于窄域语言智能体工程阶段

下表把工程角色与 AGI 证据要求拆成三列，避免用阶段名称替代能力证明。

表 2.5: 典型演进叙事在本书十层架构中的映射与边界

典型阶段或证据门	核心能力增量	对本书的映射与边界
任务型 AI（前置阶段）	在固定数据和规则边界内完成单一任务	Sakila 是固定窄域；只读 SQL 不变量刻意限制行动范围
对话	用自然语言接收意图并生成响应	Level 1 提供交互入口；流畅表达不等于自主行动
推理	分解多步问题、生成候选并进行验证	各层可调用推理模型，但本书不训练或认证通用推理能力
语言智能体（当前工程阶段）	组合工具、协议、记忆、反馈与停止条件	Levels 2–7 展示可审计窄域智能体所需的组件；这支持“进入 Agent 工程”，不支持“到达 AGI”
创新系统	提出机制，并用证据或实验使其可证伪	Levels 8–10 生成 ontology、因果问题和实验提案；尚未自主执行科学发现
组织智能	长期协调角色、资源、权限与责任	Harness 和多智能体编排只是基础设施，不等同于组织级智能
AGI 证据门	同时检查通用性与性能；另画自主性和风险画像	不是第六个工程步骤；本书没有跨领域迁移和开放世界验证证据

这里的“映射”表示本书研究相应能力阶段所需的工程前提，不表示该能力阶段已经达成。

！ 本项目已经进入窄域语言智能体工程，但还不是 AGI

这里的“进入”是架构范式判断：模型已经被置于目标、工具、观察、记忆和反馈构成的控制回路。本书进一步加入协议、冲突裁决、迭代、图谱和知识治理；但它

仍固定在 Sakila 数据域，数据库权限只读，外部服务有离线替代路径，也没有跨领域迁移或开放世界自主验证。Levels 8–10 的产物是候选知识、因果识别缺口和实验设计，不是自主创新或组织智能已经实现的证据。

2.6 三条智能体路线如何汇入 AGI 议题

本书把智能体工程组织为三条可以并行推进的路线：**模型侧 Agent RL**、由 Skills、Memory、Tools 与 Orchestration 组成的 **Harness 自治**，以及用业务对象、关系和动作约束现实世界的 **Ontology 智能体**。三者可能在创新与组织阶段汇合，但这只是用于分析控制权迁移的工程分类，不是已经形成共识的“三种 Agent”标准，更不等同于三个 AGI 等级。

可以用一手研究把三条路线改写得更可检验。AlphaZero 证明自我博弈、搜索与结果奖励能够在规则完备的棋类环境中学习强策略，但这不等于开放世界的通用自我进化 (Silver 等 2018 年)。DeepSeek-R1 把可验证结果奖励用于数学、代码等推理任务；论文同时报告了可读性、语言混合和开放域能力边界 (DeepSeek-AI 2025 年)。Agent Lightning 则把既有智能体的运行轨迹转换成强化学习转移，并把训练器与 Agent 执行解耦，在 text-to-SQL、RAG 和数学工具调用上报告了改进 (Luo 等 2025 年)。这些结果支持“模型可以从受控任务反馈中更新”，不支持“只要加入 RL 就会获得 AGI”。

Harness 路线控制的是运行期上下文、工具、记忆、验证和停止条件。ReAct、Reflexion、Generative Agents 与 Voyager 分别为行动—推理交错、语言反馈、记忆—反思—规划和技能积累提供了局部证据 (Yao 等 2023 年; Shinn 等 2023 年; Park 等 2023 年; Wang 等 2024 年)。OpenClaw、Hermes 等产品名称具有时间和实现依赖性，因此本书只讨论其背后的控制面，不以产品热度证明自主性。Ontology 路线控制的是世界中的对象、关系、允许动作和不变量；工业数字孪生研究已经在实验环境中验证了 Ontology 对领域知识与流式数据整合的作用 (C. Yang 等 2024 年)，但语义一致性并不会自动保证事实正确、权限完备或物理行动安全。

下图把模型学习、运行期控制与现实世界语义约束组织为一张证据流图。

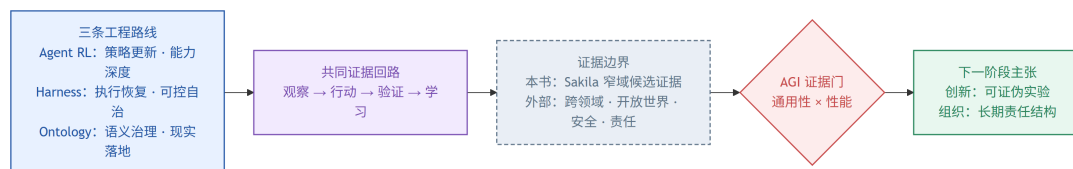


图 2.6: 三条智能体工程路线通过同一证据回路协同，但不会自动越过 AGI 证据门

下表按“控制对象—可验收证据—本书边界”比较三条路线；它不是产品排名。

表 2.6: 三条智能体路线在本书中的控制对象、证据与边界

分析路线	主要控制对象与可验收证据	本书落点与不能越过的边界
模型侧 Agent RL	控制策略参数、奖励和信用分配； 验收轨迹、奖励变化与留出任务表现	Level 5 展示迭代飞轮，但不训练 ornith-1.5:9b；受控任务改进不等于 跨域通用性
Harness 自治	控制上下文装载、工具权限、记 忆、编排、验证和终止；验收 trace、约束拒绝和恢复结果	Levels 2–7 逐步加入 Skills、协议、 Harness、循环、图和 Wiki；可审 计执行不等于完全自治
Ontology 约束	控制对象类型、关系、动作语义和 不变量；验收映射覆盖、约束违反 与溯源	Levels 6、8–10 生成 schema graph、 OWL/OBDA、因果问题和实验提 案；Ontology 不会自行保证知识为 真或行动安全
三路协同	用 Harness 采集证据，以 Ontology 定义可接受状态，再让训练回路根 据反馈更新	十层提供一个窄域研究骨架；没有 跨领域新技能获取、开放世界行动 和长期责任证据，因此不能宣称 AGI

⚠ 预测性主张不等于能力证据

DeepSeek V4、MT-OPD、DSec Sandbox、OpenClaw/Hermes、Ontology RL 和 “Agent OS 成为组织基建” 等说法具有预测或产品语境，本书不把这些名称当作能力证据。更稳健的结论是：三条路线可以同时推进和互相提供数据，但每一项能力仍须用明确环境、任务、指标、权限与失败记录独立验收。

2.7 两条平行的流水线

全书最重要的心智模型，是把数据平面与控制平面分开：



图 2.7: 十层架构中控制平面与只读数据平面的分离

控制平面从 Level 1 一直升级到 Level 10；数据平面的权限没有升级。任何层最终要读取数据，都必须通过 `SakilaDB.query()`：只接受一条 `SELECT` 或 `WITH`，拒绝写操作、SQLite 控制语句和堆叠 SQL。

这条不变量很关键。智能体 “知道更多” 不应等价于智能体 “获得更大数据库权限”。

2.8 离线模式不是假数据模式

PDF 构建全部使用 `offline=True`，以保证可复现且不依赖常驻服务。离线模式仍然会：

- 读取真实 `data/sakila.db`；
- 使用 Polars `read_database_uri` 和 `ConnectorX`；
- 执行真实 LangGraph、Pydantic、NetworkX、RDFLib 与 Ralph loop；
- 生成真实 wiki、OWL、OBDA 和 annotation 文件；
- 经过相同的只读 SQL 守卫。

它跳过的是非确定性或外部依赖：Ollama 推理、Codex app-server reviewer、Omnigent server 和显式 DeepEval judge。每章会标出这条边界。

2.9 一条贯穿全书的问题

比较两个门店的收入、租赁量和客户数，并说明观察到的差异。

固定问题让我们能清楚看见：Level 1 的主要产物是 workflow 状态，Level 3 开始出现协议 trace，Level 7 开始出现持久文档，Level 9 开始出现识别缺口，Level 10 才把业务目标重写为可实验的机制。

2.10 阅读公约

书中使用四类提示：

实现事实

可直接在当前仓库代码或执行输出中验证的行为。

教学模拟

为了离线、快速和确定性而使用的替代路径；它会明确标注，不冒充真实网络协议。

可选外部能力

只有配置并启动对应服务才会发生的调用。

生产提醒

当前 demo 刻意没有覆盖、但生产系统必须补齐的要求。

3 环境与复现

3.1 目标

本章把书稿、可执行代码和最终 PDF 串成一条可复现路径。Quarto 会把 Markdown、代码单元和执行输出交给 Pandoc，再生成 HTML 或 PDF；本项目使用带 Jupyter 执行引擎的 .qmd 章节 (Quarto Project, 不详-d)。

3.2 1. Python 项目

在仓库根目录安装全部演示和书籍依赖：

```
python -m pip install -e ".[harness,eval,dev,book]"
```

核心环境变量都有默认值：

```
$env:OLLAMA_OPENAI_BASE_URL = "http://localhost:11434/v1"  
$env:AGENT_LLM_MODEL = "ornith-1.5:9b"  
$env:SAKILA_DB_PATH = "D:\Python\agent_kungfu_levels\data\sakila.db"
```

项目刻意使用 AGENT_LLM_MODEL，而不是常见但语义不稳定的 OLLAMA_MODEL，避免与主机上“模型存储目录”变量发生名称冲突。

3.3 2. Quarto 与 PDF

安装 Quarto 后，在仓库根目录双击：

```
render_book.cmd
```

也可以直接运行：

```
quarto render my-book --profile zh --to pdf  
quarto render my-book --profile en --to pdf
```

Quarto Book 用 `_quarto.yml` 声明章节顺序与输出格式，完整渲染结果写入 `_book` 目录 (Quarto Project, 不详-a)。本项目使用 XeLaTeX 与 TeX Live 自带的 Fandol 中文字体；Quarto 官方 PDF 指南建议 Unicode 文档显式选择覆盖目标语言字符的字体 (Quarto Project, 不详-b)。

3.4 3. Ollama 实时路径

书中代码默认离线执行。要体验真实模型路径，先确认：

```
ollama list
Invoke-WebRequest http://localhost:11434/v1/models
kungfu-agent run 2
```

Ollama 暴露 OpenAI-compatible /v1 接口；项目通过 LiteLLM 把 `ornith-1.5:9b` 传给这个本地端点。OpenAI Agents SDK 的 LiteLLM 适配属于第三方模型路径，使用时应针对目标 provider 验证工具调用与 usage 行为 (OpenAI, 不详-b)。

3.5 4. 三种入口，各司其职

入口	用途	是否需要编辑器
<code>start_marimo.cmd</code>	交互选择 Level、问题和模式	否
<code>kungfu-agent run N</code>	自动化、脚本或 CI	否
本 Quarto Book	系统学习、可执行讲义、PDF	否

3.6 检查

```
from assets.book_setup import DB, PROJECT_ROOT, SETTINGS

{
    "project_root": str(PROJECT_ROOT),
    "database_exists": SETTINGS.db_path.is_file(),
    "sqlite_uri": SETTINGS.sqlite_uri,
    "tables": len(DB.table_names()),
    "model": SETTINGS.ollama_model,
}
```

```
{'project_root': 'D:\\Python\\agent_kungfu_levels',  
  'database_exists': True,  
  'sqlite_uri': 'sqlite:///D:/Python/agent_kungfu_levels/data/sakila.db',  
  'tables': 16,  
  'model': 'ornith-1.5:9b'}
```

如果 `database_exists` 为 `True` 且 `tables` 大于 0，后续离线章节即可执行。这里没有探测 Ollama，因为 PDF 生成不应依赖本地服务是否恰好在线。

4 Sakila：同一道题，十种控制面

4.1 数据来源与安全边界

Sakila 是经典的影视租赁关系数据库。这里使用 SQLite 文件 `data/sakila.db`。所有读取都汇聚到同一条代码路径：

```
return pl.read_database_uri(query=clean, uri=self.settings.sqlite_uri)
```

`clean` 不是模型承诺，而是 `validate_read_only_sql()` 的输出：它去除注释，要求只有一条语句，必须以 `SELECT` 或 `WITH` 开头，并拒绝 `INSERT`、`UPDATE`、`DELETE`、`DROP`、`PRAGMA`、`ATTACH` 等关键字。

4.2 数据概况

```
from assets.book_setup import DB

key_tables = ["film", "customer", "rental", "payment", "inventory", "store"]
profile = DB.profile()
[{"table": table, "rows": profile[table]} for table in key_tables]
```

数据库一共有 16 张业务表。`build_schema_graph()` 从 `sqlite_master` 的 DDL 提取外键，不依赖手工复制的 schema，因此图、wiki 和 ontology 都能追溯回实际数据库。

表 4.1: Sakila 关键实体规模（实时从 SQLite 读取）

```
[{'table': 'film', 'rows': 1000},
 {'table': 'customer', 'rows': 599},
 {'table': 'rental', 'rows': 16044},
 {'table': 'payment', 'rows': 16049},
 {'table': 'inventory', 'rows': 4581},
 {'table': 'store', 'rows': 2}]
```

4.3 贯穿全书的基线查询

```
from assets.book_setup import DB
from agent_kungfu.database import QUERY_CATALOG

store_baseline = DB.query(QUERY_CATALOG["store_revenue"])
store_baseline
```

表 4.2: 两个门店的描述性基线

store_id	revenue	rentals	customers	avg_payment
i64	f64	i64	i64	f64
2	33726.77	8121	599	4.15
1	33679.79	7923	599	4.25

按当前数据库，门店 2 的收入为 33726.77，门店 1 为 33679.79，差额 46.98；门店 2 多 198 笔租赁，但两店观察到的客户数都是 599；门店 1 的平均支付高 0.10。

! 描述不是因果

这些值说明数据中存在差异，却不能证明“把门店 1 的库存改成门店 2 的样子就会增加收入”。位置、需求、定价、片单和时间趋势都可能同时影响库存与收入。Level 9 才会把这些识别问题显式化。

4.4 为什么固定问题很重要

如果每层都换数据、问题和模型，我们无法判断改进来自哪里。本书固定：

- 同一个数据库；
- 同一个默认问题；
- 同一个只读边界；
- 同一个离线 query catalogue；
- 同一个 LevelResult 输出契约。

随后每层只改变控制机制。观察 answer、sql、trace、metadata 和 artifacts 的变化，就是理解十层的最短路径。

4.5 先验证守卫

```
from agent_kungfu.database import validate_read_only_sql

attempts = [
    "SELECT COUNT(*) FROM rental",
    "DELETE FROM rental",
    "SELECT 1; SELECT 2",
]
guard_results = []
for sql in attempts:
    try:
        guard_results.append({"sql": sql, "result": validate_read_only_sql(sql)})
    except ValueError as error:
        guard_results.append({"sql": sql, "result": f"REJECTED: {error}"})
guard_results
```

```
[{'sql': 'SELECT COUNT(*) FROM rental',
  'result': 'SELECT COUNT(*) FROM rental'},
 {'sql': 'DELETE FROM rental',
  'result': 'REJECTED: Only SELECT or WITH queries are allowed'},
 {'sql': 'SELECT 1; SELECT 2',
  'result': 'REJECTED: Exactly one SQL statement is allowed'}]
```

这个守卫不是完整 SQL parser，也不是数据库账号级权限；它是教学项目的一层应用防线。生产环境还应使用只读数据库账号、查询超时、资源限额和审计日志。

5 实验方法：怎样公平比较十层

5.1 研究问题

本书不是比较十个模型，而是比较十种控制机制。为了避免把模型随机性误当成架构收益，实验固定数据、问题、模型配置和结果契约，只改变主导控制面。

主问题是：

当任务从一次 SQL 生成逐渐演化为多参与者协作、迭代改进、知识积累与因果实验时，每一层新增的机制解决了哪一种具体失败？

5.2 控制变量

变量	固定值
数据	data/sakila.db
数据访问	Polars read_database_uri + ConnectorX
默认问题	比较两个门店的收入、租赁量和客户数
在线模型	Ollama ornith-1.5:9b
在线接口	http://localhost:11434/v1
SQL 权限	单条只读 SELECT/WITH
输出契约	LevelResult

5.3 两条实验轨道

5.3.1 轨道 A：可复现离线实验

本 PDF 的所有执行单元使用 `offline=True`。它验证代码结构、真实 SQL、数据库证据、图、本体和文件产物，适合 CI 与读者复现。

5.3.2 轨道 B：本地实时实验

实时轨道用于观察模型决策、工具选择、Codex protocol workers、OmniGenT 生成的 Codex 两动作计划和 DeepEval judge。它需要额外服务，输出也可能随模型版本、上下文和采样变化。

表 5.1: 统一 LevelResult 契约

```
[{'field': 'level', 'annotation': "<class 'int'>", 'required': True},
 {'field': 'driver', 'annotation': "<class 'str'>", 'required': True},
 {'field': 'question', 'annotation': "<class 'str'>", 'required': True},
 {'field': 'answer', 'annotation': "<class 'str'>", 'required': True},
 {'field': 'sql', 'annotation': 'str | None', 'required': False},
 {'field': 'row_count', 'annotation': 'int | None', 'required': False},
 {'field': 'trace',
  'annotation': 'list[agent_kungfu.contracts.ProtocolMessage]',
  'required': False},
 {'field': 'artifacts', 'annotation': 'list[str]', 'required': False},
 {'field': 'metadata',
  'annotation': 'dict[str, typing.Any]',
  'required': False}]
```

i 证据等级

书中把“离线单元执行成功”与“在线集成验证成功”分开报告。前者不能替代后者，在线 smoke 也不能替代长期可靠性测试。

5.4 观察指标

每层至少记录：

- answer：面向用户的最终输出；
- sql 与 row_count：数据证据；
- trace：跨责任边界的消息；
- metadata：manifest、评分、图摘要、循环历史等控制证据；
- artifacts：wiki、OWL、OBDA、annotation 或候选知识。

```
from agent_kungfu.contracts import LevelResult

[
    {
        "field": name,
        "annotation": str(field.annotation),
        "required": field.is_required(),
    }
    for name, field in LevelResult.model_fields.items()
]
```

5.5 通过标准

确定性 SQL evaluator 最多给四类分数：只读语句 0.35、可执行 0.35、有证据行 0.15、有决策指标 0.15。总分至少 0.9 才通过；门店问题若没有按 `store_id` 分组，最高被压到 0.74。

这不是“回答正确率”。它只是一个可解释的工程 gate，用来确保查询安全、可执行、非空且面向决策。Level 6–8 可以显式启用 DeepEval，让同一个本地模型作为 GEval judge，但 PDF 构建不会调用它。

5.6 当前验证覆盖与空白

仓库的自动测试覆盖：十层离线 smoke、真实 ConnectorX 读取、SQL 守卫、Level 4 bundle/skill 校验与确定性评价、schema graph 路径、两轮 Ralph 改进和 ontology 文件生成。

尚未自动覆盖：Ollama 在线稳定性、OpenAI Agents 工具选择、Codex app-server 全链路、OmniGenT server 上的 Level 4 bundle 上传与 Codex harness、DeepEval judge、OWL 一致性与 OBDA engine 可执行性。后续章节不会把这些空白写成已经验证的能力。即使 Level 4 实时链路连通，Windows Job Object 也只证明进程树/资源隔离，不能作为文件系统或网络策略已被强制执行的证据；这两类边界必须单独验证。

5.7 读结果的方法

遇到“层级更高”的输出，不要先问它是不是更聪明，先问：

1. 新增了哪类结构化产物？
2. 哪个角色获得或失去了控制权？
3. 哪种失败现在能被发现？
4. 发现之后是拒绝、降级、裁决还是迭代？
5. 还缺什么证据，才能把结论用于真实决策？

Part I

第一部 从提示到工具：给智能体一套可控招式

6 Level 1：让提示词指挥流程

6.1 能力验收实验

改变 Top-N 应只改变输出行数；非法 N 或模型改写用户指定 N 必须被拒绝。离线工作流真实执行；LiteLLM 模型边界的契约由测试替身验证。

```
kungfu-agent experiment 1
```

一招一句：Prompt 承载任务语义，LangGraph 决定步骤顺序，代码仍然掌握执行权限。

6.2 从问答到任务：上一层为什么不够

最原始的做法是把 schema 和问题拼成一大段字符串，然后相信模型会返回正确答案。这里至少混在一起三件事：理解问题、生成 SQL、执行 SQL。任何一步失败，都只能得到一个笼统的“模型答错了”。

智能体工程的重心正在从展示基础生成能力，转向让系统在复杂环境中可靠行动；随之出现的控制问题包括长时序规划、记忆、工具集成、风险控制和多智能体治理。这是本书对工程议题的组织方式，不是一条严格的产品年代线，也不是统一验证过的成熟度标准。

本书因此不把“问答 → 智能体”写成模型彼此替代的历史，而把它重写为**交付契约的扩展**。少样本语言模型已经能够通过文本说明和示例处理问答等任务，但实验终点仍是模型生成的文本 (Brown 等 2020 年)。InstructGPT 进一步提高了模型在所测提示分布上遵循用户意图的能力，但指令跟随本身仍不等于规划和执行 (Ouyang 等 2022 年)。

本书采用的关键系统边界出现在文本之外。WebGPT 把浏览、搜索和证据收集加入长答案生成过程，展示了“为回答而行动”的过渡形态 (Nakano 等 2021 年)；WebShop 则要求系统在交互式网站中连续导航和操作，并按任务成功而非措辞相似度验收 (Yao 等 2022 年)。ReAct 研究推理与环境行动的交替 (Yao 等 2023 年)，Toolformer 研究何时以及如何调用外部 API (Schick 等 2023 年)，AgentBench 则把评测扩展到多个交互环境中的推理和决策 (Liu 等 2024 年)。这些工作分别验证了不同组件，不能合并解释为已经实现了可靠、通用的自主智能体。

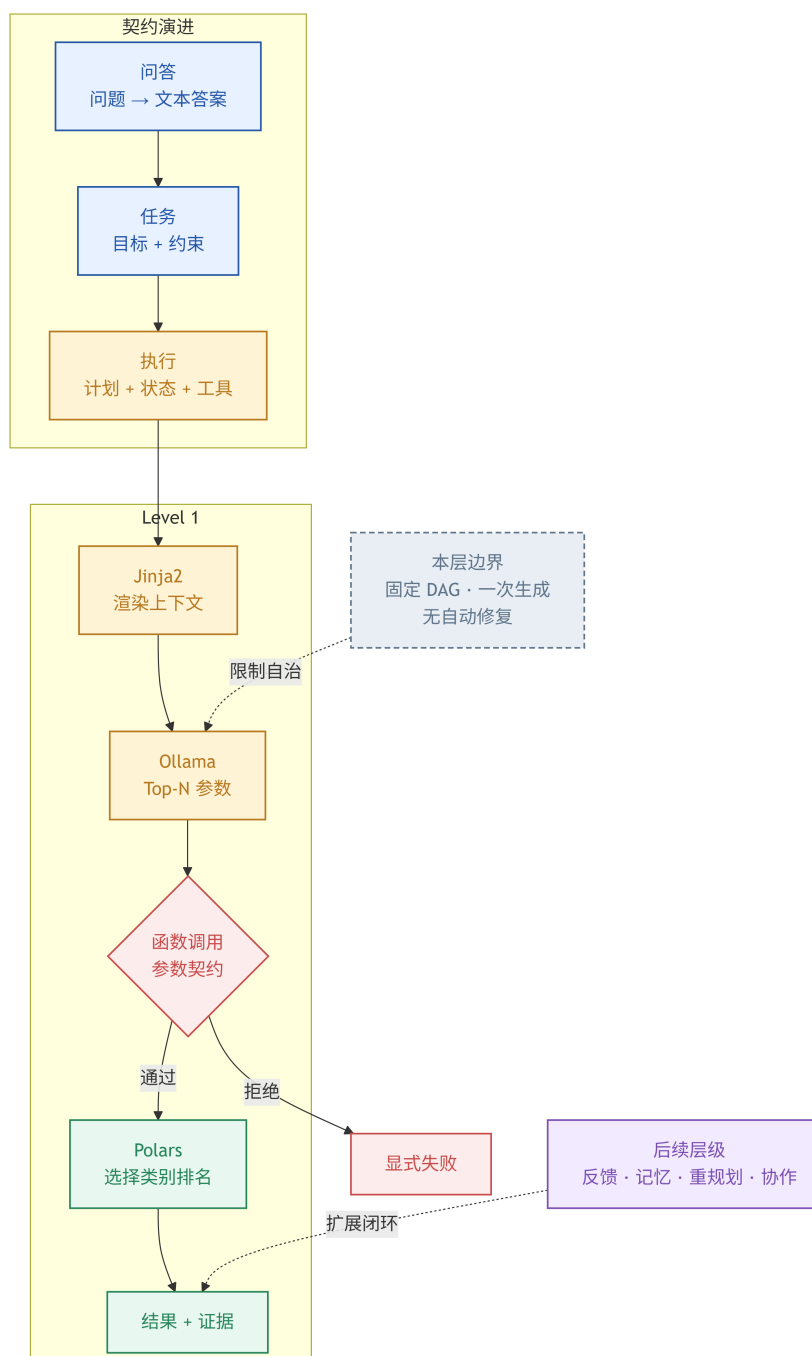


图 6.1: 从回答契约到 Level 1 受控任务契约

图展示任务契约而非完整执行时序。实际先由三个 LangGraph 节点读取五表、连接、聚合，再由 Jinja2 渲染问题和 N，LiteLLM/Ollama 选择 `top_n_categories(n)`。代码校验函数名、唯一参数、整数类型、范围及 N 与用户请求的一致性；读取使用 `SakilaDB.query()` 的只读 SQL 守卫与 Polars/ConnectorX。

协作、持久记忆、领域化、编码和自演化经常被同时放进“智能体能力”清单。为了避免把愿景误写成当前能力，下面把它们重组为可检查的控制问题：哪些能力已经进入 Level 1，哪些必须留给后续层级。

表 6.1: 问答—任务控制契约及其 Level 1 边界

控制维度	本章的单次文本问答基线	任务式系统与本书 Level 1 的边界
输入契约	一个需要回答的问题	目标与约束；本层提示词提供问题和 N；五表读取与连接关系由代码固定
完成定义	文本答案	按环境结果检查；本层检查类别收入结果、Top-N 参数和三步执行证据
控制流	单次生成	一般系统可以循环；本层固定 LangGraph DAG，模型不能改图
外部行动	交付不要求显式工具	本层只执行通过应用层校验的 Polars/SQLite 读取，不声称数据库连接本身只读
状态与证据	问题和回答	RevenueWorkflowState 保存 question、tables、table_rows、merged 和 revenue
失败处理	错误答案通常需要人工重试或改写问题	失败可定位到节点；本层无自动重试、反思或修复
自治边界	模型控制文本输出	模型只提交 Top-N 工具参数；代码控制工具、校验、执行和 END

在本层，“领域专用”仅指 Sakila 环境限定和 SQL 安全边界，不表示已经获得领域知识；多智能体协作映射到 Levels 3–4，持久记忆映射到 Level 7，编码/工具智能体涉及 Levels 2–6，自演化则只在 Levels 5 与 10 研究反馈和实验机制。这样的跨层映射表示本书研究相应能力的工程前提，不表示 Level 1 已经拥有这些能力。

这里的执行证据也不等于语义正确性。一条合法、成功返回数据的 SQL 仍可能误解用户问题；Level 1 只能证明控制链和数据来源可检查，语义评价、修复与升级仍需后续层或人工判断。

! Level 1 的准确定位

本层展示的不是“模型获得了自主性”，而是一个更小但关键的跨越：提示词不再只规定回答内容，还开始为确定性工作流提供任务语义。它是任务式、带智能体特征的工作流，不是自主智能体；真正的行动仍由程序执行，SQL 校验、数据库结果和显式终点只负责验收执行过程。

Level 1 先不追求复杂自治，而是把过程拆成可观察状态机。LangGraph 是低层编排框架，允许开发者显式定义状态、节点和边 (LangChain, 不详)；Jinja2 负责把用户问题和所请求的 N 注入模板 (Pallets Projects, 不详)。

6.3 心智模型

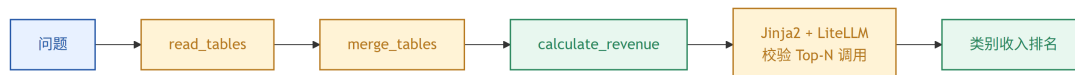


图 6.2: Level 1 的提示词工作流

这里的箭头比“Agent”这个名字更重要。每个节点只读取状态中的若干字段，再写回新字段；如果读取或连接失败，可以定位到 `read_tables` 或 `merge_tables`，而不是猜测整个黑盒发生了什么。

6.4 实现路线图

`RevenueWorkflowState` 是一个 `TypedDict`，保存问题、五张 `DataFrame`、行数、连接表和收入表。`run_workflow()` 恰好执行三个节点：

1. `read_tables` 按配置读取 `category`、`film_category`、`inventory`、`rental`、`payment`；
2. `merge_tables` 沿 `category_id`、`film_id`、`inventory_id`、`rental_id` 连接；
3. `calculate_revenue` 按类别名称汇总金额并降序排序。

图执行完成后，`run()` 在实时模式调用 `LiteLLM/Ollama` 提交 `top_n_categories(n)`；离线模式直接执行同一选择函数并明确标记模拟。

```

graph.add_edge(START, "read_tables")
graph.add_edge("read_tables", "merge_tables")
graph.add_edge("merge_tables", "calculate_revenue")
graph.add_edge("calculate_revenue", END)
  
```

i 实现事实

实时 Level 1 只调用模型一次以选择 N，不生成 SQL，也不能更改图。`LevelResult.sql` 是独立核对用的 baseline SQL，不是模型产物；实际结果来自 Polars 工作流。

6.5 问题贯穿：运行 Level 1

本节使用默认问题：“找出收入最高的前 5 个电影类别。”

```

from textwrap import wrap

from assets.book_setup import DB, run_offline
  
```

```

level_01 = run_offline(1)
print(f"driver: {level_01.driver}")
print(f"rows: {level_01.row_count}")
print("nodes:")
for node in level_01.metadata["graph_nodes"]:
    print(f"  - {node}")
print("sql:")
print("\n".join(wrap(" ".join(level_01.sql.split()), width=68)))

```

driver: Prompt Agent: Polars 三步工作流 + LiteLLM function call

rows: 5

nodes:

- read_tables
- merge_tables
- calculate_revenue

sql:

```

-- Baseline: category revenue produced by the same inner joins as
Level 1. SELECT c.name AS name, ROUND(SUM(p.amount), 2) AS revenue
FROM category c JOIN film_category fc ON c.category_id =
fc.category_id JOIN inventory i ON fc.film_id = i.film_id JOIN
rental r ON i.inventory_id = r.inventory_id JOIN payment p ON
r.rental_id = p.rental_id GROUP BY c.name ORDER BY revenue DESC,
c.name ASC;

```

```
DB.query(level_01.sql)
```

表 6.2: Level 1 独立 SQL 基线的核对证据

name	revenue
str	f64
"Sports"	5314.21
"Sci-Fi"	4756.98
"Animation"	4656.3
"Drama"	4587.39
"Comedy"	4383.58
...	...
"Horror"	3722.54
"Children"	3655.55
"Classics"	3639.59
"Travel"	3549.64
"Music"	3417.72

离线模式真实执行三个 LangGraph 节点和 Polars 运算，仅模拟模型的 Top-N 选择。metadata["rows"] 保存选中的 N 行，revenue_rows 记录完整类别数。上面的 SQL 表是独

立基线，可用于核对聚合结果；离线通过不证明提示词质量或真实模型的工具调用能力。

6.6 SQL 守卫拒绝演练

本层不让模型生成 SQL。下面单独测试公共数据库包装器：即使提示词或未来调用方要求写入，文字也不能替代权限边界。正确分工是：

- Prompt 缩小意图空间；
- workflow 隔离失败位置；
- SQL 守卫执行应用层拒绝规则；
- 数据库返回语法与 schema 错误，但不判断查询是否正确回答了问题。

```
from textwrap import fill

try:
    DB.query("DELETE FROM customer")
except ValueError as error:
    print("rejected: True")
    print(fill(f"reason: {error}", width=68))
else:
    raise AssertionError("SQL guard accepted a write statement")
```

```
rejected: True
reason: Only SELECT or WITH queries are allowed
```

这个演练直接测试数据库包装器，证明应用层校验会拒绝写语句；它不证明 SQLite 连接或操作系统文件权限已经配置为只读，也不单独演示 LangGraph 的错误状态。实际图执行中的同类异常会从 `read_tables` 节点传播。

当前层没有自动修复循环。如果实时模型产生无效工具参数，选择步骤会失败；这正是 Level 5 要解决的一类问题。

6.7 从评测框架看 Level 1 的场景局限

有效的 Agent 评测链从真实任务产生 Bad Case 开始：先记录和分型，再让分型证据进入调优；同时必须把整体任务评价与规划、工具、反思、记忆等子模块评价分开。相关综述同样把核心能力、专用场景、通用智能体和评测框架分层讨论，并指出真实度、细粒度、成本、安全与鲁棒性仍是评测缺口 (Yehudai 等 2025 年)。因此，本书不采用“某个未统一定义的等级已经成为主流”这类缺少口径与数据的判断。

证据至少分三层：**结果层**判断任务是否完成，**轨迹层**判断规划、工具与状态转换是否合规，**运行层**检查安全、时延与成本。存在金标准时，reference-based 检查通常最可复核；没有唯一答案时，reference-free judge 只能提供带版本和校准记录的评价信号；synthetic case 适合扩大边界覆盖，却不能证明真实世界错误的发生比例。

6.7.1 先区分整体评价与子模块评价

对本书的 Level 1，整体问题是“类别收入排名是否符合指定口径和 N”，而不是“SQL 是否运行”；子模块问题则可以拆成下面四项：

1. **规划**：实时模式只提交 Top-N 参数，没有目标分解、备选计划或重规划。
2. **工具**：代码固定调用一条数据库读取路径，模型只能调用一个获准的 Top-N 函数，没有多步函数调用和复杂参数映射。
3. **反思**：异常从 read_tables 传播后即停止；evaluate_sql() 不在本层图内，也没有把反馈送回模型。
4. **记忆**：RevenueWorkflowState 只服务一次调用；LevelResult 保存结果行和调用证据，但不保存完整 prompt、节点时延或跨轮 Bad Case 历史。反馈文本与情景记忆可以形成另一类学习回路，但那已经超出 Level 1 (Shinn 等 2023 年)。

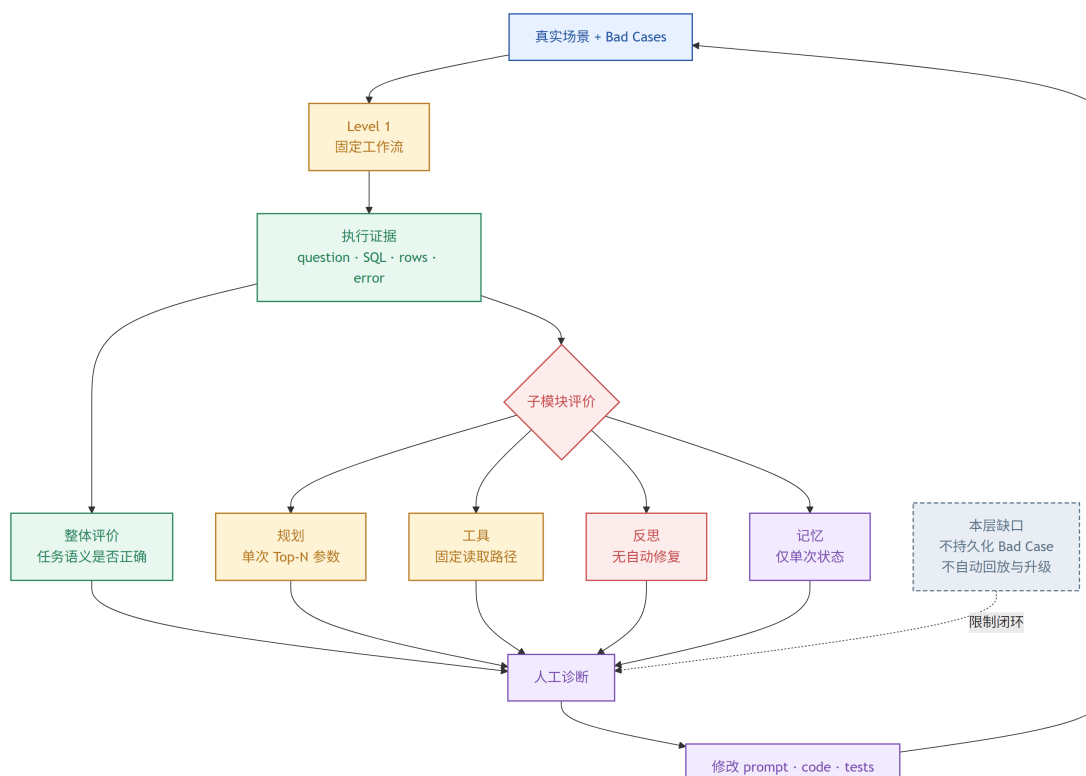


图 6.3: Level 1 Bad Case 诊断飞轮

这个诊断飞轮要求把失败从“结果不对”升级为结构化证据。当前 Level 1 已有可观察节点，却没有持久化评测记录；如果没有问题、schema 版本、prompt、模型参数、SQL、异常、期望结果和判定理由，一次失败还不能成为可回放的 Bad Case。

6.7.2 五类场景的压力测试

表 6.3: Level 1 场景局限与升级触发条件

场景与关联证据	Level 1 可以验收什么	主要局限与升级触发
Web 应用：真实网站要求长时序交互和功能正确性 (Zhou 等 2024 年)	对网站后端的一次只读数据问题给出 SQL 证据	不能浏览页面、处理多模态状态或跨轮恢复；需要 Level 2 工具选择与 Level 5 反馈循环，视觉操作仍超出本书范围
科学辅助：应先评价科学流程中的独立任务、程序、执行结果与成本 (Chen 等 2025 年)	查询已有观测数据并形成可追溯摘要	不能提出并检验假设、设计实验或处理同行反馈；需要 Levels 5、9–10，且必须保留人工科学审查
对话客服：动态用户、API、数据库和领域策略必须共同进入评测 (Yao 等 2025 年)	处理一个问题和一个固定数据库路径	没有对话状态、策略冲突、澄清问题或多工具事务；需要 Levels 2、5、7
编程工程：真实 issue 要求跨文件修改并由测试验证 (Jimenez 等 2024 年)	执行固定 Polars 聚合；公共 SQL 守卫拒绝写操作	不能读取仓库、生成 patch、运行测试—修复循环或判断项目级编排；需要 Levels 2、5–6
通用 Agent：多轮、部分可观察任务需要过程指标，而不只是最终成功率 (Ma 等 2024 年)	在单一路径上定位 读取、连接、聚合或 Top-N 选择步骤	没有工具路由、轨迹进度、协议关联或多智能体冲突控制；需要 Levels 3–4，迭代改进再进入 Level 5

跨场景结论不是“Level 1 表现差”，而是**它的能力声明必须与评测单元同尺度**：可以声称固定数据路径可执行、可定位、可拒绝明显写操作；不能声称它具备多轮任务完成、科学自治、动态客服、软件工程或通用工具编排能力。历史离线 query catalogue 已不用于本层；当前确定性 Top-N 替身适合回归工作流布线，却不能替代真实模型、真实交互和 Bad Case 数据集的评测。

6.8 何时停在这一层

当任务只有一个模型生成步骤、流程固定、工具很少，而且失败可以由调用者处理时，Level 1 已经足够。不要仅为了“更像 Agent”而引入多协议或多智能体。

6.9 升级门槛

Level 1 由代码固定调用一条数据库路径，TypedDict 只是内部状态类型。当你希望模型在一组代码批准的工具中选择，并用运行时 schema 检查每个工具的输入输出时，进入 Level 2。

6.10 章末练习

1. 把 Top-N 从 3 改为 5，再故意让模型返回错误 N，观察参数契约是否拒绝。

2. 为什么 Prompt 中的“不要写数据库”不能替代 `validate_read_only_sql()`?
3. 如果删除 `read_tables` 节点，本层还算数据智能体吗？它还能提供什么？

7 Level 2：把能力从文本搬进代码

7.1 能力验收实验

离线使用确定性模型回复，但真实运行 OpenAI Agents Runner、负责人和两个专家工具。观察 sdk_tool_calls，并验证结果与收入基线一致。

```
kungfu-agent experiment 2
```

一招一句：模型负责选择工具，代码负责定义工具到底能做什么。

7.2 从“请这样做”到“你只能这样做”

Level 1 由代码固定读取、连接和聚合，模型只提交 Top-N 参数。Level 2 把能力封装为 skill：QA leader 通过 Agents SDK 委派读取与计算两个专家，专家只能调用代码授予的工具。

OpenAI Agents SDK 的 Agent 组合 instructions、tools 与模型，Runner 执行模型—工具循环；Python 函数可以包装为 function tool (OpenAI, 不详-c, 不详-d)。项目使用的发行包名是 openai-agents，导入名是 agents。

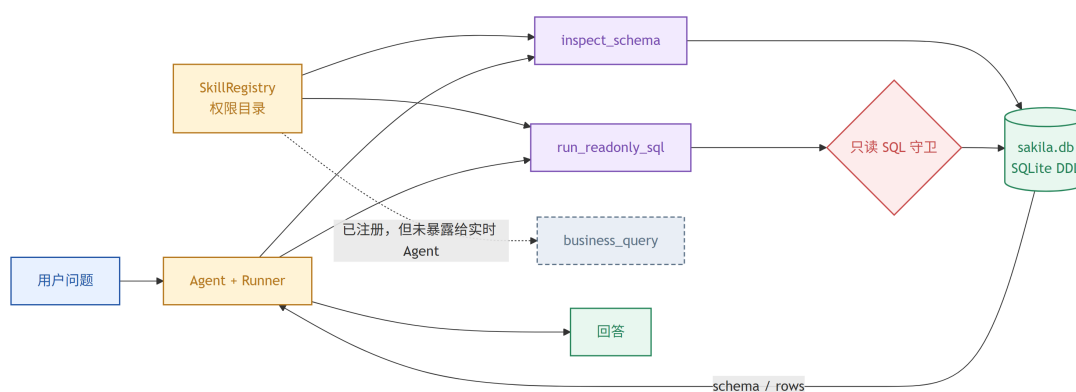


图 7.1: Level 2 中由代码定义的 Skill 权限边界

7.3 SkillRegistry 是权限目录

以下 registry 是独立的权限目录示例。当前主 demo 由 QA Leader 通过 Agent.as_tool 委派读取与计算专家，代码位于 level2/agents.py。目录示例注册三个能力：

Skill	输入	输出	数据权限
inspect_schema	无	实际 SQLite DDL	读取 schema
run_readonly_sql	SQL	最多 50 行 JSON	只读查询
business_query	查询目录键	最多 50 行 JSON	已审核目录查询

下方独立示例仅暴露前两个 function tools；主 demo 则只暴露读取与计算专家。工具是否暴露由代码决定。

```
@function_tool
def query_sakila(sql: str) -> str:
    """Execute one read-only SELECT/CTE and return up to 50 rows as JSON."""
    return registry.get("run_readonly_sql").invoke(sql)

agent = Agent(
    name="sakila-skill-agent",
    tools=[inspect_sakila_schema, query_sakila],
    model=model,
)

run = Runner.run_sync(agent, question, max_turns=6)
```

7.4 运行代码拥有的 skill

```
from assets.book_setup import DB, run_offline
from agent_kungfu.skills import SkillRegistry

registry = SkillRegistry(DB)
level_02 = run_offline(2)
{
    "registered_skills": registry.names(),
    "reported_sdk": level_02.metadata["sdk"],
    "rows": level_02.row_count,
    "sdk_tool_calls": len(level_02.metadata["sdk_tool_calls"]),
}
```

```
{'registered_skills': ['business_query', 'inspect_schema', 'run_readonly_sql'],
 'reported_sdk': 'openai-agents',
 'rows': 5,
 'sdk_tool_calls': 2}
```

```
try:
    registry.get("run_readonly_sql").invoke("DELETE FROM payment")
except ValueError as error:
    rejected_write = str(error)
rejected_write
```

```
'Only SELECT or WITH queries are allowed'
```

这个失败发生在 Python 工具内部，不需要模型“同意”或“理解”。这就是代码拥有权限的含义。

7.5 离线与实时的真实差异

💡 教学模拟

离线 Level 2 实际构造 Agent 团队并调用 Runner。确定性 LessonModel 仅替代模型回复；SDK 仍执行嵌套专家和真实 Polars 工具。该实验验证 SDK 与数据路径，不验证真实模型的工具选择能力。

⚠️ 可选外部能力

实时路径使用 LitellmModel 和相同的 Agent 团队，负责人最多八轮，每个专家最多两轮。is_enabled 和工作区状态控制先读后算，并拒绝重复计算。

另外，skills/sakila-analyst/SKILL.md 是面向人和代码智能体的规范资产；当前 Python runtime 没有动态读取它。书中把它视为可迁移的技能说明，而不是隐式运行的插件。

7.6 OpenClaw：通用智能体的工程突破，而非通用智能证明

判断一个通用智能体框架是否取得工程进展，不能看市场故事或产品排名，而应拆成六个可检验主题：技能、持续运行、受控行动、协作、进化与通用性。每一项都需要对应的运行机制、权限边界和失败证据；组件齐全本身不证明通用智能。

OpenClaw 的关键进展不是发明了一种新的推理算法，而是把原本分散的能力拼成一个可持续运行的智能体框架：SKILL.md 提供可发现的过程知识，Gateway 与 session/queue

管理一次次运行，模型—工具循环执行动作，Markdown 文件保存显式记忆，tool policy 与 sandbox 限制副作用 (OpenClaw Project 2026 年 d, 2026 年 a, 2026 年 b, 2026 年 c)。这使同一模型能在不重新训练权重的情况下扩大可到达的任务面，是通用智能体工程的重要突破。

但“技能可见”不等于“工具授权”。OpenClaw 文档明确把 agent allowlist 与主机 shell 授权分开；记忆文件也只是可供上下文读取的状态，并不执行安全策略 (OpenClaw Project 2026 年 d, 2026 年 b)。因此，技能说明负责告诉模型“何时、如何做”，代码和运行环境仍必须决定“能不能做、在哪里做、做坏了如何停止”。行动半径越大，潜在效用和故障爆炸半径会同时扩大，权限、预算、超时、会话隔离和验证强度必须同步收紧。这正是 Level 2 的核心边界。

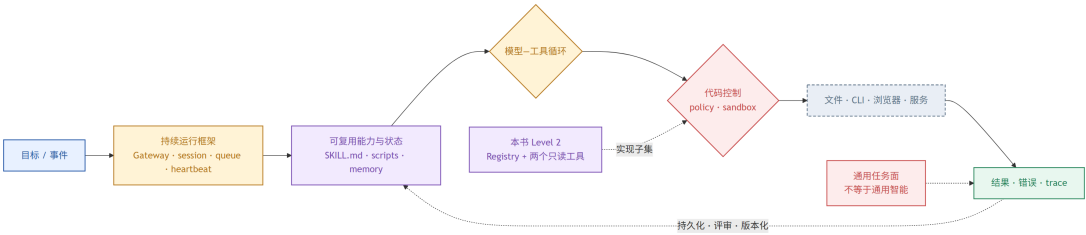


图 7.2: 由技能包到受控行动：OpenClaw 对通用智能体工程的跨层启示

! “通用”至少有三个层次

1. **通用工具面**：能接触文件、命令行、浏览器和服务；

2. **通用型执行框架**：能路由、记忆、调度、恢复并约束这些工具；

3. **通用智能**：面对未见任务仍能可靠迁移、学习并完成目标。
- OpenClaw 的架构主要推进前两层。它扩大了智能体的行动半径，却不能仅凭组件齐全证明第三层。

下面用本书“控制面—证据—升级门槛”的语言建立工程验收矩阵。

表 7.1: OpenClaw 的工程验收矩阵

重写后的主题	通用智能体进展及可验收证据	本书映射与硬边界
可复用技能	把过程知识做成可发现、可筛选、可版本化的资产；验收选择记录、参数校验、回放与版本	Level 2；本项目只有进程内 registry，不动态加载 SKILL.md
持续运行	Gateway、session、queue、heartbeat 与持久化让任务跨轮次延续；验收超时、重启、压缩和 trace	跨 Level 3、4、7；heartbeat 只是调度器唤醒，不等于自主性
受控行动	文件、CLI、浏览器和服务形成广工具面；验收 allow/deny、隔离、最小凭证与副作用日志	Level 2、4；skill 可见性不是授权，sandbox 也不是绝对安全边界
多智能体协作	路由与子任务会话扩展并行能力；验收关联 ID、隔离、冲突规则和总成本	Level 3、4；更多 agent 也会放大延迟、费用和不一致

重写后的主题	通用智能体进展及可验收证据	本书映射与硬边界
技能进化	从成功/失败轨迹中提炼候选 skill； 验收独立测试集增益、人工批准、 回滚和版本差异	Level 5；自写文件不等于已经学习， 更不等于稳定改进
通用性主张	跨应用任务覆盖比单一聊天框更 广；验收必须使用执行式基准和失 败分类	框架通用不等于 agent 能泛化，更 不等于 AGI

这种分层也解释了为何“不断增加社区 skill”不能直接推出通用智能。ReAct 证明了把推理轨迹与动作交错的有效性，Voyager 展示了可复用技能库对开放式探索的价值 (Yao 等 2023 年; Wang 等 2024 年)；较新的 SkillIRL 预印本进一步研究从经验中蒸馏层级技能并与策略共同演化，但证据仍来自 ALFWorld、WebShop 和搜索任务，不能自动外推到任意真实环境 (Xia 等 2026 年)。另一方面，OSWorld 的 369 个真实计算机任务中，论文报告人类成功率超过 72.36%，当时最优模型只有 12.24%，说明“能连接许多工具”与“能可靠完成开放任务”之间仍有巨大距离 (Xie 等 2024 年)。

对企业实践，更稳妥的结论是 **SOP + 有界智能体路径**：高频、高风险任务走确定性 workflow；低频、长尾且可回滚的任务允许智能体在权限、预算和复核门内探索。Sakila Level 2 只有 QA leader 和两个受限专家；写操作没有进入工具白名单。OpenClaw 展示的是这个原则如何扩展成跨层系统，而不是证明 Level 2 已经拥有 Gateway、heartbeat、持久记忆或强化学习。

7.7 本层仍然不能保证什么

- 模型可能选择错误工具或提供错误参数；
- function tool 的 docstring 不等于 authorization policy；
- LevelResult 保留 SQL、行数、真实 SDK 工具调用和模型边界标记；跨进程传输另需协议证据；
- 一个进程内 registry 不能解释跨进程消息在哪里丢失。

前两项需要 guardrail 和测试；后两项把我们推向协议与可观测性。

7.8 何时停在这一层

单进程、少量工具、无需跨 agent 审计的只读数据助手，通常可以停在 Level 2。代码工具比“堆更多 Prompt”更值得优先投资。

7.9 升级门槛

当 planner、reviewer 和 worker 属于不同责任主体，或者必须追踪一项工作穿过哪些边界时，进入 Level 3。

7.10 章末练习

1. SkillRegistry 注册而未暴露的 skill，模型能否调用？为什么？
2. 给 run_readonly_sql 增加允许表清单，应放在 docstring 还是代码中？
3. max_turns=6 与“最多执行一次查询工具”是不是同一个约束？

Part II

第二部 从协议到飞轮：让协作可追踪、可 裁决、可纠错

8 Level 3：让边界可以被追踪

8.1 能力验收实验

工具动作通过序列化 JSON-RPC 请求与响应执行。重复请求 ID 和未知方法必须在执行前被拒绝；A2A/MCP 标签仍是教学消息，不表示互操作认证。

```
kungfu-agent experiment 3
```

一招一句：能力不只要能调用，还要知道谁通过什么边界把什么交给了谁。

8.2 工具调用之外的问题

Level 2 很适合单进程工具循环。Level 3 把“谁负责读表、谁负责计算”移到 `agent.md` 中，由工厂动态生成 Reader 和 Calculator 两个 Codex worker 规格。一旦责任跨过独立 thread，仅有函数名就不够了：这个 worker 来自哪个 manifest 版本？哪个 Codex thread 提议了这个动作？宿主是否只执行了白名单中的 Polars 能力？读表与计算证据是否属于同一次请求？

Level 3 不再使用 `openai-agents` 作为本层运行时。它用 Codex Python SDK 驱动两个独立 thread，并用统一的 `ProtocolMessage envelope` 记录：

- `protocol`：本层实际记录 A2A、Codex app-server JSON-RPC 与 MCP；
- `sender` 与 `recipient`；
- `kind`：例如 `analysis.request`、`task.delegate`、`turn.completed` 与 `tool.result`；
- `payload`；
- `correlation_id`；
- 时间戳与唯一消息 ID。

8.3 一次请求的时序

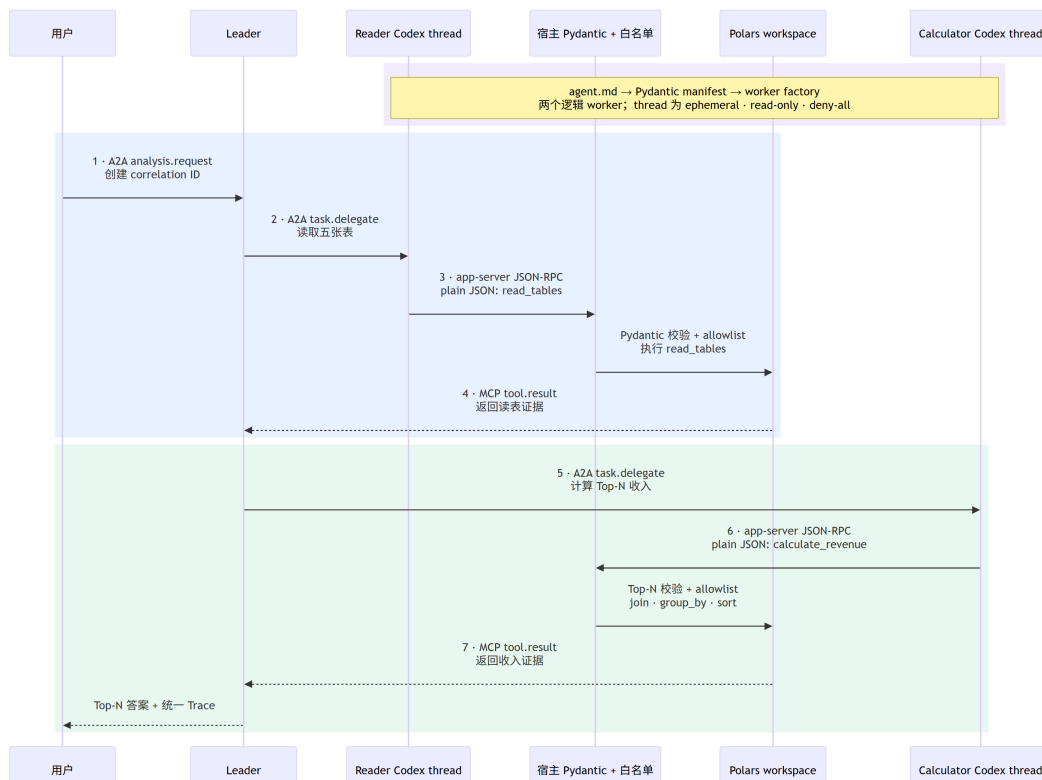


图 8.1: Level 3 的实际架构: `agent.md` 生成两个 Codex worker, 宿主白名单执行 Polars

这里的统一 envelope 不是要抹平协议差异, 而是让观测层有共同字段。A2A 描述 Leader 与 worker 之间的任务边界, Codex JSON-RPC 描述 SDK 与本地 app-server 之间的 thread/turn 边界, MCP 描述工具结果的证据语义。它们在 trace 中共享同一个 `correlation_id`, 但仍保留不同的责任语义。

8.4 Harness: 稳定性不是模型属性, 而是系统产物

Prompt、RAG、skills、memory、protocol 与 orchestration 并不是互相替代的层, 它们需要由运行期 harness 统一治理。由此得到一个可验收命题: **harness 不是“模型外面的一堆组件”, 而是把模型能力转换成稳定输出的证据与控制系统**。新近综述也把执行环境、工具、上下文、状态、生命周期和验证视为同一治理对象, 并把 memory、skills 与 protocols 解释为需要统一协调的外置认知工件 (Meng 等 2026 年; Zhou 等 2026 年)。

这种稳定性不能从“模型很强”直接推出。分布式系统早已用 Dapper 展示了低开销 trace 对跨模块定位的价值; OpenTelemetry 进一步用 trace ID、span ID 和上下文传播把不同进程产生的信号关联起来 (Sigelman 等 2010 年; OpenTelemetry Project 2026 年)。ProtocolMessage.correlation_id 借用了同一思想, 但它只是本书的教学 envelope, 并不声称实现了 OpenTelemetry。机器学习系统的系统级技术债研究也提醒我们, 边界侵蚀、

隐藏反馈环、配置与外部变化会让可靠性问题超出模型或单段代码本身 (Sculley 等 2015 年)。

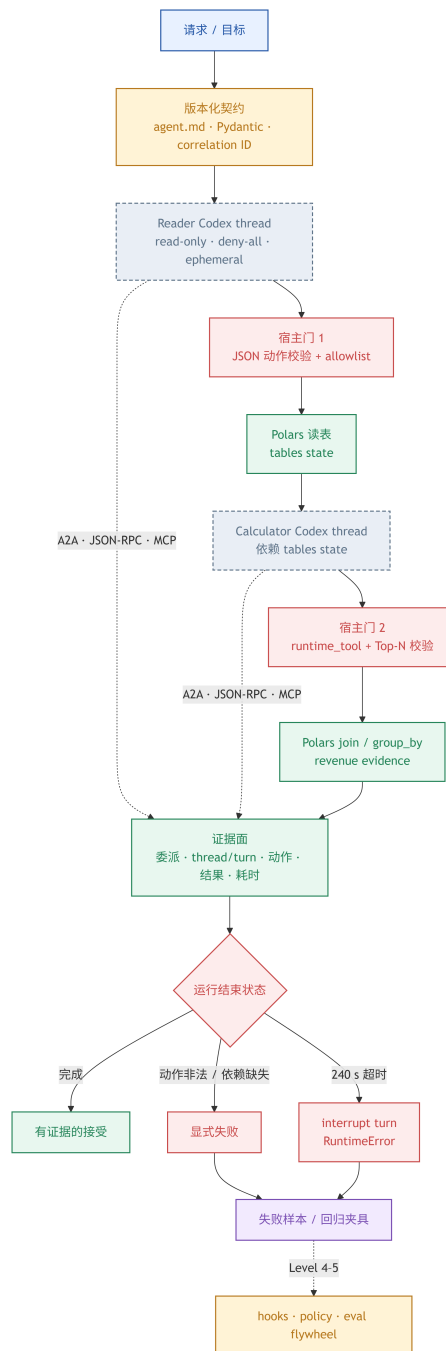


图 8.2: Harness 稳定性供应链：Level 3 先让边界与失败可追踪

图中的“稳定输出”不是保证每次都成功，而是保证运行以一种可解释的状态结束：成功有证据，非法动作在宿主门被拒绝，超时会中断 turn 并显式失败。当前 Level 3 没有“超时后继续使用旧结果”的 fallback，因此不把失败包装成成功。只有最终答案而没有阶段证据时，系统得到的是一次结果；当输入、边界、状态和输出可以关联时，系统才得到可诊断、可比较、可回归的稳定性基础。

⚠️ 多步骤衰减只是模型，不是实测结论

如果严格串行的 n 个步骤必须全部成功，并且暂时假设各步独立，则端到端成功率可写为

$$P_{\text{end}} = \prod_{i=1}^n p_i.$$

因此十步都取 $p_i = 0.8$ 时， $0.8^{10} \approx 10.7\%$ 。这是独立步骤假设下的教学模型，不是对真实 agent 的经验估计。现实步骤往往相关，且 harness 会通过校验、有限重试、fallback 或人工升级改变有效成功率。这个公式真正说明的是：只看最终结果无法知道损失发生在哪一段。

下面把运行治理整理为七个控制面，并用“渐进披露、架构约束、测试、hooks、独立审计、失败回归”六条原则检查每一面。

表 8.1: Harness 稳定性控制矩阵

重写后的控制面	稳定输出需要回答的问题与最小证据	本书层级与边界
范围与上下文	这次任务允许读什么、注入了哪个上下文版本？记录 scope、context hash 与来源	Level 1–3；Level 3 由 agent.md 定义 worker 契约，但尚未记录 manifest hash
工具与协议	谁调用了哪个工具、参数是否符合 schema、返回属于哪次请求？记录 sender、recipient、kind、protocol、参数与结果摘要	Level 2–3；本章已有 CodexAction 校验、宿主白名单与最小关联 envelope
状态与记忆	状态来自本轮、session 还是长期存储，何时写入？记录 state version、读写事件和提交结果	Level 3、7；当前 demo 没有跨运行持久记忆
生命周期与调度	第几次尝试、为何重试或停止、是否走 fallback？记录 attempt、timeout、termination 与 fallback reason	Level 3–5；当前可配置上限为每个 Codex turn 240 秒，超时会 interrupt 并报错，没有自动 fallback
可观测性与成本	失败位于哪个阶段，延迟和 token/费用花在哪里？记录 status、duration、usage 与 parent/trace 关系	Level 3；Codex turn 事件已记录 status 与 duration，metadata 可保留 usage，但仍缺 parent span 与成本换算
验证与反馈	哪条测试或 judge 判定通过，失败能否进入回归集？记录 evaluator version、score、fixture 与处置	Level 5–6；Level 3 只生产证据，不声称完成评价飞轮
环境与安全	在哪个快照、沙箱和权限集内执行，副作用是否越界？记录 environment digest、policy version 与 side-effect log	Level 3 已将两个 Codex thread 设为 read-only、deny-all、ephemeral，并以宿主白名单授权；数据库身份、环境摘要与更广策略尚未完成

这张表也界定了“harness 成为稳定性输出的系统解决方案”的含义：它统一执行契约、状态、工具、生命周期、观测、验证和环境控制，使失败能够被定位并转化为确定性约

束。2026 年的综述把 harness 描述为执行环境、工具、上下文、状态、生命周期和验证的统一治理对象，但该六组件框架仍是概念性提出，作者也明确把经验验证列为后续工作 (Meng 等 2026 年)。另一篇 externalization 综述把 memory、skills、protocols 视为外置认知工件，并把 harness 定义为协调它们的统一层 (Zhou 等 2026 年)；这些材料均为新近预印本，应当作为架构假设而不是成熟标准。

Natural-Language Agent Harnesses 尝试把 run-level policy 写成可执行、可比较、可消融的文档对象，说明 harness 本身可以成为检查对象 (Pan 等 2026 年)。Meta-Harness 则进一步搜索 harness 代码并在若干任务上报告跨模型收益，但这属于 Level 5 的优化闭环，不能倒推 Level 3 的 trace 已经具备自我改进能力 (Lee 等 2026 年)。OpenAI 的 Codex 工程复盘同样把可读环境、版本化知识、日志/指标/trace、机械约束和反馈环视为可靠工作的基础；它是有价值的生产案例，但不是受控学术实验 (Lopopolo 2026 年)。

i 同名异义的引用清理

arXiv:2605.23572 虽然题名含 HARNESS-LM，研究对象却是广告搜索检索器的小模型蒸馏与线上延迟，不是 agent execution harness，因此本书不把它作为本章证据 (Gupta 等 2026 年)。

8.4.1 Level 3 是稳定性底座，不是完整稳定性承诺

本章的实现覆盖统一 envelope、agent.md manifest 校验、两个独立 ephemeral thread、correlation ID、时间戳、消息 ID、turn status/duration、宿主 Pydantic 动作校验、严格白名单以及当前配置为 240 秒的超时中断。Reader 和 Calculator 名称是宿主 trace 中的逻辑角色标签，不是 ephemeral thread metadata。它尚未提供 parent span、attempt、成本换算、environment digest，也没有把 tests、hooks、judge 和失败样本接成闭环。换言之，Level 3 让稳定性问题可以被定位；Level 4 才让冲突与权限被确定性裁决，Level 5 再让失败持续变成回归与改进。

8.5 追踪一次离线运行

```
import polars as pl
from assets.book_setup import run_offline

level_03 = run_offline(3)
roles = {
    "user": "User", "Dynamic Sakila QA Leader": "QA",
    "Dynamic Table Read Agent": "Reader",
    "Dynamic Revenue Calculate Agent": "Calculator",
    "codex-app-server": "Codex",
}
```

```

trace_rows = [
    {
        "protocol": message.protocol.value,
        "sender": roles.get(message.sender, message.sender),
        "recipient": roles.get(message.recipient, message.recipient),
        "kind": message.kind,
    }
    for message in level_03.trace
]
pl.DataFrame(trace_rows)

```

表 8.2: Level 3 的协议 trace

protocol	sender	recipient	kind
str	str	str	str
"a2a"	"User"	"QA"	"analysis.request"
"a2a"	"QA"	"Reader"	"task.delegate"
"codex-jsonrpc"	"Reader"	"Codex"	"turn.simulated"
"mcp"	"Reader"	"QA"	"tool.result"
"a2a"	"QA"	"Calculator"	"task.delegate"
"codex-jsonrpc"	"Calculator"	"Codex"	"turn.simulated"
"mcp"	"Calculator"	"QA"	"tool.result"

```

request_id = level_03.trace[0].id
{
    "messages": len(level_03.trace),
    "expected_seven_events": len(level_03.trace) == 7,
    "protocols": level_03.metadata["protocols"],
    "request_id": request_id,
    "all_followups_correlated": all(
        message.correlation_id == request_id for message in level_03.trace[1:]
    ),
}

```

```

{'messages': 7,
 'expected_seven_events': True,
 'protocols': ['a2a', 'codex-jsonrpc', 'mcp'],
 'request_id': '3a7c8c29-9759-4151-b923-28e88957b63c',
 'all_followups_correlated': True}

```

七个事件由一条根请求和两组三步 worker 事件组成：A2A task.delegate → Codex JSON-RPC turn → MCP tool.result。这条断言比看日志字符串更强：根事件之后的六条消息都能由机器判定为同一项工作。

8.6 Codex workers 的真实边界

实时模式先把 agent.md 的 YAML front matter 校验为 AgentManifest，再动态构造 Reader 与 Calculator 两个 CodexWorker。它们共享一个 Codex app-server client 上下文，但每个 worker 都启动自己的 ephemeral thread。Reader 与 Calculator 是宿主保留在 manifest 和 trace 中的逻辑角色名：

```
manifest = load_agent_manifest()
workers = build_codex_workers_from_manifest(manifest, timeout_seconds=240)
config = CodexConfig(
    cwd=str(PROJECT_ROOT),
    env={
        "OPENAI_BASE_URL": settings.ollama_base_url,
        "OPENAI_API_KEY": settings.ollama_api_key,
    },
)

rpc = RevenueRpc(workspace, requested_n)
with Codex(config) as codex:
    for worker in workers:
        action, turn = worker.run(
            codex,
            settings=settings,
            requested_n=requested_n,
            workspace=workspace,
        )
        rpc.call(action.runtime_tool, action.n)
```

worker.run() 内部使用的边界不是普通 Python 函数工具调用，而是 Codex Python SDK 通过 app-server JSON-RPC 控制本地 Codex agent (OpenAI, 不详-a)：

```
thread = codex.thread_start(
    approval_mode=ApprovalMode.deny_all,
    base_instructions=worker.definition.instructions,
    model=settings.ollama_model,
    model_provider="ollama",
    sandbox=Sandbox.read_only,
    ephemeral=True,
)
handle = thread.turn(prompt, effort="none")
```

Codex app-server 不允许为 `ephemeral=True` 的 `thread` 写入 `metadata`，因此实现不调用 `thread.set_name()`。如需稳定识别角色，应使用 `agent.md` 中的 `worker name` 作为宿主侧 `trace label`，而不是把它误写为 SDK `thread metadata`。

`Reader` 和 `Calculator` 的模型输出都只是一个包含 `runtime_tool`、`n` 和 `summary` 的普通 JSON 建议。宿主把它解析为 `CodexAction`，检查 `runtime_tool` 必须与 `manifest` 声明相同、`Calculator` 的 `n` 必须等于用户请求，最后才在严格白名单中映射为 `workspace.load_tables()` 或 `workspace.calculate()`。本层协议没有把 SQL 或 Polars 方法暴露为 `worker` 动作；被这条 workflow 接受的数据执行只能经过宿主白名单。`read-only` 防止写入、`deny-all` 阻止升权，但它们不等于“Codex 无法尝试任何只读命令”；生产系统还应收窄 `cwd`/文件系统可见性，并使用独立的只读数据库身份。

⚠ 本地模型的结构化输出兼容性

在本书锁定的 Ollama + `ornith-1.5:9b` 环境中，为 Codex `turn` 设置 `SDK output_schema` 会在 `sampler grammar` 初始化时返回 400。因此当前实现要求模型返回普通 JSON，再由宿主 `Pydantic` 校验。这是针对当前本地组合的兼容性取舍，不应外推为所有 Codex 模型都不支持 `output_schema`。

这个 `turn` 只负责将狭窄的路由意图转换成 JSON 动作，不需要扩展性推理，因此设置 `effort="none"`。当前每个 `turn` 的可配置上限是 240 秒；超时时宿主会调用 `interrupt()` 并抛出 `RuntimeError`，而不是保留一个未验证的旧 `proposal`。这使“超时”成为可归属失败，不是隐性降级。

❗ 不要把标签说成实现

离线模式不启动 Codex app-server 或真实 `thread`；它按同一 `manifest` 和白名单模拟两个动作，并把 Codex 事件标记为 `turn.simulated`。实时路径则确实启动两个 `ephemeral` Codex `thread`，两个 `turn` 都穿过 app-server JSON-RPC。两种模式中的 A2A 与 MCP 都是进程内教学 `envelope`，并没有启动 A2A 或 MCP 网络 server。

8.7 本层新增了什么，也没新增什么

新增：可声明的 `worker` 规格、动态 Codex `thread` 边界、`read-only` + `deny-all` + `ephemeral` 约束、宿主动作校验与白名单，以及一条关联 A2A、Codex JSON-RPC 和 MCP 的七事件 `trace`。它们共同构成最小协议 `harness`，使稳定性从主观感受变成可以定位的证据。

未新增：当两个合法 `proposal` 不一致时的选择规则。协议可以忠实记录冲突，却不会自动解决冲突。这是 Level 4 的工作。

8.8 何时停在这一层

如果责任主体已经分离,但每一步只有单一候选结果,Level 3 的 trace harness 足以提供可诊断性。出现竞争 proposal、权限策略或强制 hooks 时,升级到 Level 4 的 policy harness;需要让失败自动进入迭代时,再进入 Level 5。

8.9 章末练习

1. 为什么所有消息共享 envelope,不代表所有消息使用同一种真实网络协议?
2. 为什么一次完整运行恰好产生七个 trace 事件? 根请求之后的六个事件如何分组?
3. 已经设置 Sandbox.read_only 和 ApprovalMode.deny_all,为什么宿主仍需要 Pydantic 动作校验和严格白名单?
4. 将 SDK output_schema 改为“普通 JSON + Pydantic”解决了哪个兼容性问题? 又把哪些验证责任移给了宿主?
5. 为什么一次成功的最终回答不能证明 harness 稳定? 至少还需要哪三类阶段证据?

9 Level 4：用 Harness 封住智能体行为

9.1 能力验收实验

注入一个可执行但收入翻倍的高置信度方案，与正确方案竞争。独立基线必须决定胜者；所有方案错误时拒绝交付。这是明确注入的冲突实验。

```
kungfu-agent experiment 4
```

一招一句：OmniGenT 让两个 Codex 子智能体提出动作，宿主只执行白名单动作，再用独立证据验收整次运行。

9.2 问题：可追踪不等于可控制

Level 3 已能把任务、协议封装和结果串成可追踪链路，但 trace 只能说明“发生了什么”，不能保证智能体只做允许的事。模型仍可能改变步骤顺序、扩大参数范围、遗漏技能约束，或把一个貌似合理的结果直接交给用户。

Level 4 把控制单元从“协议消息”升级为“受 harness 约束的动作计划”。当前 Sakila 实现建立三道连续边界：

1. **生成边界**：OmniGenT bundle 只声明 Reader 与 Calculator 两个 Codex 子智能体；
2. **执行边界**：子智能体只提出动作，宿主只接受 read_tables 与 calculate_revenue；
3. **评价边界**：结果必须通过结构、来源、数值域与 SQL 基线的一组确定性检查。

9.3 架构：一个 bundle、两个子智能体、一个执行宿主

omnigent-agent/config.yaml 是根 harness；它按固定顺序引用两个子 bundle：

- **Polars Table Reader**：只能提出 read_tables，并且 n=null；
- **Polars Revenue Calculator**：只能提出 calculate_revenue，并保留请求中的 n。

两者都声明 executor.config.harness: codex，使用本地 Ollama 模型 ornlith-1.5:9b。根配置与两个子配置还显式声明相同的 Codex os_env: sandbox: auto、write_paths: []、allow_network: false、start_in_scratch: true；OmniGenT 0.11 的 Codex executor 以 approvalPolicy=never 启动 turn。宿主 evaluator 核对这些配置与运行契约，但真正可跨

平台验证的能力硬边界仍是：协议只接受 worker 的类型化 proposal，宿主只授权两个 SQLite/Polars 白名单分支。

⚠ Windows 上的边界不能夸大

Windows Job Object 约束进程树与资源，却不强制文件系统或网络策略。因此，sandbox: auto、空 write_paths 和 allow_network: false 在 Windows 上属于纵深防御声明，不能单独证明“绝对只读”或“网络必然被阻断”。Level 4 的跨平台安全结论应落在 proposal/execute 分离、Pydantic 契约和宿主 allowlist 上。

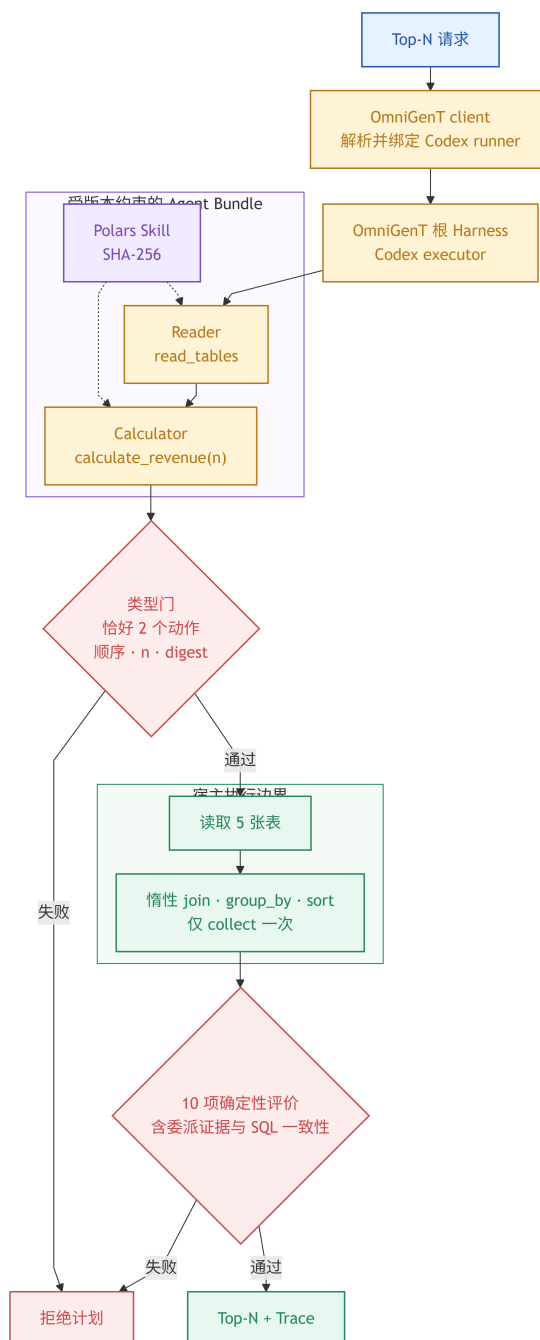


图 9.1: Level 4: OmniGenT 生成两个 Codex 子智能体，宿主白名单执行并独立验收

这里故意把“提出动作”和“执行动作”分开。即使模型生成了额外工具名、颠倒顺序、篡改 n ，或回传错误的 skill digest，计划也会在宿主触发数据库动作前被拒绝。

9.4 Polars skill 是运行输入，不是装饰标签

`load_polars_skill()` 会读取 `level4/polars/SKILL.md` 与五个规定的 reference 文件，检查 frontmatter、目录名、空文件、符号链接和越界路径，再对文件名与内容计算 SHA-256。宿主侧 worker definition 为 Reader 生成 lazy API + contexts 的 `base_instructions`，为 Calculator 生成 `contexts + expressions + insight recipe`；当前运行尚未单独消费这两个 role-specific 子集。实际 `build_agent_bundle()` 会把完整 skill 与全部 references 复制到根 agent 和两个子 agent 的作用域，并写入来源摘要。

因此，本层能够回答两个来源问题：**智能体依据了哪一版技能？两个动作是否确认了同一个摘要？**如果 skill 内容改变，摘要随之改变，旧动作计划不能悄悄复用。

9.5 宿主白名单：模型规划，代码授权

模型返回的不是 SQL 或 Python，而是严格的 JSON 动作计划。`WorkerAction` 把 agent 与 runtime tool 限定为两个字面值，并施加额外不变量：Reader 的 n 必须为空；Calculator 的 n 必须是正整数。随后 `_validate_plan()` 再检查：

- 评价结果已通过；
- 动作必须恰好是 Reader 后接 Calculator；
- 两个动作都携带当前 skill digest；
- Calculator 的 n 与用户请求相同。

`_execute_plan()` 只有 `read_tables` 和 `calculate_revenue` 两个执行分支；宿主动作层没有任意 SQL、shell、文件或 Python eval 分支。这就是 harness 的关键：模型拥有**提议权**，宿主代码保留**授权权与执行权**。

9.6 一次 collect 的 Polars 计算

宿主的 `read_tables` 分支先通过只读数据库适配器装载 `category`、`film_category`、`inventory`、`rental` 与 `payment` 五张表。随后，宿主的 `calculate_revenue` 分支把它们转换为 lazy frames，按四条 Sakila 外键路径依次连接，再执行分组、求和、排序：

```
summary = (  
    category.join(film_category, on="category_id")  
    .join(inventory, on="film_id")  
    .join(rental, on="inventory_id")  
    .join(payment, on="rental_id")
```

```

        .group_by("name")
        .agg(
            pl.col("amount").sum().round(2).alias("revenue"),
            pl.len().alias("_joined_rows"),
        )
        .sort(["revenue", "name"], descending=[True, False])
        .collect()
    )

```

整个 join、aggregation 与 sort 形成一个优化计划，只在末尾 collect() 一次。连接行数也在同一次聚合中计算，避免为审计统计额外收集中间大表。

9.7 实验：离线教材仍执行真实数据路径

```

from assets.book_setup import run_offline

level_04 = run_offline(4)
meta = level_04.metadata
{
    "generated_agents": meta["generated_agents"],
    "agent_calls": meta["agent_calls"],
    "delegation": [call["name"] for call in meta["omnigent_tool_calls"]],
    "skill_digest": meta["skill_digest_sha256"][:12],
    "joined_rows": meta["joined_rows"],
    "evaluation_score": meta["evaluation"]["score"],
    "omnigent_connected": meta["omnigent_connected"],
    "mode": meta["mode"],
}

```

```

{'generated_agents': ['Polars Table Reader', 'Polars Revenue Calculator'],
 'agent_calls': ['read_tables', 'calculate_revenue'],
 'delegation': ['read_tables', 'calculate_revenue'],
 'skill_digest': '97494980cda1',
 'joined_rows': 16044,
 'evaluation_score': 1.0,
 'omnigent_connected': False,
 'mode': 'offline-simulated-omnigent-codex'}

```

offline=True 只模拟 OmniGenT/Codex 的计划生成与委派事件：它构造相同的两个类型化动作与两条 executed_by="simulated" 的 tool-call evidence，并明确把 turn 标记为 simulated。五表读取、Polars join、Top-N 计算和 SQL 基线比较仍然真实执行。因此，离线输出可以证明数据与控制代码可复现，却不能证明本机 OmniGenT server 或 Codex harness 已成功联通；其 codex_threads_started 为 0。

9.8 十项评价：不是让生成者给自己打分

最终 evaluator 由宿主确定性代码运行，而不是把模型的自评当成结论。它逐项检查：

检查	拒绝的风险
与模式匹配的 OmniGenT delegation evidence	在线没有由 server 依次调用 Reader、Calculator，或离线模拟证据顺序错误
恰好两个 worker	偷偷增加或遗漏角色
skill provenance	两个动作依据不同技能版本
os_env 最小权限声明与 approvalPolicy=never	bundle 配置或 Codex executor 约束偏离预期；Windows 上仍不能据此推导 FS/network 已被硬隔离
动作顺序	未读表就计算或重复调用
五表完整性	缺表或空表仍继续
Top-N 形状	行数或类别重复
收入数值域与降序	NaN、负数或排序错误
连接基数	不期望的 join 爆炸或丢行
SQL baseline parity	Polars 结果与独立基线不一致

当前固定数据集应得到 16 个类别、16,044 条连接记录，并与 baseline/revenue.sql 精确到分一致。任何一项失败，harness 都 fail closed，不返回看似合理的排名。

9.9 实时模式：OmniGenT client 连接 Codex harness

实时路径必须显式设置 OMNIGENT_URL。代码优先使用 OMNIGENT_RUNNER_ID，否则通过 omnigent-client 查找声明 codex harness 的在线 runner；找到后才上传可复现的 tar.gz bundle，并把新 session 绑定到该 runner。bundle 包含根配置、两个子 agent 配置、三份作用域内的 Polars skill 和来源摘要。OmniGenT 以 Codex harness 运行本地 ornith-1.5:9b；StreamHooks 同时捕获 server 实际发出的两次子 agent tool call。只有调用顺序为 Reader、Calculator 且均由 server 执行，返回的 JSON plan 才会进入宿主的类型校验、白名单执行和十项确定性结果评价。

```
$env:OMNIGENT_URL = "http://localhost:6767"
# 可选：自动发现不可用或需要固定 runner 时设置
# $env:OMNIGENT_RUNNER_ID = "runner_..."
kungfu-agent run 4 --question "收入最高的 5 个影片类别"
```

! 没有静默降级

实时模式缺少 `OMNIGENT_URL` 或在线/显式 Codex runner 都会直接报错。若要运行可复现教材，必须明确使用 `offline=True`；系统不会把本地模拟伪装成 OmniGenT 已连接。

9.10 结论与升级门槛

Level 4 的新增控制权不是“再加一个 agent”，而是把角色数量、技能来源、动作集合、顺序、参数和验收条件一起写进 harness。它已经能拒绝坏计划与坏结果，却仍然只运行一次：失败不会自动变成修复动作。

当评价反馈应当进入下一次计划，系统需要重试预算、停止条件和可审计的迭代轨迹时，进入 Level 5。

9.11 章末练习

1. 修改 Polars skill 中一个 reference 后，为什么旧计划的 digest 必须失效？
2. 如果 OmniGenT 返回第三个动作，拒绝应发生在数据库读取之前还是之后？
3. SQL baseline 与 Polars 同时使用了错误的连接假设时，当前评价还能发现问题吗？还需要哪种独立证据？
4. 为什么 `offline=True` 能验证结果，却不能作为 Codex harness 在线联通的证据？

10 Level 5：让失败成为下一轮输入

10.1 能力验收实验

提前计算的错误提案被拒绝后，反馈必须进入重试，并最终完成 16 次探表与一次计算。持续错误耗尽每步重试预算后停止；拒绝记录与成功步骤分开审计。

kungfu-agent experiment 5

一招一句：循环的价值不在重复，而在评价、反馈、记忆和停止条件。

10.2 问题：怎样探索完整数据空间，并在目标达成时停下

Level 4 已经能约束一份固定的 Reader → Calculator 计划；Level 5 进一步控制跨轮轨迹。本章给智能体一个明确目标：先检查 Sakila 目录中的全部 16 张表，再根据发现的关系计算分类收入，验证成功后立即退出。评价器的输出不只是日志，而是下一轮必须确认已经消费的状态。

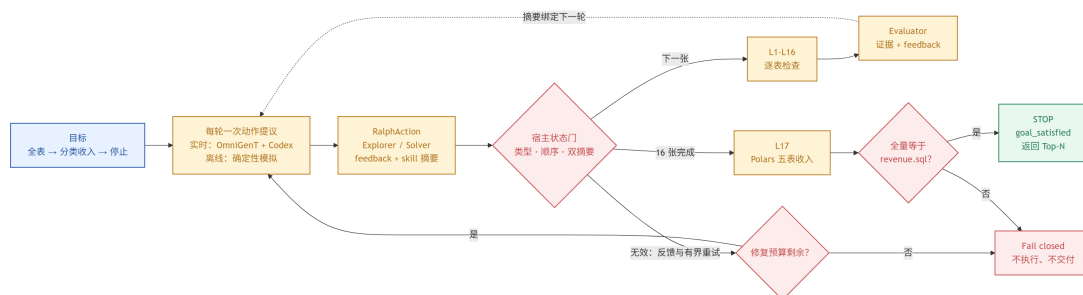


图 10.1: Level 5 的 16+1 Ralph loop：逐表探索、反馈绑定、收入验收与立即停止

实际状态机位于 `src/agent_kungfu/levels/level5/ralph_omnigent.py`。每轮保存一个 `RalphIterationRecord(iteration, action, evidence, evaluation, decision)`。RalphAction 只能是 `inspect_table` 或 `calculate_revenue`；模型提出动作，只有宿主能够读取 SQLite 或执行 Polars。宿主会把上一轮 feedback 规范化为 JSON 并计算 SHA-256，同时取得已加载 Polars skill 的 SHA-256；下一份 action 必须原样带回两份摘要，否则在执行前被拒绝。这既绑定循环状态，也绑定能力版本。

i 本例的确定性退出合同

1. 启动时冻结按名称排序的 SQLite 目录；当前数据库必须有 16 张唯一表。
2. 迭代 1-16 只能逐一检查下一张未见表。此时 `passed=false` 表示全局目标仍未完成，而不是表检查失败。
3. 只有完整覆盖后，迭代 17 才能调用 `calculate_revenue(n)`。
4. 只有完整 16 类 Polars 结果与 SQL baseline 相等、连接记录为 16,044 且 Top-N 形状正确时，才设置 `decision=stop` 与 `stop_reason=goal_satisfied`。
5. 成功后不运行第 18 轮；17 轮计有效步骤，每步另有最多 2 次提案修复。动作越界在执行前拒绝并反馈；修复预算耗尽、委派证据缺失或 baseline 不一致均抛错，不返回最后一个失败结果。

一个可治理的迭代回路要求会话先获得测试、构建或截图差异等验证信号，在人看到结果之前检查并修复错误；生产指标越过控制区间后，诊断结果还应成为下一份 `intent.md`，重新进入生命周期 (Claxton 2026 年)。Self-Refine 说明同一模型可以用显式反馈迭代改写输出，Reflexion 则把语言反馈保存为情景记忆供后续尝试使用 (Madaan 等 2023 年; Shinn 等 2023 年)。这些研究证明了反馈机制可以改善其评测任务，却不保证模型自评等于外部正确性；本层因此优先使用数据库执行结果和确定性规则作为 gate。

10.3 从单项修复到 PRD 与 SDLC 三层 LOOP

“给智能体一个反馈循环”不能只意味着“失败后再试”。把本章的窄域状态机推广到生产设计时，可以区分三个时间尺度：**单项内环**在一次会话中把验证失败变成修复输入；**PRD 聚合环**只在所有需求分别通过后声明 Done；**SDLC 运营外环**把发布后的异常变成新的意图和优先级。本仓库只实现前者的 16+1 Sakila 实例；后两者是设计延伸。单项通过不代表 PRD 完成，PRD 完成也不代表生产环境永久安全。

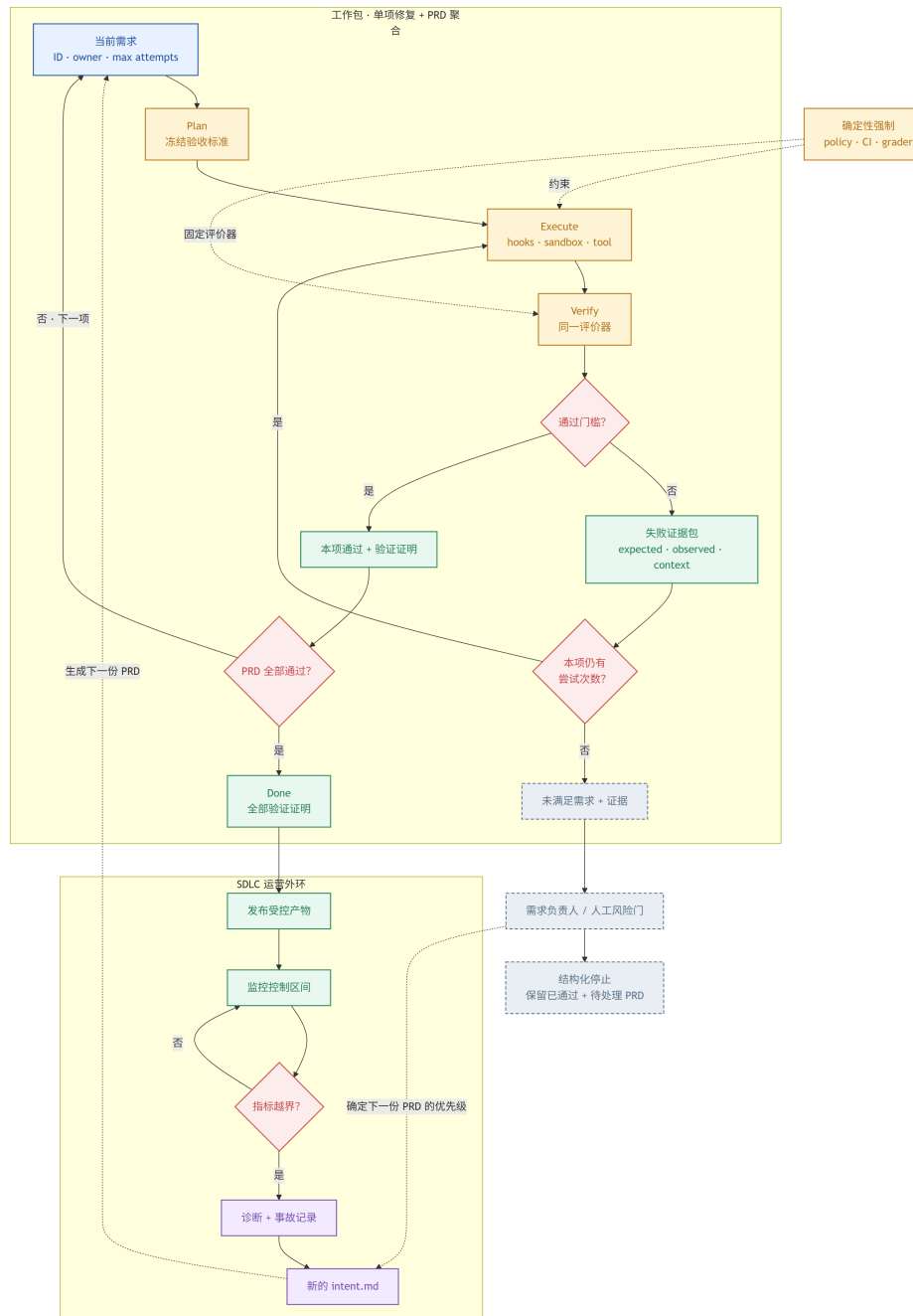


图 10.2: 三层反馈控制：单项证据回流、PRD 聚合完成与 SDLC 运营回写

失败只有经过结构化，才是下一轮能够消费的输入。一个最小失败证据包至少应保存 requirement_id、负责人、冻结的验收标准、当前尝试与 max_attempts、产物和环境版本、期望与实际结果、评价器及其版本、失败类别、可重试性、已通过与待处理需求，以及最终处置。SWE-bench 把真实 issue、仓库状态和可执行测试结合起来，也说明“任务描述 + 环境 + oracle”比一句自然语言批评更接近可复现的工程反馈 (Jimenez 等 2024 年)。Anthropic 的智能体评测指南进一步把任务、trial、transcript、outcome 与 grader 分开；本章据此把反馈视为可审计记录，而不是一段随手拼回 Prompt 的文字 (Anthropic 2026 年 a)。

下表把六项测试实践与维护阶段的回写机制，组织为 Level 5 可以验收的控制合同：

表 10.1: 反馈回路控制合同

循环实践	控制合同	必须保存的证据	当前 Sakila demo
把检查命令包装为单一目标，失败返回非零码 给出健康输出示例和量化目标	验证器必须机器可调用，成功与失败不可含糊 先定义 oracle 和通过线，再允许行动	命令、退出码、日志、环境版本 期望模式、阈值、grader 版本	十项确定性 harness check 任一失败即抛错 固定 16 表、17 轮预算、16,044 条连接记录和 SQL baseline
修 bug 前先写失败测试，且修复时不改测试	冻结验收标准，防止通过“改题”制造成功	测试先因预期原因失败，修复后同一测试通过	配置、类型合同和 revenue.sql 在循环外固定
UI 使用截图或浏览器闭环 报告完成前运行验证并展示输出	评价器必须观察任务真正交付的模式 单项进展只推进轨迹；Done 必须具有完整目标证明	基线、差异、容差和人工例外 action、evidence、evaluation、decision、最终 checks 和停止原因	不在只读 Sakila SQL 范围内 每轮记录四元组并保存完整表覆盖与 baseline 对账证据；尚无通用 PRD 聚合器
生产指标越界后诊断并生成新意图	外环把运行失败写回需求，不直接偷偷修生产	指标窗口、事故记录、intent.md、批准人	未实现生产监控和意图回写

10.4 实验：用 17 轮完成“探索—计算—停止”

离线轨道不请求模型猜一条 SQL。它先从真实 sqlite_master 固定目录，按字母顺序从 actor 走到 store。每次 inspect_table 只读取一行样本的列边界、真实 COUNT(*) 和可能连接收入链的信号列。全部 16 张表都有证据后，第 17 轮才执行 calculate_revenue。

```
import polars as pl
from assets.book_setup import run_offline

level_05 = run_offline(5)
iteration_rows = []
for item in level_05.metadata["iterations"]:
    iteration_rows.append(
        {
            "iteration": item["iteration"],
            "action": item["action"]["runtime_tool"].split("_")[0],
            "table": item["action"].get("table") or "—",
            "score": item["evaluation"]["score"],
            "decision": item["decision"],
            "feedback": "; ".join(item["evaluation"]["feedback"])[28],
        }
    )
```

```
)  
pl.DataFrame(iteration_rows)
```

表 10.2: Ralph loop 的 16 次表检查与第 17 次收入验收

iteration	action	table	score	decision	feedback
i64	str	str	f64	str	str
1	"inspect"	"actor"	0.0588	"continue"	"Coverage incomplete: inspect"
2	"inspect"	"address"	0.1176	"continue"	"Coverage incomplete: inspect"
3	"inspect"	"category"	0.1765	"continue"	"Coverage incomplete: inspect"
4	"inspect"	"city"	0.2353	"continue"	"Coverage incomplete: inspect"
5	"inspect"	"country"	0.2941	"continue"	"Coverage incomplete: inspect"
...	...	...	...	...	...
13	"inspect"	"payment"	0.7647	"continue"	"Coverage incomplete: inspect"
14	"inspect"	"rental"	0.8235	"continue"	"Coverage incomplete: inspect"
15	"inspect"	"staff"	0.8824	"continue"	"Coverage incomplete: inspect"
16	"inspect"	"store"	0.9412	"continue"	"All tables inspected; calcul"
17	"calculate"	"—"	1.0	"stop"	"Goal satisfied: every table "

默认 Top-5 的第一名是 Sports，收入为 5314.21；但 evaluator 不用第一行代替正确性证明。它把完整的 16 类 Polars 结果同独立的 baseline/revenue.sql 逐行比较，同时检查连接后 16,044 条记录。前 16 轮的分数表示覆盖进度，最后一轮通过后立即停止。没有 evaluator，“再试一次”只是随机重采样；没有 stop condition，成功后仍可能继续消耗资源甚至把正确答案改坏。

10.5 实时 loop 怎样工作

实时路径要求 OMNIGENT_URL，并优先使用 OMNIGENT_RUNNER_ID；若没有显式 ID，则只解析在线 Codex runner。omnigent-client 上传包含 root、Explorer、Revenue Solver 与 Polars skill 的可复现 bundle，绑定 session，然后每次提案调用 chat.query()。StreamHooks 必须观察到恰好一次预期工具调用，且 executed_by="server"；返回文本还必须通过严格的 RalphAction 解析、当前表/动作、feedback digest 与 skill digest 校验，宿主才会执行。无效提案转为失败反馈，每步最多修复 2 次；拒绝提案保留在 repairs，不计入 17 个有效步骤。服务权限或委派证据错误直接失败，不通过重试绕开。

离线轨道只把 提议与委派 替换成确定性模拟，并把每条 tool-call evidence 标为 executed_by="simulated"；它不会启动 Codex turn，也不能证明本机 OmniGenT server、runner 或 Ollama 已连通。表目录、逐表 SQLite 观测、五表 Polars 计算和 SQL baseline 对账仍然真实运行。实时环境缺少 URL 或在线 runner 时会明确失败，不会暗中回退到离线结果。

loops/sakila-improvement/RALPH.md 是可移植的人类可读规格；可执行状态机位于 src/agent_kungfu/levels/level5/ralph_omnigent.py。本例实现了窄域的 16+1 探索任务和停止合同，仍不是通用 PRD 聚合器，也不是已经持续运行的 AI 原生 SDLC。

10.6 评价器决定飞轮方向

最终 harness 运行十项确定性检查：目录快照、完整规范顺序、类型化动作轨迹、feedback 与 skill digest 绑定、与模式匹配的 OmniGenT/Codex 调用证据、两个只读 worker 的声明、逐表观测、收入形状、SQL baseline 一致性，以及有界立即停止。它适合作为教学 gate，却不能验证：

- join 是否造成重复计数；
- 指标定义是否符合业务口径；
- 结果是否对用户决策真正有用；
- 回答是否把观察性差异误写成因果关系。

因此“Ralph loop 已通过”只意味着通过了这个 evaluator，而不是获得普遍正确性。

! 生产提醒

当前实现对无效动作先拒绝执行并反馈重试；预算不足、重试耗尽、委派证据无效或最终十项检查不全通过都会抛出异常，不构造 LevelResult。若扩展到生产 PRD，还应把异常升级为包含 status、unmet_requirement、owner、attempts、max_attempts、evidence、accepted_requirements 和 pending_requirements 的持久停止记录。

10.7 结论与升级门槛

Level 5 适合可被机器评价、允许有限自动修复的任务。最重要的工程资产通常不是 Prompt，而是评价器、失败分类、预算和停止规则。

当修复需要理解真实表之间的关系，而且策略、行数和协议也要变成类型约束时，进入 Level 6。

10.8 章末练习

1. 让第 5 轮再次检查 actor，解释为什么拒绝必须发生在新的 SQLite 查询之前。
2. 为什么预算必须至少为 `table_count + 1`，又为什么通过第 17 轮后不能继续第 18 轮？
3. 设计一个 exhaustion stop record，说明负责人会收到哪些目录、feedback digest、委派和 baseline 证据。

Part III

第三部 从数据契约到知识生产：让系统理解关系并积累经验

11 Level 6：把规则 and 关系写进类型与图

11.1 能力验收实验

外键图决定 JoinPlan，任务 DAG 调度读取、连接与聚合。删除 film_category→category 关系必须阻止计算；完整图的 Polars 收入必须匹配 SQL 基线。

kungfu-agent experiment 6

一招一句：Harness schema 管规则，schema graph 管关系，evaluator 管最低质量。

11.2 问题：反馈知道结果差，却不知道表怎样相连

Level 5 的 evaluator 可以说“缺少决策指标”，但 SQL planner 若不了解 payment 如何连到 film，只能继续在 schema 文本里搜索。与此同时，max_rows=50、只读模式和最多三轮若只是散落的参数，也难以审核。

Level 6 同时提升两个控制面：

1. 用 Pydantic HarnessManifest 类型化协议、行数、迭代和证据要求；
2. 从真实 DDL 外键构建 NetworkX 有向图，恢复 join path。

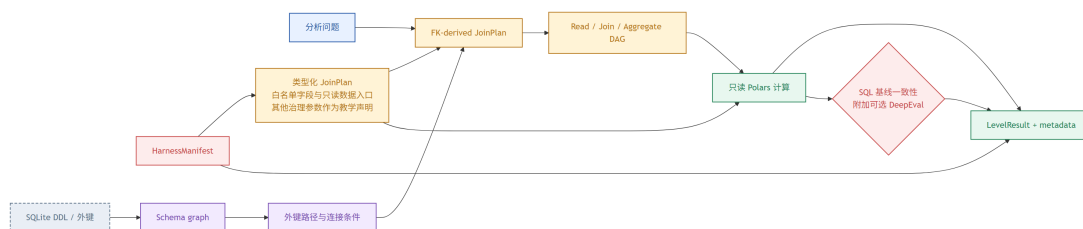


图 11.1: Level 6 的契约、关系证据与评价控制面

Pydantic 会在对象边界验证字段约束 (Pydantic, 不详)，NetworkX 提供图算法与结构查询 (NetworkX Developers, 不详)。

11.3 从技能目录到功能图

当任务从“调用一个能力”升级为“交付一个业务结果”，单个 Skill 很快不够用；Skills、MCP、hooks、subagents、Agent Teams 和 Plugins 必须围绕同一个功能合同协同。这里关注的是依赖、类型、权限与验收机制，而不是用组件数量或市场规模推断系统能力。

功能不是一个更长的 Skill。 Skill 是局部、可复用的过程知识；功能是面向用户结果的承诺。一个复杂功能必须把多个技能组织成依赖图，并同时规定：

1. 每个技能消费和产生什么类型；
2. 哪些边是数据依赖，哪些边是控制依赖；
3. 哪些节点可以并行，哪些节点必须等待验收；
4. 每个节点需要哪些权限、预算和副作用边界；
5. 局部失败如何回滚、替代或升级；
6. 整体功能用什么证据判定完成。

HuggingGPT 把复杂请求分解为子任务，再依据功能描述选择模型、执行并汇总；MetaGPT 用角色和标准作业流程组织多智能体的中间产物；AutoGen 允许开发者用代码与自然语言定义 agent、工具和人工参与者的交互模式 (Shen 等 2023 年; Hong 等 2023 年; Wu 等 2023 年)。这些研究支持“复杂功能需要显式编排”的机制判断，却不能推出“技能越多，结果必然越好”：错误会沿依赖边传播，交接格式不兼容也会让每个局部技能都成功、整体功能仍失败。

下面用本书已有的 Pydantic 类型、Sakila DDL graph、SQL guard、Polars 结果和 evaluator，画出一个“门店库存改进建议”功能需要的技能协作图：

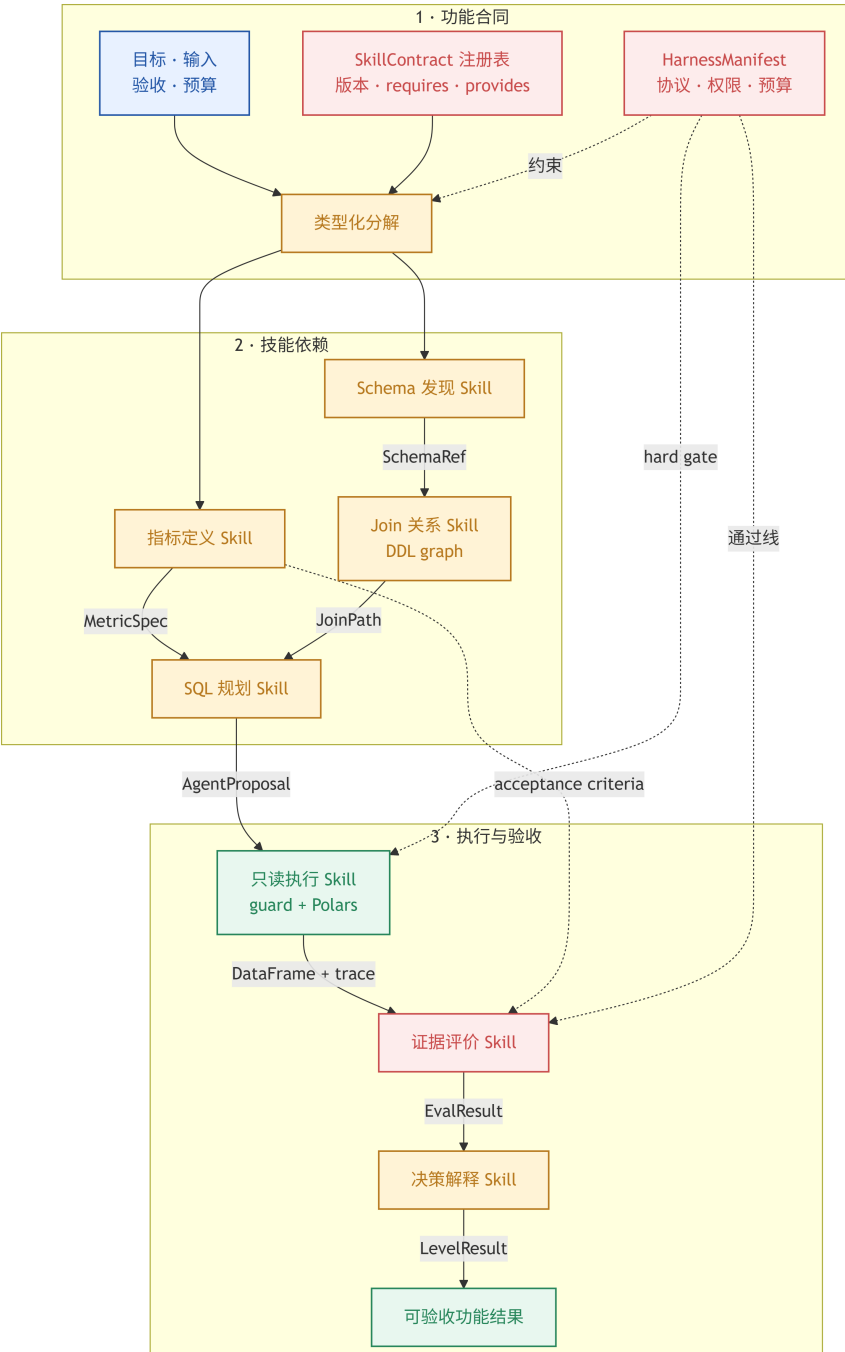


图 11.2: 复杂功能的技能依赖图：局部能力通过类型化边和统一验收组成业务结果

粒度、封装和治理必须分开建模。下面把相关概念整理为五种工程对象，避免把功能、插件和市场混成同一个概念：

表 11.1: 技能—功能—分发对象矩阵

工程对象	本书中的重构定义	必须机器化的合同	Level 6 当前状态
Skill	完成一个局部步骤的可复用过程知识	输入、输出、版本、权限、有技能与函数，但没有统一失败模式	— SkillContract 注册表

工程对象	本书中的重构定义	必须机器化的合同	Level 6 当前状态
Tool / MCP	让技能到达外部能力的调用边	协议、参数 schema、身份、超时、副作用	allowed_protocols 只写入 metadata，尚未 hard gate
Function	对用户结果负责的技能依赖图	DAG、类型化边、预算、回滚、整体验收	本节提出合同模型；runtime 尚未执行功能图
Plugin / package	分发技能、hooks、agent 和配置的版本化载体	依赖、兼容版本、签名、升级与撤回	本项目没有插件打包或安装生命周期
Marketplace	对多个包进行发现、准入和退役治理	所有者、审核、权限、成本、评分、弃用	超出 Level 6；市场存在不等于组合正确

功能不等于插件。功能描述“系统必须完成什么并如何验收”，插件描述“代码和配置如何被分发”。一个插件可以提供多个功能，一个功能也可能跨越多个插件。只有功能图及其类型化边可以回答“这些技能能否安全地拼起来”。

以“哪个门店应该改善哪些影片库存”为例，最小协作合同可以写成：

表 11.2: Sakila 复杂功能的多技能类型化协作合同

Skill	消费类型	产生类型	失败门与责任
指标定义	问题、业务口径	MetricSpec	指标不可计算或口径未批准时转人工
Schema 发现	SQLite DDL	SchemaRef	引用不存在的表或列时拒绝
Join 关系	SchemaRef、DDL graph	JoinPath	无路径或多条歧义路径时不得猜测
SQL 规划	MetricSpec、JoinPath	AgentProposal	类型不兼容或缺少决策指标时回到规划
只读执行	AgentProposal、policy	DataFrame + trace	写操作、超时或超行数立即失败
证据评价	结果、验收标准	EvalResult	未通过时进入 Level 5 回路或耗尽策略
决策解释	EvalResult、证据行	LevelResult	不得把观察差异表述为因果结论

当前代码已经分别拥有 AgentProposal、EvalResult、LevelResult、HarnessManifest 和 schema graph，但没有 SkillContract 类型，也没有执行技能依赖 DAG。所以上图是基于现有构件的**可实现设计**，不是对 runtime 已有能力的宣称。Level 6 真正新增的思想是：数据表之间的关系进入 DDL graph，技能之间的关系也应进入 dependency graph；两类图必须由类型和 gate 连接，不能只靠 Prompt 约定。

11.4 实验一：类型化 manifest

```

from pydantic import ValidationError
from agent_kungfu.contracts import HarnessManifest, Protocol

manifest = HarnessManifest(
    name="sakila-graph-harness",
    allowed_protocols=[Protocol.SKILL, Protocol.MCP, Protocol.A2A],
)
try:
    HarnessManifest(
        name="unbounded",
        allowed_protocols=[Protocol.SKILL],
        max_iterations=11,
    )
except ValidationError as error:
    invalid_manifest_error = error.errors()[0]["msg"]

{
    "valid": manifest.model_dump(mode="json"),
    "invalid_max_iterations": invalid_manifest_error,
}

```

```

{'valid': {'name': 'sakila-graph-harness',
  'version': '1',
  'allowed_protocols': ['skill', 'mcp', 'a2a'],
  'read_only': True,
  'max_rows': 50,
  'max_iterations': 3,
  'conflict_policy': 'safe-executable-highest-confidence',
  'evidence_required': True},
  'invalid_max_iterations': 'Input should be less than or equal to 10'}

```

类型约束把错误从“运行到第十一轮才发现”提前到配置构造时。不过，当前 `allowed_protocols` 只出现在 metadata，还没有连接 `ProtocolBus` 做运行时 enforcement；它是明确契约，不是已经完成的 policy engine。

11.5 实验二：从 DDL 恢复关系

```
from assets.book_setup import DB, run_offline
from agent_kungfu.graph_engineering import (
    build_schema_graph,
    graph_summary,
    shortest_join_path,
)

schema_graph = build_schema_graph(DB)
level_06 = run_offline(6)
{
    "summary": graph_summary(schema_graph),
    "payment_to_film": shortest_join_path(schema_graph, "payment", "film"),
    "evaluation": level_06.metadata["evaluation"],
}
```

```
{'summary': {'tables': 16,
  'relationships': 21,
  'components': 2,
  'most_connected': [('store', 5),
    ('staff', 5),
    ('address', 4),
    ('customer', 4),
    ('film', 4)],
  'cycles': [['staff', 'store']],
  'payment_to_film': ['payment', 'rental', 'inventory', 'film'],
  'evaluation': {'passed': True,
    'score': 1.0,
    'evaluator': 'graph-baseline-parity',
    'feedback': []}}
```

当前数据库产生 16 个节点、21 条外键边和 2 个连通分量。payment → rental → inventory → film 是图转为无向后得到的最短路径；方向被忽略，是因为 SQL join 可以沿外键任一方向表达。

当前 level6/graph_agent.py 从外键路径生成 JoinPlan，再按任务图实际执行 Polars 连接与聚合。删除关键外键会阻止规划；结果必须与独立 SQL 基线一致。最短路径本身仍不保证业务口径正确，因此保留基线作为独立验收。

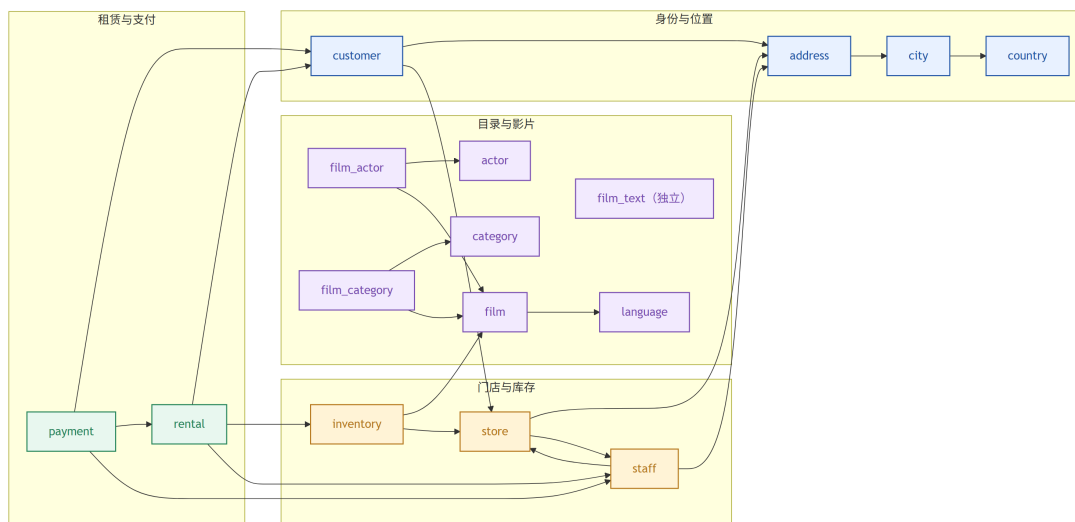


图 11.3: 从 SQLite DDL 提取的 Sakila schema graph

11.6 Graph、lineage 与 ontology 不相同

这里的 graph 只解析 DDL 中符合正则的显式 FOREIGN KEY ... REFERENCES ...。它不是：

- SQL 查询 lineage；
- 业务概念图；
- 带推理规则的 ontology；
- 自动证明正确 join 的优化器。

图能回答“哪些表有声明关系”和“最短连接路径是什么”，不能回答“这条路径是否符合当前业务口径”。

11.7 确定性 gate 与 DeepEval

Level 6 默认执行完整收入结果与 SQL 基线的一致性验证。只有 `offline=False` 且显式 `use_deepeval=True` 时，才附加本地 `DeepEvalBaseLLM`，用 `ornith-1.5:9b` 运行 `GEval`；其结果保存在独立 `deepeval` 字段，不替代数值验收 (Confident AI, 不详)。

确定性 gate 便宜、稳定、易定位；LLM judge 能评价自然语言相关性和因果措辞，但更慢、会漂移，也可能与被评模型共享偏差。生产系统通常需要组合两者，而不是二选一。

11.8 结论与升级门槛

Level 6 把关系提升为可执行连接计划，并用任务依赖组织计算。当前实验覆盖类别收入链；更广的业务指标和多智能体任务图仍需额外设计与验证。

但 graph 每次运行都从 schema 重建，系统不会记住解释、口径和历史观察。当知识必须跨运行复用时，进入 Level 7。

11.9 章末练习

1. 为什么最短 join path 不一定是正确的业务 join path?
2. 怎样把 `allowed_protocols` 从 metadata 变成运行时 hard gate?
3. DeepEval 与确定性 evaluator 各自更适合发现哪类错误?

12 Level 7: 让系统记得昨天

12.1 能力验收实验

同一任务第二次运行应检索已验证查询知识。修改隔离数据库副本后，数据指纹变化，系统重新验证并保留新的知识版本；本实验的检索范围是相同任务。

```
kungfu-agent experiment 7
```

一招一句：把易失的运行上下文变成可检查、可版本化的 Markdown 知识快照。

12.2 问题：图每次都能重建，但解释每次都要重来

Level 6 能从 DDL 重建 schema graph，却没有跨运行记忆。模型每次都要重新接收 schema、表规模和关系摘要；团队也无法对某次知识状态做 diff、review 或引用。

Level 7 的 `generate_wiki()` 把结构化事实写成普通 Markdown：

- README.md：表索引和行数；
- 每个表一页：行数、相邻表和 DDL；
- 基础 17 页：1 个索引 + 16 个表页；
- 每个已验证任务版本另存 `experience-*.md` 和 JSON；实时模型读取检索到的经验页解释证据。

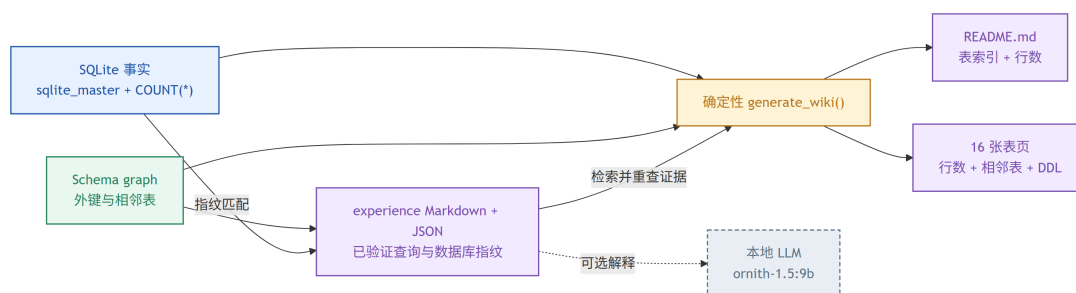


图 12.1: 从数据库事实到持久 Wiki，再到可选 LLM 概览

12.3 开源知识生产的三条架构路线

本章选取 OpenKB、SAG、axiom-wiki、deepwiki-open、Graphiti、GraWiki、Knowledge Catalog、llm-wiki-agent、llmwiki、obsidian-wiki 和 Synthadoc 作为架构样本。它们没有共同复现实验、统一数据集或一致成本口径，因此适合用于发现**架构模式**，不适合排出性能名次；项目自报数字也不作为独立效果证据。

这些实现可以组织为三条路线。第一条把文档或代码在摄取阶段编译成互链页面；第二条把事件、实体、记忆和时间关系写入图结构；第三条把知识生产提升到企业目录、治理和联邦数据源层。它们的共同点不在存储介质，而在于把知识从一次回答的副产品，升级成有维护生命周期的系统资产。

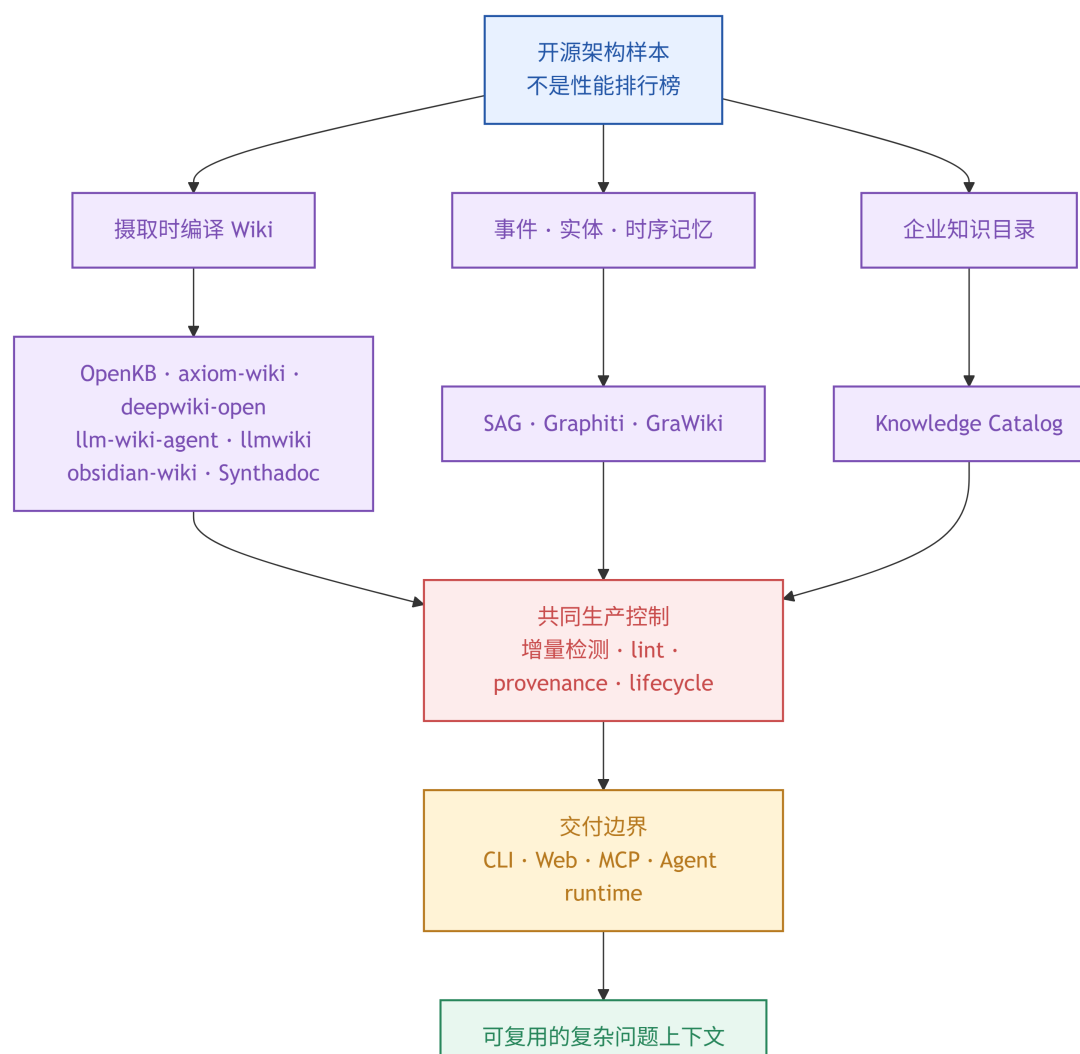


图 12.2: 开源 LLM-Wiki 实现呈现的三条知识生产路线

下面按 Level7 关心的“知识如何产生、维护和交付”统一编码定位、检索结构、存储、接口和维护控制：

表 12.1: 开源项目知识生产矩阵

项目	知识生产单元	主要维护控制	交付边界	架构路线
OpenKB	长文档层级索引与互链页面	文件监听、Wiki Foundation	CLI、Skill 导出	编译 Wiki
SAG	事件—实体检索单元	检索轨迹、双模式召回	Web、MCP	轻量关系检索
axiom-wiki	Markdown 页面与混合索引	哈希增量、确定性与 Agent 两级维护	CLI、REPL、MCP	编译 Wiki
deepwiki-open	代码库说明与架构页面	仓库分析、多轮研究	Web、问答	代码 Wiki
Graphiti	带有效时间的实体关系	增量构图、episode 溯源	Python、MCP、REST	时序记忆图
GraWiki	图实体与 Agent 记忆节点	去重、后端抽象	Python facade	图记忆
Knowledge Catalog	企业数据语义与元数据	治理、策略、血缘	Agent、MCP、工具包	企业目录
llm-wiki-agent	front matter 页面	确定性 health 与语义 lint 分离	宿主编码 Agent	Agent-as-Runtime
llmwiki	VaultFS 页面与批注	定时维护、受保护页面	Web、浏览器、MCP	产品化 Wiki
obsidian-wiki	Vault 页面、链接与 manifest	Dream Cycle、dry-run、staged writes	多种编码 Agent	Agent-as-Runtime
Synthadoc	摄取时编译的引用页面	多 Agent 编排、成本护栏、lifecycle	CLI、Obsidian、Web、MCP	可信编译 Wiki

这张表是架构分析图，不是共同环境中的复现实验。项目版本、接口覆盖和产品状态会变化；未在同一环境复现的字段，不承担“谁更先进”的证据功能。

12.4 为什么知识驱动的生产正在进入复杂问题求解

知识生产不是生成更多文本。它是把来源转成可以复用的事实、链接、摘要、冲突记录与版本，并让后续 agent 能按问题选择它们。这里也不是 RAG 的简单替代：RAG 将参数记忆和外部非参数记忆分开，在查询时检索证据 (Lewis 等 2020 年)；LLM-Wiki 路线进一步把一部分结构化、归并和检查工作前移到摄取与维护阶段。

研究路径呈现出三个连续变化。Generative Agents 把经验记录、检索、反思和计划连接起来，说明跨时段行为需要可积累记忆 (Park 等 2023 年)；MemGPT 用分层记忆和上下文换入换出处理窗口限制 (Packer 等 2023 年)；GraphRAG 在查询前构建实体图和社区摘要，以支持面向整套语料的全局问题 (Edge 等 2024 年)。Zep/Graphiti 又把时间有效性和历史关系纳入知识图，使“昨天为真、今天失效”可以被表示 (Rasmussen 等 2025 年)。

由此可以得到一个有限但重要的趋势判断：问题越需要跨文档综合、跨会话连续性、全局主题归纳和变化追踪，系统就越不能只在最后一次 Prompt 中临时拼上下文。它需要一条持续生产知识资产的管线，并把摄取成本换成查询时的可复用性。这个判断不意味

着所有问题都要建 Wiki；短问题、低复用信息和快速变化且无法验证的来源，仍可能更适合即时检索。

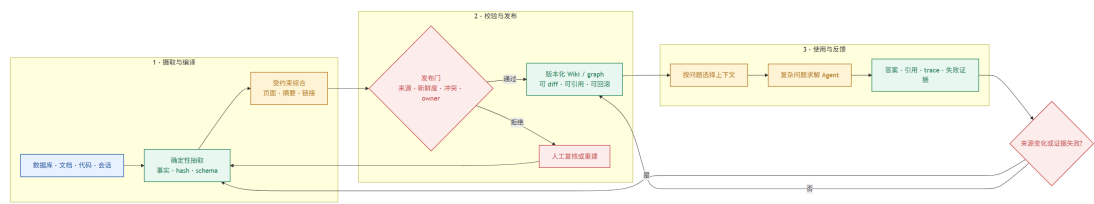


图 12.3: 复杂问题求解中的知识生产—使用—反馈闭环

表 12.2: 知识驱动生产闭环的控制对象与 Level 7 实现边界

生产阶段	必须产出的工程对象	最小发布门	本项目 Level 7 当前状态
来源登记	source URI、时间、内容 hash	来源可定位且格式可读	只有本地 SQLite 路径
事实编译 语义综合	表、行数、DDL、关系 页面摘要、业务解释、跨页链接	可从数据库重新计算 引用事实且标记解释层	已确定性生成 只在实时模式生成概览
关系组织 质量校验	页面链接或知识图边 lint、冲突、新鲜度、owner	边可追溯、无悬空目标 失败不得发布为可信知识	仅有 schema 邻接关系 尚未实现
版本发布	snapshot ID、diff、rollback	通过校验且保留上一版本	同名文件直接覆盖
问题使用与反馈	context pack、引用、trace、失败证据	只选择相关页面；失败可触发重建	当前按相同任务与数据指纹检索经验

上图和表格描述的是基于现有 `generate_wiki()` 可以演进的工程目标。当前 `runtime` 没有检索器、版本仓库、冲突检查或反馈触发器；这是**可实现的升级目标，不是当前 `runtime` 已具备的完整闭环**。

12.5 实验：从数据库生成知识快照

```
from pathlib import Path
from assets.book_setup import run_offline

level_07 = run_offline(7)
wiki_paths = [Path(path) for path in level_07.artifacts]
{
    "wiki_pages": level_07.metadata["wiki_pages"],
    "all_exist": all(path.is_file() for path in wiki_paths),
    "index": str(wiki_paths[0]),
    "evaluation": level_07.metadata["evaluation"],
}
```

```
{'wiki_pages': 18,
  'all_exist': True,
  'index': 'D: \ \Python\ \agent_kungfu_levels\ \outputs\ \book\ \level07_wiki\
\README.md',
  'evaluation': {'passed': True,
  'score': 1,
  'evaluator': 'retrieved-query-baseline-parity'}}
```

```
index_lines = wiki_paths[0].read_text(encoding="utf-8").splitlines()
index_excerpt = "\n".join(index_lines[:10])
customer_page = next(path for path in wiki_paths if path.name == "customer.md")
customer_lines = customer_page.read_text(encoding="utf-8").splitlines()
customer_excerpt = "\n".join(customer_lines[:12])
print(index_excerpt)
print("\n--- customer.md ---\n")
print(customer_excerpt)
```

Sakila Knowledge Wiki

Generated from the live SQLite schema.

- [actor](actor.md) — 200 rows
- [address](address.md) — 603 rows
- [category](category.md) — 16 rows
- [city](city.md) — 600 rows
- [country](country.md) — 109 rows
- [customer](customer.md) — 599 rows

--- customer.md ---

customer

Rows: 599

Related tables: address, payment, rental, store

Schema

```sql

```
CREATE TABLE customer (
 customer_id INT NOT NULL,
```

```
store_id INT NOT NULL,
```

这些文本不是缓存的模型答案，而是从当前 `sqlite_master`、外键图和实时 `COUNT(*)` 确定性生成。`customer.md` 的 599 行可以回查数据库；表关系可以回查 DDL。

## 12.6 为什么 Markdown 是有价值的中间层

Markdown 并不神奇，价值来自它同时服务三类读者：

1. 人可以在代码审查中阅读和讨论；
2. 版本控制可以显示 schema 与行数变化；
3. LLM 可以把索引和相关页面作为显式上下文。

与每次把完整 schema 塞进 Prompt 相比，wiki 让知识有文件名、更新时间和审查边界。它也让“模型上下文”变成可以离线构建的工程资产。

## 12.7 “LLM Wiki” 需要准确解释

### i 实现事实

基础 17 页由确定性代码生成。新增经验记录保存任务、数据库指纹、生成器版本、已验证 SQL 与知识说明。实时模型读取检索到的经验 Markdown；离线模式也执行相同的知识检索与数值验证。

没有匹配经验时，系统调用 Level 6 的图驱动规划并保存已验证查询；命中时复用该查询，重新读取数据库并对照基线。任务或数据库指纹变化时重新学习。

## 12.8 知识积累的新风险

持久化同时引入知识漂移：

- 基础 schema 页仍覆盖更新，经验记录按任务与数据指纹保留版本；
- 当前检索只覆盖相同任务，不提供语义相似检索或跨问题迁移；
- 新数据版本使旧经验失效，但还没有人工审核与版本合并流程；
- LLM overview 是解释性内容，不能和确定性事实享有同等信任。

生产系统应给每页加入 `provenance`、`freshness`、`owner` 和 `validation status`，并在发布前比较 schema fingerprint。

## 12.9 结论与升级门槛

Level 7 适合需要跨运行复用 schema 说明、业务口径和分析背景的系统。知识第一次成为独立于 Prompt 的持久资产。

但 Markdown 页面主要描述“数据库里有什么”，不能严格表达类、属性、domain/range 与映射约束。当知识需要可交换语义结构时，进入 Level 8。

## 12.10 章末练习

1. 为 wiki 页面设计最小 provenance front matter。
2. 怎样检测 customer.md 已落后于数据库？
3. 哪些内容应由确定性生成，哪些适合 LLM 起草后人工审核？

## 13 Level 8：从知识记录到知识生产

### 13.1 能力验收实验

关系候选必须通过 OWL 类、domain/range、外键映射与数据库计数验证才能进入语义知识页。错误值域应阻止发布；当前校验不等于完整 OWL 推理或 OBDA 执行。

kungfu-agent experiment 8

一招一句：Wiki 方便阅读，ontology 让类、属性和映射成为机器可交换结构。

### 13.2 本体工程的开源实现版图

本章审计的开源实现覆盖生成、学习、对齐、需求协作、信息抽取与综述。审计检查了 18 个 GitHub 仓库，并把 OntoGenix 主程序与实验仓库视为一个项目族。2026 年 8 月 31 日的核验确认这些地址均可访问，但“仓库存在、仍有更新”只说明可检查性，不证明生产成熟度或效果领先。

下面的表用三个问题比较这些实现：它生产哪一种语义资产？它改变了知识生产的哪个环节？它的证据边界在哪里？相关论文支持几个代表性判断：LLMs4OL 把学习拆为术语分类、分类体系发现和非分类关系抽取，但原论文同时警告基础模型尚不足以独立承担高推理、强领域知识的本体构建 (Babaei Giglou 等 2023 年)；OntoGPT/SPIRES 用既定 schema 与外部词表约束零样本抽取 (Caufield 等 2024 年)；DeepOnto 与 OLLM 分别代表神经—符号工具链和整体子图学习 (He 等 2023 年; Lo 等 2024 年)；OntoAligner 则把轻量匹配、检索、RAG 与 LLM 对齐包装为可替换模块 (Babaei Giglou 等 2025 年)。

表 13.1: 开源本体项目的语义资产、知识生产环节与证据边界

| 项目或项目族                            | 重写后的主要任务                             | 对知识生产的贡献                          | 证据与适用边界                                    |
|-----------------------------------|--------------------------------------|-----------------------------------|--------------------------------------------|
| Ontologies_by_LLMs                | 比较 one-shot、分解提示和 X-HCOME 人机协作生成     | 把专家反馈放回概念化循环                      | 临床案例研究；不能外推为所有领域的自动化排名                     |
| LLM-Automatic-Ontology-Generation | 在金标准上判断 broader、narrower、same-as 等关系 | 把“像不像”转为可计量的关系判别                  | 是关系评测而非完整 ontology generator；模型间数字依赖同一实验设置 |
| OntoAligner                       | 组合字符串、检索、LLM、RAG、few-shot 与 KGE 对齐   | 为异构概念生成可替换、可复现的 mapping candidate | 模块化工具包；候选映射仍需领域校验                          |

| 项目或项目族                         | 重写后的主要任务                                        | 对知识生产的贡献                          | 证据与适用边界                                         |
|--------------------------------|-------------------------------------------------|-----------------------------------|-------------------------------------------------|
| OntoGPT / SPIRES               | 按 LinkML/schema 递归抽取，并接地到外部词表                   | 把文本片段变成带标识符的结构化 assertion         | grounding 降低标识歧义，但不保证原文本、关系或因果解释为真              |
| automatic-KG-creation-with-LLM | 用 competency questions 驱动论文到 KG/ontology 的构建与回查 | 让预期问题成为设计和验收条件                    | 闭环回答 CQ 证明任务可用性，不等于逻辑完备或领域批准                    |
| Agent-OM / ontology-llm        | 以 retrieval agent 和 matching agent 分工           | 把大规模候选召回与精确语义判断解耦                 | Agent 化改善流程边界；向量召回与 LLM 判断都可能漏配或误配              |
| Onto-Generation                | 按 CQ 生成局部 OWL，再合并片段                             | 用局部上下文减少一次性长提示的干扰                 | 研究型生成策略；合并后仍需一致性和冗余检查                           |
| OntoGenix + Experiments        | GUI 引导 CSV 到 OWL/RML，并比较模型与质量指标                 | 让非专家能逐步修订实体、关系和映射                 | GUI 降低门槛，不替代 ontology engineer；主仓库和实验仓库按一个项目族计数 |
| OntoLearner                    | 用统一 API 组织 term typing、taxonomy 和关系学习           | 把零散脚本变成可复现任务与数据接口                 | 数据覆盖广不等于跨域正确；仍受 benchmark 与标签定义约束               |
| OntoChat                       | 通过对话和 competency questions 获取需求                 | 将 LLM 从“生成器”改为访谈者与建模顾问            | 对话产出是需求候选，不是已批准 axiom                           |
| LLMs4OL                        | 建立术语分类、层级发现和非分类关系三类任务                           | 为 ontology learning 提供共享任务语言和评测入口 | 原研究不支持“基础 LLM 已可独立建本体”的结论                       |
| DeepOnto                       | 组合 OWLAPI 推理、规范化、投影与深度模型                        | 连接形式语义与可学习表示                      | 是综合工程包，不是纯 LLM prompt 系统；逻辑与统计模块各有失败模式          |
| llm4oe-slr                     | 组织 LLM 本体工程论文、任务和证据                             | 为项目生态提供元层 taxonomy                | 是综述数据仓库而非可运行生成器                                 |
| OLLM                           | 微调整体生成 taxonomy 子图并使用结构指标                       | 从逐边判断推进到结构共同学习                    | 重点是分类骨架，不覆盖完整公理、治理和实例真值                         |
| LLMs4OM                        | 用 RAG 与父子概念上下文进行 ontology matching              | 证明概念表示粒度是匹配控制变量                   | 匹配结果是 correspondence proposal，不是自动合并授权          |
| SovereignOntologyLLM           | 本地 RDF/SHACL 查询后由 LLM 转述                        | 展示“事实先入受控结构、模型后表述”的可审计思路          | 个人探索项目，缺少与成熟系统可比的独立评测；量子路由不作为有效性证据              |
| OntoEKG                        | 将实体/属性抽取与层级蕴含分给不同模型                             | 把发现与逻辑归类拆成可替换阶段                   | Notebook 级流水线；多模型分工不自动消除误差传播                    |

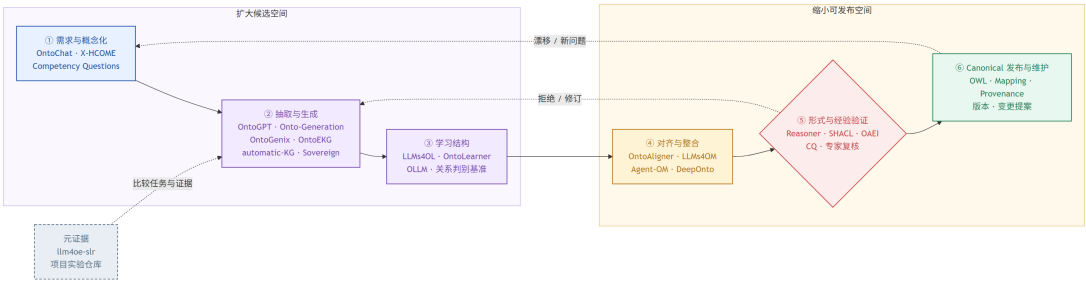


图 13.1: 开源实现呈现的 本体知识生产生命周期：项目控制不同环节，而不构成一条性能排行榜

13.3 趋势：从生成文件转向治理知识生产生命周期

2026 年的系统综述分析了 36 篇论文、49 个任务级研究，发现 LLM 已进入需求、实现、发布和维护等环节，但任务定义、数据集、指标和实验协议仍不统一，跨论文直接比较通常没有意义 (Li 等 2026 年)。因此，下表只呈现演化方向，而不构造冠军榜。

表 13.2: 本体工程的六条控制迁移

| 从什么迁移              | 迁移到什么                             | 代表项目证据                                       | 新增控制问题                     |
|--------------------|-----------------------------------|----------------------------------------------|----------------------------|
| 一次 Prompt 生成整份 OWL | CQ 分解、递归抽取、局部生成和迭代合并              | OntoGPT、<br>Onto-Generation、<br>automatic-KG | 每个片段由哪个问题触发，怎样合并和回滚？       |
| 单点术语或关系预测          | taxonomy 子图与任务间结构共同学习             | LLMs4OL、OntoLearner、<br>OLLM                 | 节点正确但整体层级错误时怎样计量？          |
| 单模型端到端包办           | retrieval/LLM、双 Agent、神经—符号和多模型分工 | OntoAligner、Agent-OM、<br>DeepOnto、OntoEKG    | 阶段误差怎样传播，哪一层拥有最终裁决权？       |
| 只追求可生成             | grounding、reasoner、SHACL、CQ 和专家复核 | OntoGPT、DeepOnto、<br>OntoGenix               | 语法合法、逻辑一致、数据合规和业务正确分别由谁证明？ |
| 自动化替代专家            | 非专家参与、专家闸门、candidate/canonical 分层 | X-HCOME、OntoChat、<br>OntoGenix               | 哪些概念是事实发现，哪些是组织选择或规范决定？    |
| 静态文件作为交付物          | 可映射、可查询、可追溯、持续修订的控制面              | RML/OBDA、PROV、<br>Agent pipeline             | 谁在何时依据什么证据发布了哪一版知识？        |

这六条迁移共同说明：本体发展并不是“让 LLM 写出更多三元组”，而是让模型负责扩大候选空间，让形式结构、验证器、来源和人类负责缩小可发布空间。模块化会成为关键，因为抽取、概念化、对齐、验证和发布依赖不同证据，不能由一次语言生成同时证明 (Shimizu 和 Hitzler 2025 年)。

13.4 认识论重组：本体不是知识本身，而是对可说之物的显式承诺

Gruber 的经典定义把 ontology 表述为对 conceptualization 的显式规格；这意味着 ontology 首先说明一个系统承认哪些类、关系和约束，而不是宣称自己已经穷尽现实 (Gruber 1993

年)。形式本体研究进一步强调，信息系统一旦使用某组类别解释记录，就已经作出了本体承诺；知识组织因此从词语排序进入了对象身份、类型、关系、时间和公理的共同建模 (Guarino 1998 年; Herre 2013 年)。

从认识论看，最重要的变化是把“流畅陈述”拆开。数据库值是观测记录；带时间、主体和来源的句子是 assertion；类型与关系把 assertion 放进共享概念框架；逻辑、shape、映射测试和专家审查才提供不同种类的 justification。这个分层是本书的工程化定义，不声称解决哲学上关于 knowledge 的所有争论。**本体重新组织信息，但不会自动把信息升级为真理。**

表 13.3: 从数据、信息到可发布知识的本体化重组

| 认识层                        | Sakila 示例                          | 本体化后的组织方式                                  | 发布前必须回答的问题                     |
|----------------------------|------------------------------------|--------------------------------------------|--------------------------------|
| 观测/数据                      | payment.amount = 5.99              | 保留值、字段、记录 ID 与采集时间                         | 记录来自哪个系统，是否缺失、重复或被修订？          |
| assertion/信息               | “付款 P 在时间 T 记录金额 5.99”             | 把值绑定到事件、主体、时间和 provenance                  | 这是原始记录、推断结果还是模型生成陈述？           |
| conceptualization/概念承诺     | Payment 是事件；Customer 与 Store 是参与对象 | 定义 class、property、domain、range、identity    | 为什么采用这种分类，而不是把 Payment 当属性或凭证？ |
| candidate knowledge/候选知识   | “租赁 R 对应付款 P”“金额可能构成收入事件”          | LLM、规则或对齐器生成带来源的 candidate axiom/mapping   | 支持证据是什么；与已有 ontology 是否冲突？     |
| justified canonical/经验证知识  | 批准的 RevenueEvent 定义和映射             | OWL 推理、SHACL、mapping test、CQ 与专家审批共同把关     | 通过的是语法、逻辑、数据形状、任务可用性还是业务真值？    |
| operational knowledge/行动知识 | 按批准定义计算门店收入并触发异常检查                 | 查询、策略和 agent action 只消费指定 ontology version | 行动权限、停止条件和结果反馈如何记录？            |

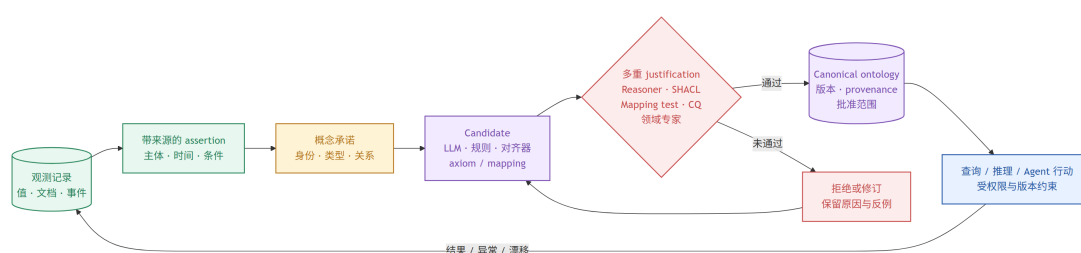


图 13.2: 本体驱动的认识循环：LLM 扩大候选，验证与治理缩小 canonical，行动反馈再修订概念

PROV-O 为 assertion 的来源、活动和责任主体提供可交换表示，SHACL 为 RDF 数据形状提供验证语言；它们与 OWL 的开放世界语义承担不同职责 (W3C Provenance Working Group 2013 年; W3C RDF Data Shapes Working Group 2017 年; W3C OWL Working Group, 不详)。一个系统可以在 OWL 下“不知道某关系是否存在”，同时用 SHACL 要求某类发布记录必须恰有一个金额字段。把这两者混为“本体正确性”会掩盖验证究竟证明了什么。

## ! 项目地图不是性能排行榜

项目仓库和论文的更新时间、数据集、模型、领域与指标不同。本章只比较知识生产环节和控制机制，不根据 GitHub star、单项 F1 或作者自报结果排列总体优劣。SovereignOntologyLLM 等探索性主张只作为设计假设，不作为已验证效果。

## 13.5 问题：页面写了关系，但系统不知道关系的类型

Level 7 的 rental.md 可以列出相邻表，却没有形式化说明“某个外键是 ObjectProperty，它的 domain 是 Rental，range 是 Customer”。Level 8 从 live schema graph 生成四类资产：

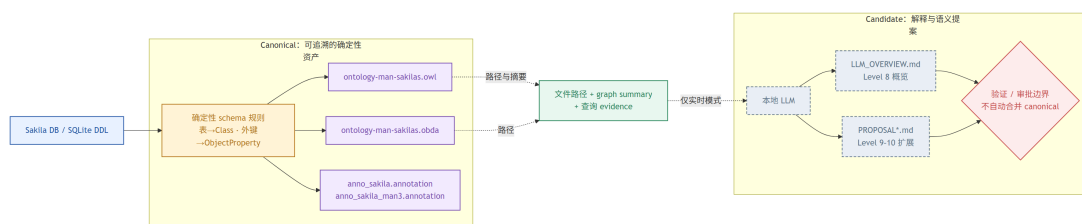


图 13.3: Level 8 的 canonical ontology 管线与 candidate 边界

OWL 提供类与属性的语义结构 (W3C OWL Working Group, 不详)；OBDA 文本把数据库表映射为 ontology class 实例；annotation 文件保留表与外键对应关系。文件命名参考用户指定的 ontology-driven extraction 项目，但 Sakila 内容来自本地数据库，不复制示例 ontology (Syamsiyah, 不详)。

## 13.6 实验：生成并解析 canonical OWL

```
from pathlib import Path
from rdflib import Graph, OWL, RDF
from assets.book_setup import run_offline

level_08 = run_offline(8)
artifact_paths = [Path(path) for path in level_08.artifacts]
owl_path = next(path for path in artifact_paths if path.suffix == ".owl")
ontology_graph = Graph().parse(owl_path)
{
 "artifacts": len(artifact_paths),
 "all_exist": all(path.is_file() for path in artifact_paths),
 "owl_classes": len(set(ontology_graph.subjects(RDF.type, OWL.Class))),
 "object_properties": len(
```

```

 set(ontology_graph.subjects(RDF.type, OWL.ObjectProperty))
),
 "rdf_triples": len(ontology_graph),
}

```

```

{'artifacts': 22,
 'all_exist': True,
 'owl_classes': 20,
 'object_properties': 21,
 'rdf_triples': 103}

```

离线共 21 个产物: 4 个 ontology/mapping 文件加 17 个 wiki 文件。代码为每张表生成一个 OWL.Class, 为每条外键生成一个 OWL.ObjectProperty, 并额外加入 RentalTransaction、RevenueEvent、CustomerValue、FilmDemand 与 amount datatype property。

```

obda_path = next(path for path in artifact_paths if path.suffix == ".obda")
print("\n".join(obda_path.read_text(encoding="utf-8").splitlines()[1:18]))

```

```

[PrefixDeclaration]
sakila: https://example.org/sakila#

[MappingDeclaration] @collection []
mappingId actor
target sakila:Actor/{rowid} a sakila:Actor .
source SELECT rowid, * FROM actor

mappingId address
target sakila:Address/{rowid} a sakila:Address .
source SELECT rowid, * FROM address

mappingId city
target sakila:City/{rowid} a sakila:City .
source SELECT rowid, * FROM city

mappingId category
target sakila:Category/{rowid} a sakila:Category .

```

## 13.7 Schema graph、ontology 与业务真理

生成合法 RDF/XML 只证明序列化结构可解析, 不证明语义正确。当前转换规则是教学型投影: 表变 class, 外键变 property。真实领域本体还需要回答:

- payment 是实体、事件还是属性集合？
- 同一客户跨门店的身份怎样定义？
- 收入、退款、税和贡献之间是什么概念关系？
- 哪些约束应该是 OWL axiom，哪些应该是 SHACL validation？

### ! 实现边界

当前 runtime 解析 OWL 类及属性的 domain/range，再对照外键映射查询关系数量，校验通过才发布语义知识页。收入计算复用 Level 6 的图驱动计划。这里没有完整 OWL reasoner、SPARQL endpoint、Ontop/OBDA engine 或 SHACL；前述完整生命周期仍是分析框架。

## 13.8 Canonical 与 candidate 必须分层

Level 8 的 canonical OWL 由 schema 规则确定性生成。实时模型收到从 OWL 解析出的允许关系词表，并返回类型化候选。宿主验证候选、查询真实映射证据，再发布 SEMANTIC\_KNOWLEDGE.md。

Level 9–10 从合成实验变量与修订目标生成独立候选 OWL 和知识页，明确标注合成范围，不能直接并入 canonical OWL。知识生产与知识发布之间保持验证边界。

## 13.9 评价

本层可选运行 DeepEval，评价回答是否相关、有证据并避免无依据因果确定性；它不会验证 OWL 一致性或 OBDA 可执行性。语义资产需要专门的 reasoner、shape 和 mapping tests。

## 13.10 结论与升级门槛

Level 8 让知识从“可阅读说明”升级为“可交换、可验证、可追溯的候选语义结构”。这些开源实现共同显示，真正的知识生产不是一键生成 OWL，而是需求、抽取、学习、对齐、验证、发布和修订组成的生命周期。适合跨系统共享概念、映射数据源或建立治理词汇。

然而更丰富的 ontology 仍无法回答“库存增加是否导致收入增加”。当系统必须先表达干预、混杂与识别缺口时，进入 Level 9。

## 13.11 章末练习

1. 为什么“每张表一个 class”可能制造贫血 ontology?
2. 给 amount 设计一个 SHACL 约束，但说明它解决不了哪类业务语义错误。
3. 生成文件能被 RDFLib 解析，与能被业务批准之间还缺哪些测试？

## Part IV

# 第四部 从问题到创新：从关联分析走向可证伪机制

## 14 Level 9：先问能不能识别，再问怎么优化

### 14.1 能力验收实验

用固定种子生成带混杂、已知干预效应的合成数据，对比朴素差异与分层调整。缺乏共同支持时拒绝估计；该效应不能外推为真实 Sakila 门店效应。

kungfu-agent experiment 9

一招一句：问题驱动不是更快给答案，而是先把处理、结果、混杂与缺失证据说清楚。

### 14.2 问题：门店 2 收入更高，能推出什么

前八层可以越来越可靠地查询“发生了什么”，却仍可能把门店差异写成干预建议。因果问题的关键不是画一张漂亮的 DAG，而是定义：

- treatment：希望改变什么；
- outcome：希望影响什么；
- confounders：哪些共同原因同时影响 treatment 与 outcome；
- identification：现有数据和假设是否足以把因果效应从观察分布中分离出来。

Level 9 用手写 DAG 强迫系统表达这些假设 (Pearl 2009 年)。

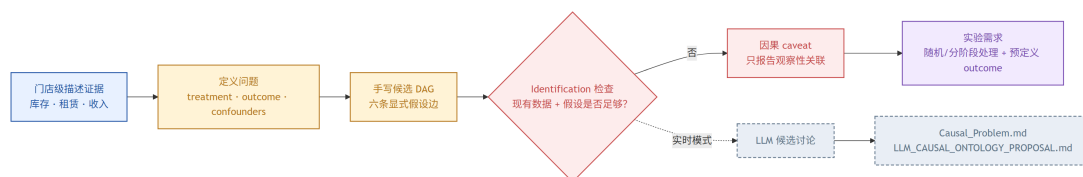


图 14.1: Level 9 从描述性差异到可识别性问题的管线

### 14.3 实验：扩大描述性证据，但不越过边界

```
from assets.book_setup import DB, run_offline

level_09 = run_offline(9)
DB.query(level_09.sql)
```

表 14.1: 库存广度、租赁和收入的门店级观察

| store_id | inventory_copies | rentals | customers | revenue  |
|----------|------------------|---------|-----------|----------|
| i64      | i64              | i64     | i64       | f64      |
| 1        | 2270             | 7923    | 599       | 33679.79 |
| 2        | 2311             | 8121    | 599       | 33726.77 |

门店 1 有 2270 个库存副本、7923 笔租赁和 33679.79 收入；门店 2 分别为 2311、8121 和 33726.77。相关方向与“更多库存可能带来更多交易”一致，但同时也与“需求更高的门店配置更多库存”一致。

## 14.4 DAG：把假设变成可争论对象

代码固定六条边：

```
causal_lab = level_09.metadata["causal_experiment"]
{
 "causal_dag": level_09.metadata["causal_dag"],
 "synthetic_known_effect": causal_lab["known_effect"],
 "synthetic_adjusted_effect": round(causal_lab["estimates"]["adjusted"], 3),
}
```

```
{'causal_dag': [('inventory_breadth', 'rentals'),
 ('rentals', 'revenue'),
 ('customer_demand', 'rentals'),
 ('pricing', 'revenue'),
 ('store_location', 'customer_demand'),
 ('store_location', 'inventory_breadth')],
'synthetic_known_effect': 2.0,
'synthetic_adjusted_effect': 2.007}
```

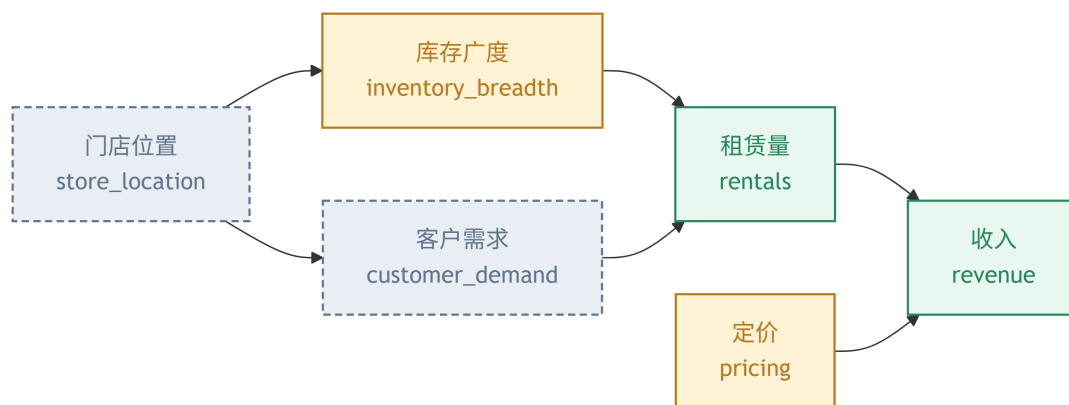


图 14.2: Level 9 的候选因果 DAG

store\_location 同时影响需求与库存广度，形成潜在后门路径；customer\_demand 影响租赁；定价直接影响收入。Sakila 缺少营销曝光、随机分配、竞争环境和丰富时间变化协变量，因此当前数据不足以识别“库存广度对收入的因果效应”。

## 14.5 本层产物的信任等级

两种模式均生成观察性上下文，以及明确标记的合成实验产物：

- synthetic\_experiment.json：种子、真值、DAG、估计与置信区间；
- Identified\_Synthetic\_Effect.md：合成实验知识；
- synthetic-causal.owl：从模拟变量生成的独立候选词表。

实时模型解释已计算的实验报告。候选词表标明 synthetic SCM 范围，不自动并入 canonical ontology。

### ⚠ 实现限制

真实 Sakila SQL 仍固定为门店观察对比。新增效应估计只针对已知结构的合成数据，按预先声明的 demand 混杂变量分层调整；它不自动发现调整集，也不证明真实门店的干预效果。95% 区间条件于观测到的分层权重。

## 14.6 从识别缺口到下一步实验

一个诚实的输出应分三段：

1. **关联**：门店 2 同时有略多库存、租赁和收入；
2. **假设**：位置、需求和定价可能混杂；
3. **下一步**：随机或分阶段调整库存，采集 treatment assignment、可用库存时间和预先定义 outcome。

## 14.7 结论与升级门槛

Level 9 适合任何可能诱发“相关即因果”决策的分析。它的成果是更好的问题、明确假设和实验需求。

当团队还需要从业务不变量推导可证伪机制，而不满足于让 LLM 自由头脑风暴时，进入 Level 10。

## 14.8 章末练习

1. 在 DAG 中加入 promotion，说明它可能打开或关闭哪些路径。
2. 定义库存实验的 treatment、unit、outcome 和 estimand。
3. 什么新增数据仍不足以识别因果效应？为什么？

## 15 Level 10: 从不变量推导可证伪创新

### 15.1 能力验收实验

按库存守恒枚举调拨，比较采购与维持基线。用每库存周期净贡献否证收入最高但成本过高的方案；平衡需求的阴性对照应返回没有支持的改善。结论限定于合成模拟器。

kungfu-agent experiment 10

一招一句：第一性原理不是“大胆想象”，而是从目标、不变量和约束推出能被数据推翻的机制。

### 15.2 问题：DAG 说明不知道什么，然后呢

Level 9 阻止了过度因果解释，但只列出识别缺口仍不能产生可执行学习。Level 10 把创新问题拆成五层：

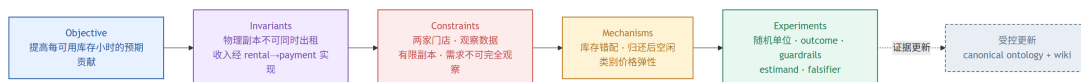


图 15.1: 从第一性目标到可证伪实验的五层推导

目标不是模糊的“提高收入”，而是“提高每个可用库存小时的预期贡献”。不变量包括实体库存不能同时服务两次租赁、收入通过关联 rental 的 payment 实现、需求不可完全观察。约束包括只有两家门店、历史观察数据、有限副本与客户体验。

### 15.3 实验：读取第一性原理结构

```

from assets.book_setup import run_offline

level_10 = run_offline(10)
principles = level_10.metadata["first_principles"]
principles

```

```
{'objective': 'contribution per inventory period, rather than revenue alone',
 'invariants': ['one copy serves at most one rental per period',
 'served rentals cannot exceed demand',
 'contribution = revenue - operating cost - intervention cost'],
 'constraints': ['two simulated stores',
 'nonnegative integer copies',
 'rebalancing conserves total copies',
 'explicit intervention costs'],
 'mechanisms': ['reallocate idle copies',
 'purchase additional copies',
 'retain baseline'],
 'experiments': ['evaluate all feasible transfers',
 'falsify revenue-only purchasing',
 'balanced-demand negative control']}
```

当前可执行实验比较调拨、采购与维持基线三个机制族。库存守恒推导出有限调拨候选；成本与需求假设决定每个候选的净贡献。提醒与定价实验属于后续研究扩展。

## 15.4 从机制到可证伪实验

以“库存错配”机制为例：

| 设计元素               | 一个可执行定义                               |
|--------------------|---------------------------------------|
| Randomization unit | film × 两店配对；周是重复观测窗口                  |
| Treatment          | 在配对内部，将一份库存从低预测缺货门店调往高预测缺货门店          |
| Control            | 匹配片目维持原库存分配                           |
| Primary proxy      | 配对层面的每总 copy-day 租赁量或收入               |
| Secondary outcomes | 分别报告供体店与受体店的缺货、租赁量和收入                 |
| Guardrails         | 供体店缺货、逾期、复租率、客户投诉                     |
| Estimand           | 调拨分配对配对层代理指标的 intention-to-treat 平均差异 |
| Falsifier          | 配对层产出没有改善，或任一供体护栏明显恶化                 |

先按历史需求和片目特征匹配合格影片，再在匹配组内把“调拨”或“不调拨”随机分配给完整的两店配对。这样，库存从供体移到受体造成的门店内干扰被纳入同一个实验单位。影片之间仍可能因替代效应互相干扰，因此还应预先检查类别层 spillover，必要时按类别成组随机。实验前冻结 outcome、样本窗口和停止规则，避免结果出来后重新定义成功。

### ！代理指标不是最终目标

Sakila 有 rental、return 与 payment，但没有完整库存成本、搬运成本、门店营业状态和明确的“可用库存小时”字段。因此这个实验只能推翻或支持“调拨会改善可观测吞吐或收入”的代理机制，不能证明“每可用库存小时贡献”提高。后者仍是设计

目标；在提出该结论前，必须新增可用库存小时、净贡献与调拨成本数据，并预先定义代理指标怎样映射到目标。

## 15.5 实现事实：Level 10 怎样继承 Level 9

Level 10 以离线模式复用 Level 9 的观察证据与合成因果实验，再枚举可行库存干预。每个候选都经过资源约束与净贡献模拟，并运行平衡需求阴性对照。实时模型解释已计算的结果，不替代评价器选择胜者。

两种模式均写出：

- hypothesis\_tests.json：候选、评价与阴性对照；
- Tested\_Principles.md：目标修订和实验知识；
- candidate-principles.owl：独立候选概念。

当前模拟器使用固定需求、显式成本和每周期每份库存最多服务一次的假设。它能证明有限候选空间内的比较结果，不能证明真实世界创新或部署收益；候选概念不自动合并 canonical OWL。

## 15.6 第一性原理与自由头脑风暴的差别

自由头脑风暴问“还能做什么”；第一性原理设计追问：

1. 哪些事实在任何方案下都必须成立？
2. 哪个约束使现有方案失败？
3. 哪个机制连接 intervention 与 outcome？
4. 什么结果会推翻该机制？
5. 失败后哪些 ontology/wiki 知识需要修正？

可证伪性让创新进入工程循环：假设不是需要维护的面子，而是需要尽快接受高质量检验的资产。

## 15.7 结论：最高层也没有获得更大权限

Level 10 的创新产物仍继承相同只读 SQL 边界。层级提升的是问题结构、知识与实验设计，不是数据库写权限。

真正进入生产下一步，不是 Level 11，而是把实验接入审批、随机化、监控和因果评估基础设施，并把证据反馈到 canonical ontology 与 wiki 的受控发布流程。

## 15.8 章末练习

1. 写出一个会推翻“减少周转空闲时间能提高收入”的观察结果。
2. 为什么收入增长不能成为价格实验的唯一成功指标？
3. 哪些新字段必须加入数据模型，才能计算“每可用库存小时贡献”？

## 16 结论：选择最低但足够的控制层

### 16.1 十层不是排行榜，而是选择矩阵

| Level | 控制者                      | 主要产物                | 适合场景    | 残余风险        |
|-------|--------------------------|---------------------|---------|-------------|
| 1     | Prompt + workflow        | state、SQL           | 固定单步分析  | 无自动修复       |
| 2     | 代码 skill                 | tool calls          | 单进程工具助手 | 工具选择错误      |
| 3     | protocol envelope        | trace               | 跨责任主体协作 | 只记录、不裁决     |
| 4     | OmniGenT + Codex harness | 类型化动作计划、evaluation  | 固定双角色分析 | 计划与评价器共享盲点  |
| 5     | evaluator + loop         | iteration history   | 可机器修复任务 | 指标投机、耗尽策略   |
| 6     | typed manifest + graph   | contract、join path  | 复杂关系查询  | 图不等于业务语义    |
| 7     | durable wiki             | Markdown knowledge  | 跨运行积累   | 漂移与来源治理     |
| 8     | ontology pipeline        | OWL/OBDA/annotation | 语义交换    | 合法结构不等于正确知识 |
| 9     | causal problem           | DAG、识别缺口            | 干预决策前分析 | 没有效果估计      |
| 10    | first principles         | 机制与实验               | 可证伪创新   | 尚无实验执行基础设施  |

## 16.2 控制权迁移图

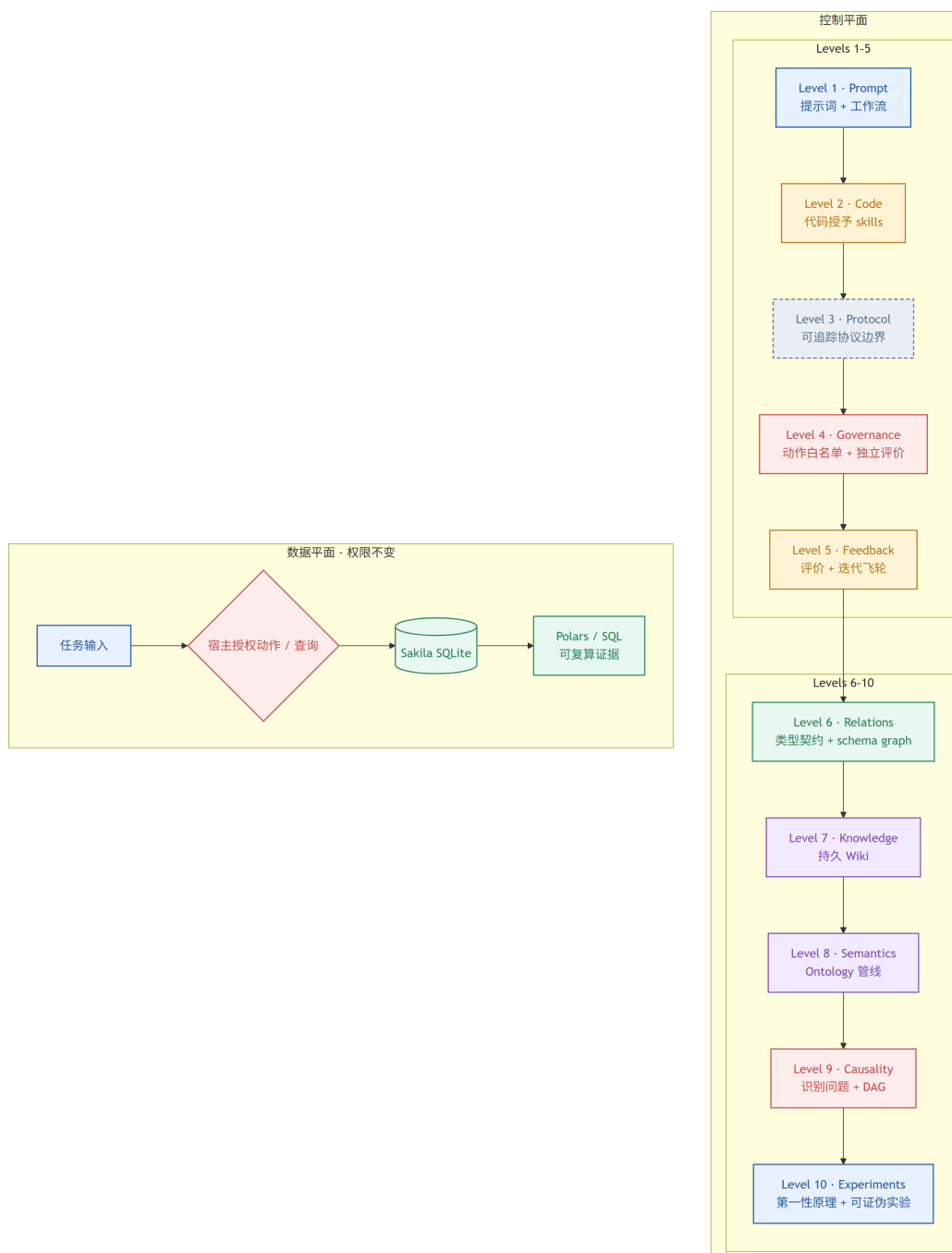


图 16.1: 十层控制面的累积迁移

这不是说每层都严格包含前层全部运行时，而是说需要管理的控制问题逐渐增多。选择层级时，先找当前最昂贵、最危险、最难发现的失败，再选择能直接控制它的最低层。

## 16.3 十条工程心法

1. **Prompt 是软约束**。权限必须在代码、sandbox 或数据库层执行；sandbox 的强制范围要按操作系统验证，不能仅凭配置名推断。
2. **Tool surface 就是 capability surface**。未暴露的能力，模型不应自行获得。
3. **协议标签不是协议实现**。明确哪些边界是真网络，哪些是教学 envelope。
4. **模型自评不是最终证据**。动作形状、技能来源与结果必须由宿主和独立基线检查。
5. **没有 evaluator 的 loop 只是重复**。没有停止条件的 loop 是风险放大器。
6. **Schema graph 不是 ontology**。最短路径也不是业务正确路径。
7. **持久知识需要 freshness 与 provenance**。写入文件只是治理的开始。
8. **生成合法 OWL 不等于产生正确知识**。Canonical 与 candidate 必须分层。
9. **先识别，再优化**。观察差异不能自动变成干预效果。
10. **创新必须可证伪**。不能说明如何失败的机制，还不是可检验方案。

## 16.4 一条建议学习路线

### 16.4.1 90 分钟快速课

- 运行 Marimo 的 Level 1、2、3，观察 answer、tool 和 trace 的变化；
- 运行 Level 4，检查委派证据、skill digest、白名单动作与十项评价；再运行 Level 5，读取两轮 evaluator feedback；
- 运行 Level 8，打开 OWL 与 OBDA；
- 运行 Level 9–10，把“结论”改写为“识别缺口 + 实验”。

### 16.4.2 一天工程工作坊

1. 为 SQL 守卫增加只读数据库账号与 timeout；
2. 给 Level 3 trace 增加 latency、fallback 和 error category；
3. 篡改 Level 4 的动作顺序、n 与 skill digest，验证三种情况都在宿主执行前 fail closed；
4. 为 Level 5 设计 exhaustion policy；
5. 为 wiki 加 schema hash；
6. 用 SHACL 或 reasoner 验证 ontology；
7. 写一份预注册的库存实验方案。

## 16.5 中英文同步维护规则

本书用 Quarto profiles 维护两套内容 (Quarto Project, 不详-c)：中文文件位于根目录和 chapters/，英文镜像位于 en/。任何概念、实现事实、限制、练习或引用的修改，都应在同一次变更中应用到两版。两种版本必须保持：

- 文件名与章节编号一一对应；
- heading anchor、cell label、引用键一致；
- Python 代码语义一致，只翻译输出说明与 caption；
- 表格行、限制条件和实现事实同步更新；
- 两本 PDF 都通过执行与文本抽取检查后才发布。

公共的 `_quarto.yml` 保存执行、格式和字体配置；`_quarto-zh.yml` 与 `_quarto-en.yml` 分别定义章节清单、语言和输出目录。Quarto profiles 会把 profile 配置与公共项目配置合并 (Quarto Project, 不详-c)。

16.6 术语速查

| 术语             | 本书中的含义                                              |
|----------------|-----------------------------------------------------|
| 数据平面           | 真正读取 Sakila 并返回证据的路径                                |
| 控制平面           | 决定计划、工具、协议、裁决、知识和实验的机制                              |
| Skill / tool   | 代码授予模型的可调用能力                                        |
| Envelope       | 带 sender、recipient、kind、payload 和 correlation 的消息外壳 |
| Harness        | 执行权限、评价、裁决与预算的治理边界                                  |
| Ralph loop     | 有 evaluator、反馈记忆和停止条件的改进循环                          |
| Schema graph   | 从外键恢复的表关系图                                          |
| Wiki           | 可读、可 diff 的持久知识快照                                   |
| Ontology       | 显式表达类、属性、关系与约束的语义模型                                 |
| Canonical      | 已通过确定性或治理流程批准的知识层                                   |
| Candidate      | 尚待验证、不可直接发布的模型提案                                    |
| Identification | 从观察分布和假设中恢复因果目标的条件                                  |
| Estimand       | 实验或分析要估计的精确定义量                                      |
| Guardrail      | 防止主指标改善掩盖严重副作用的约束指标                                 |

16.7 最后的读者测试

如果你能准确回答下面问题，就掌握了这套十层框架：

1. 为什么只读 SQL 助手通常停在 Level 2，而不需要 Level 8？
2. Level 2 与 Level 3 的差别是工具数量，还是边界显式化？
3. Level 4 的两个 Codex 子智能体能否直接读取数据库？计划、授权、执行与评价分别由谁负责？
4. 为什么 Ralph loop 必须同时有 evaluator 与最大迭代数？
5. Wiki 与把更多 schema 塞进 Prompt 有何不同？
6. 哪些 ontology 产物是 deterministic canonical，哪些只是 LLM proposal？
7. Sakila 能回答哪些描述问题，不能识别哪些因果问题？
8. 第一性原理设计与自由头脑风暴有什么区别？
9. 为什么升到 Level 10 后，数据库权限仍没有扩大？

## 参考资料

- Anthropic. 2023 年 a. 《Claude's Constitution》. 五月 9. <https://www.anthropic.com/news/claudes-constitution>.
- Anthropic. 2023 年 b. 《Introducing Claude》. 三月 14. <https://www.anthropic.com/news/introducing-claude>.
- Anthropic. 2024 年 a. 《Building Effective Agents》. 十二月 19. <https://www.anthropic.com/engineering/building-effective-agents>.
- Anthropic. 2024 年 b. 《Expanding Access to Claude for Government》. 六月 26. <https://www.anthropic.com/news/expanding-access-to-claude-for-government>.
- Anthropic. 2024 年 c. 《Introducing the Model Context Protocol》. 十一月 25. <https://www.anthropic.com/news/model-context-protocol>.
- Anthropic. 2025 年 a. 《Beyond Permission Prompts: Making Claude Code More Secure and Autonomous》. 十月 20. <https://www.anthropic.com/engineering/claude-code-sandboxing>.
- Anthropic. 2025 年 b. 《Claude Code: Best Practices for Agentic Coding》. 四月 18. <https://www.anthropic.com/engineering/claude-code-best-practices>.
- Anthropic. 2025 年 c. 《Effective Context Engineering for AI Agents》. 九月 29. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anthropic. 2025 年 d. 《Equipping Agents for the Real World with Agent Skills》. 十月 16. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.
- Anthropic. 2026 年 a. 《Demystifying Evals for AI Agents》. 一月 9. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>.
- Anthropic. 2026 年 b. 《Scaling Managed Agents: Decoupling the Brain from the Hands》. 四月 8. <https://www.anthropic.com/engineering/managed-agents>.
- Babaei Giglou, Hamed, Jennifer D'Souza, 和 Sören Auer. 2023 年. 《LLMs4OL: Large Language Models for Ontology Learning》. *The Semantic Web – ISWC 2023*, 408 ~ 27. [https://doi.org/10.1007/978-3-031-47240-4\\_22](https://doi.org/10.1007/978-3-031-47240-4_22).

- Babaei Giglou, Hamed, Jennifer D'Souza, Oliver Karras, 和 Sören Auer. 2025 年. 《OntoAligner: A Comprehensive Modular and Robust Python Toolkit for Ontology Alignment》. *arXiv preprint arXiv:2503.21902*, 网络首发. <https://doi.org/10.5281/zenodo.14533132>.
- Becker, Joel, Nate Rush, Elizabeth Barnes, 和 David Rein. 2025 年. 《Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity》. *arXiv preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>.
- Brown, Tom B., Benjamin Mann, Nick Ryder, 等. 2020 年. 《Language Models are Few-Shot Learners》. *Advances in Neural Information Processing Systems* 33: 1877 ~ 901. <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- Caufield, J. Harry, Harshad Hegde, Vincent Emonet, 等. 2024 年. 《Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES): A Method for Populating Knowledge Bases Using Zero-Shot Learning》. *Bioinformatics* 40 (3): btae104. <https://doi.org/10.1093/bioinformatics/btae104>.
- Chen, Zirui, Shijie Chen, Yuting Ning, 等. 2025 年. 《ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery》. *International Conference on Learning Representations*. [https://proceedings.iclr.cc/paper\\_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html).
- Chollet, François. 2019 年. 《On the Measure of Intelligence》. *arXiv preprint arXiv:1911.01547*. <https://arxiv.org/abs/1911.01547>.
- Claxton, Louis. 2026 年. 《The AI-Native SDLC Playbook》. Anthropic, 八月 21. <https://claude.com/blog/the-ai-native-sdlc-playbook>.
- Confident AI. 不详. 《DeepEval Documentation》. 见于 2026 年 8 月 30 日. <https://deepeval.com/docs/getting-started>.
- DeepSeek-AI. 2025 年. 《DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning》. *Nature* 645: 633 ~ 38. <https://doi.org/10.1038/s41586-025-09422-z>.
- Edge, Darren, Ha Trinh, Newman Cheng, 等. 2024 年. 《From Local to Global: A Graph RAG Approach to Query-Focused Summarization》. *arXiv preprint arXiv:2404.16130*. <https://arxiv.org/abs/2404.16130>.
- Gruber, Thomas R. 1993 年. 《A Translation Approach to Portable Ontology Specifications》. *Knowledge Acquisition* 5 (2): 199 ~ 220. <https://doi.org/10.1006/knac.1993.1008>.

- Guarino, Nicola. 1998 年. 《Formal Ontology and Information Systems》. *Formal Ontology in Information Systems: Proceedings of the First International Conference*, 3 ~ 15. <https://iaoa.org/fois/1998.html>.
- Gupta, Vipul, Shikhar Mohan, Lakshya Kumar, 等. 2026 年. 《HARNESS-LM: A Three-Phase Training Recipe for Harnessing SLMs in Sponsored Search Retrieval》. *arXiv preprint arXiv:2605.23572*, 网络首发. <https://doi.org/10.48550/arXiv.2605.23572>.
- He, Yuan, Jiaoyan Chen, Hang Dong, 等. 2023 年. 《DeepOnto: A Python Package for Ontology Engineering with Deep Learning》. *arXiv preprint arXiv:2307.03067*. <https://arxiv.org/abs/2307.03067>.
- Herre, Heinrich. 2013 年. 《Formal Ontology and the Foundation of Knowledge Organization》. *Knowledge Organization* 40 (5): 332 ~ 39. <https://doi.org/10.5771/0943-7444-2013-5-332>.
- Hong, Sirui, Mingchen Zhuge, Jiaqi Chen, 等. 2023 年. 《MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework》. *arXiv preprint arXiv:2308.00352*. <https://arxiv.org/abs/2308.00352>.
- Jimenez, Carlos E., John Yang, Alexander Wettig, 等. 2024 年. 《SWE-bench: Can Language Models Resolve Real-World GitHub Issues?》. *International Conference on Learning Representations*. [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html).
- Jones, David, Chris Snider, Aydin Nassehi, Jason Yon, 和 Ben Hicks. 2020 年. 《Characterising the Digital Twin: A Systematic Literature Review》. *CIRP Journal of Manufacturing Science and Technology* 29 (Part A): 36 ~ 52. <https://doi.org/10.1016/j.cirpj.2020.02.002>.
- Kaelbling, Leslie Pack, Michael L. Littman, 和 Anthony R. Cassandra. 1998 年. 《Planning and Acting in Partially Observable Stochastic Domains》. *Artificial Intelligence* 101 (1–2): 99 ~ 134. [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X).
- LangChain. 不详. 《LangGraph Overview》. 见于 2026 年 8 月 30 日. <https://docs.langchain.com/oss/python/langgraph/overview>.
- Lee, Yoonho, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, 和 Chelsea Finn. 2026 年. 《Meta-Harness: End-to-End Optimization of Model Harnesses》. *arXiv preprint arXiv:2603.28052*, 网络首发. <https://doi.org/10.48550/arXiv.2603.28052>.
- Legg, Shane, 和 Marcus Hutter. 2007 年. 《Universal Intelligence: A Definition of Machine Intelligence》. *Minds and Machines* 17 (4): 391 ~ 444. <https://doi.org/10.1007/s11023-007-9079-x>.

- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, 等. 2020 年. 《Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks》. *Advances in Neural Information Processing Systems* 33. <https://arxiv.org/abs/2005.11401>.
- Li, Jiayi, Daniel Garijo, 和 María Poveda-Villalón. 2026 年. 《Large Language Models for Ontology Engineering: A Systematic Literature Review》. *Semantic Web: Interoperability, Usability, Applicability* 17 (4): 1 ~ 45. <https://doi.org/10.1177/22104968261465514>.
- Lian, KJ, 和 Dan Krpata. 2024 年. 《How AWS Can Help Mission-Focused Organizations Comply with the White House National Security Memorandum on AI》. Amazon Web Services, 十一月 15. <https://aws.amazon.com/blogs/publicsector/how-aws-can-help-mission-focused-organizations-comply-with-the-white-house-national-security-memorandum-on-ai/>.
- Liu, Xiao, Hao Yu, Hanchen Zhang, 等. 2024 年. 《AgentBench: Evaluating LLMs as Agents》. *International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Lo, Andy, Albert Q. Jiang, Wenda Li, 和 Mateja Jamnik. 2024 年. 《End-to-End Ontology Learning with Large Language Models》. *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.5555/3737916.3740683>.
- Lopopolo, Ryan. 2026 年. 《Harness Engineering: Leveraging Codex in an Agent-First World》. 二月 11. <https://openai.com/index/harness-engineering/>.
- Luo, Xufang, Yuge Zhang, Zhiyuan He, 等. 2025 年. 《Agent Lightning: Train ANY AI Agents with Reinforcement Learning》. *arXiv preprint arXiv:2508.03680*, 网络首发. <https://doi.org/10.48550/arXiv.2508.03680>.
- Ma, Chang, Junlei Zhang, Zhihao Zhu, 等. 2024 年. 《AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents》. *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.52202/079017-2365>.
- Madaan, Aman, Niket Tandon, Prakhar Gupta, 等. 2023 年. 《Self-Refine: Iterative Refinement with Self-Feedback》. *Advances in Neural Information Processing Systems* 36. <https://arxiv.org/abs/2303.17651>.
- Meng, Qianyu, Yanan Wang, Liyi Chen, 等. 2026 年. *Agent Harness for Large Language Model Agents: A Survey*. Preprints.org. <https://www.preprints.org/manuscript/202604.0428/v1>.
- Morris, Meredith Ringel, Jascha Sohl-Dickstein, Noah Fiedel, 等. 2024 年. 《Levels of AGI for Operationalizing Progress on the Path to AGI》. *Proceedings of the 41st International Conference on Machine Learning*. <https://arxiv.org/abs/2311.02462>.

- Nakano, Reiichiro, Jacob Hilton, Suchir Balaji, 等. 2021 年. 《WebGPT: Browser-Assisted Question-Answering with Human Feedback》. *arXiv preprint arXiv:2112.09332*. <https://arxiv.org/abs/2112.09332>.
- NetworkX Developers. 不详. 《NetworkX Documentation》. 见于 2026 年 8 月 30 日. <https://networkx.org/documentation/stable/>.
- OpenAI. 不详-a. 《Codex SDK》. 见于 2026 年 8 月 30 日. <https://learn.chatgpt.com/docs/codex-sdk>.
- OpenAI. 不详-b. 《Models—OpenAI Agents SDK》. 见于 2026 年 8 月 30 日. <https://openai.github.io/openai-agents-python/models/>.
- OpenAI. 不详-c. 《Running Agents》. 见于 2026 年 8 月 30 日. [https://openai.github.io/openai-agents-python/running\\_agents/](https://openai.github.io/openai-agents-python/running_agents/).
- OpenAI. 不详-d. 《Tools—OpenAI Agents SDK》. 见于 2026 年 8 月 30 日. <https://openai.github.io/openai-agents-python/tools/>.
- OpenClaw Project. 2026 年 a. 《Agent Loop》. <https://docs.openclaw.ai/concepts/agent-loop>.
- OpenClaw Project. 2026 年 b. 《Memory Overview》. <https://docs.openclaw.ai/concepts/memory>.
- OpenClaw Project. 2026 年 c. 《Sandbox vs. Tool Policy vs. Elevated》. <https://docs.openclaw.ai/gateway/sandbox-vs-tool-policy-vs-elevated>.
- OpenClaw Project. 2026 年 d. 《Skills》. <https://docs.openclaw.ai/tools/skills>.
- OpenTelemetry Project. 2026 年. 《Context Propagation》. <https://opentelemetry.io/docs/concepts/context-propagation/>.
- Ouyang, Long, Jeffrey Wu, Xu Jiang, 等. 2022 年. 《Training Language Models to Follow Instructions with Human Feedback》. *Advances in Neural Information Processing Systems* 35: 27730 ~ 44. [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html).
- Packer, Charles, Sarah Wooders, Kevin Lin, 等. 2023 年. 《MemGPT: Towards LLMs as Operating Systems》. *arXiv preprint arXiv:2310.08560*. <https://arxiv.org/abs/2310.08560>.
- Palantir Technologies. 2024 年. 《Anthropic and Palantir Partner to Bring Claude AI Models to AWS for U.S. Government Intelligence and Defense Operations》. 十一月 7. <https://invest>

[ors.palantir.com/news-details/2024/Anthropic-and-Palantir-Partner-to-Bring-Claude-AI-Models-to-AWS-for-U.S.-Government-Intelligence-and-Defense-Operations/](https://www.palantir.com/news-details/2024/Anthropic-and-Palantir-Partner-to-Bring-Claude-AI-Models-to-AWS-for-U.S.-Government-Intelligence-and-Defense-Operations/).

Palantir Technologies. 2026 年. 《Foundry Platform Summary for LLMs》. 七月 2. <https://www.palantir.com/docs/foundry/getting-started/foundry-platform-summary-llm>.

Pallets Projects. 不详. 《Jinja Documentation》. 见于 2026 年 8 月 30 日. <https://jinja.palletsprojects.com/>.

Pan, Linyue, Lexiao Zou, Shuo Guo, Jingchen Ni, 和 Hai-Tao Zheng. 2026 年. 《Natural-Language Agent Harnesses》. *arXiv preprint arXiv:2603.25723*, 网络首发. <https://doi.org/10.48550/arXiv.2603.25723>.

Park, Joon Sung, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, 和 Michael S. Bernstein. 2023 年. 《Generative Agents: Interactive Simulacra of Human Behavior》. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. <https://doi.org/10.1145/3586183.3606763>.

Pearl, Judea. 2009 年. *Causality: Models, Reasoning, and Inference*. 2 版. Cambridge University Press.

Peng, Sida, Eirini Kalliamvakou, Peter Cihon, 和 Mert Demirer. 2023 年. 《The Impact of AI on Developer Productivity: Evidence from GitHub Copilot》. *arXiv preprint arXiv:2302.06590*. <https://arxiv.org/abs/2302.06590>.

Pydantic. 不详. 《Pydantic Documentation》. 见于 2026 年 8 月 30 日. <https://docs.pydantic.dev/>.

Quarto Project. 不详-a. 《Creating a Book》. 见于 2026 年 8 月 30 日. <https://quarto.org/docs/books/>.

Quarto Project. 不详-b. 《PDF Basics》. 见于 2026 年 8 月 30 日. <https://quarto.org/docs/output-formats/pdf-basics.html>.

Quarto Project. 不详-c. 《Project Profiles》. 见于 2026 年 8 月 30 日. <https://quarto.org/docs/projects/profiles.html>.

Quarto Project. 不详-d. 《Tutorial: Hello, Quarto—Jupyter》. 见于 2026 年 8 月 30 日. <https://quarto.org/docs/get-started/hello/jupyter.html>.

- Rajasekaran, Prithvi. 2026 年. 《Harness Design for Long-Running Application Development》. Anthropic, 三月 24. <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- Rasmussen, Preston, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, 和 Daniel Chalef. 2025 年. 《Zep: A Temporal Knowledge Graph Architecture for Agent Memory》. *arXiv preprint arXiv:2501.13956*. <https://arxiv.org/abs/2501.13956>.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, 等. 2023 年. 《Toolformer: Language Models Can Teach Themselves to Use Tools》. *Advances in Neural Information Processing Systems* 36. <https://doi.org/10.52202/075280-2997>.
- Sculley, D., Gary Holt, Daniel Golovin, 等. 2015 年. 《Hidden Technical Debt in Machine Learning Systems》. *Advances in Neural Information Processing Systems* 28. <https://proceedings.neurips.cc/paper/2015/hash/86df7dcfd896fcac2674f757a2463eba-Abstract.html>.
- Shen, Yongliang, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, 和 Yueting Zhuang. 2023 年. 《HuggingGPT: Solving AI Tasks with ChatGPT and Its Friends in Hugging Face》. *arXiv preprint arXiv:2303.17580*. <https://arxiv.org/abs/2303.17580>.
- Shimizu, Cogan, 和 Pascal Hitzler. 2025 年. 《Accelerating Knowledge Graph and Ontology Engineering with Large Language Models》. *Journal of Web Semantics*, 100862. <https://doi.org/10.1016/j.websem.2025.100862>.
- Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, 和 Shunyu Yao. 2023 年. 《Reflexion: Language Agents with Verbal Reinforcement Learning》. *Advances in Neural Information Processing Systems* 36. <https://doi.org/10.52202/075280-0377>.
- Sigelman, Benjamin H., Luiz André Barroso, Mike Burrows, 等. 2010 年. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google. <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>.
- Silver, David, Thomas Hubert, Julian Schrittwieser, 等. 2018 年. 《A General Reinforcement Learning Algorithm That Masters Chess, Shogi, and Go Through Self-Play》. *Science* 362 (6419): 1140 ~ 44. <https://doi.org/10.1126/science.aar6404>.
- Syamsiyah, Alifah. 不详. 《Ontology-driven Extraction Example》. 见于 2026 年 8 月 30 日. <https://github.com/alifahsyamsiyah/ontology-driven-extraction>.
- W3C OWL Working Group. 不详. 《OWL 2 Web Ontology Language Document Overview》. 见于 2026 年 8 月 30 日. <https://www.w3.org/TR/owl2-overview/>.

- W3C Provenance Working Group. 2013 年. 《PROV Overview》. 四月 30. <https://www.w3.org/TR/prov-overview/>.
- W3C RDF Data Shapes Working Group. 2017 年. 《Shapes Constraint Language (SHACL)》. 七月 20. <https://www.w3.org/TR/shacl/>.
- Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, 等. 2024 年. 《Voyager: An Open-Ended Embodied Agent with Large Language Models》. *Transactions on Machine Learning Research*. <https://openreview.net/forum?id=ehfRiF0R3a>.
- Wei, Jason, Xuezhi Wang, Dale Schuurmans, 等. 2022 年. 《Chain-of-Thought Prompting Elicits Reasoning in Large Language Models》. *Advances in Neural Information Processing Systems* 35. <https://arxiv.org/abs/2201.11903>.
- Wu, Qingyun, Gagan Bansal, Jieyu Zhang, 等. 2023 年. 《AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation》. *arXiv preprint arXiv:2308.08155*. <https://arxiv.org/abs/2308.08155>.
- Xia, Peng, Jianwen Chen, Hanyang Wang, 等. 2026 年. 《SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning》. *arXiv preprint arXiv:2602.08234*, 网络首发. <https://doi.org/10.48550/arXiv.2602.08234>.
- Xie, Tianbao, Danyang Zhang, Jixuan Chen, 等. 2024 年. 《OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments》. *Advances in Neural Information Processing Systems* 37. <https://doi.org/10.52202/079017-1650>.
- Yang, Chao, Yuan Zheng, Yuan Hua, Riku Ala-Laurinaho, Udayanto Dwi Atmojo, 和 Kari Tammi. 2024 年. 《Knowledge-Enhanced Digital Twin for Industrial Production Process》. 2024 IEEE 22nd International Conference on Industrial Informatics, 94 ~ 99. <https://doi.org/10.1109/INDIN58382.2024.10774266>.
- Yang, John, Carlos E. Jimenez, Alexander Wettig, 等. 2024 年. 《SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering》. *arXiv preprint arXiv:2405.15793*. <https://arxiv.org/abs/2405.15793>.
- Yao, Shunyu, Howard Chen, John Yang, 和 Karthik Narasimhan. 2022 年. 《WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents》. *Advances in Neural Information Processing Systems* 35. [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/82ad13ec01f9fe44c01cb91814fd7b8c-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/82ad13ec01f9fe44c01cb91814fd7b8c-Abstract-Conference.html).

- Yao, Shunyu, Noah Shinn, Pedram Razavi, 和 Karthik Narasimhan. 2025 年. 《 $\tau$ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains》. *International Conference on Learning Representations*. <https://openreview.net/forum?id=roNSXZpUDN>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, 等. 2023 年. 《ReAct: Synergizing Reasoning and Acting in Language Models》. *International Conference on Learning Representations*. [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X).
- Yehudai, Asaf, Lilach Eden, Alan Li, 等. 2025 年. 《Survey on Evaluation of LLM-based Agents》. *arXiv preprint arXiv:2503.16416*, 网络首发. <https://doi.org/10.48550/arXiv.2503.16416>.
- Young, Justin. 2025 年. 《Effective Harnesses for Long-Running Agents》. Anthropic, 十一月 26. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>.
- Zhou, Chenyu, Huacan Chai, Wenteng Chen, 等. 2026 年. 《Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering》. *arXiv preprint arXiv:2604.08224*, 网络首发. <https://doi.org/10.48550/arXiv.2604.08224>.
- Zhou, Shuyan, Frank F. Xu, Hao Zhu, 等. 2024 年. 《WebArena: A Realistic Web Environment for Building Autonomous Agents》. *International Conference on Learning Representations*. [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html).